

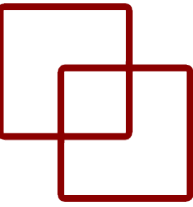
Fundamentos de Deep Learning I



LAB CTIC UNI

Dr. Manuel Castillo-Cara
Deep Learning con Python y Keras
Web: www.smartcityperu.org

Índice

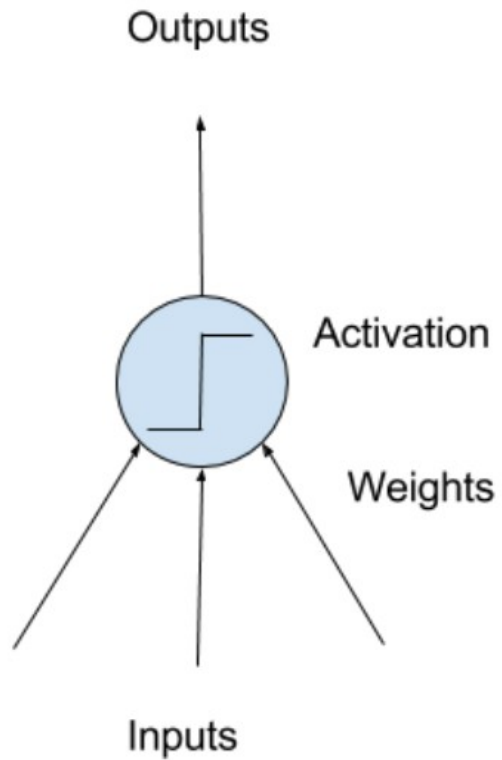
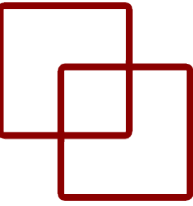


- Background
- Neurona
- Simulación
- Perceptron Multicapa
- Cómo opera el MLP
- Backpropagation
- Veamos una simulación

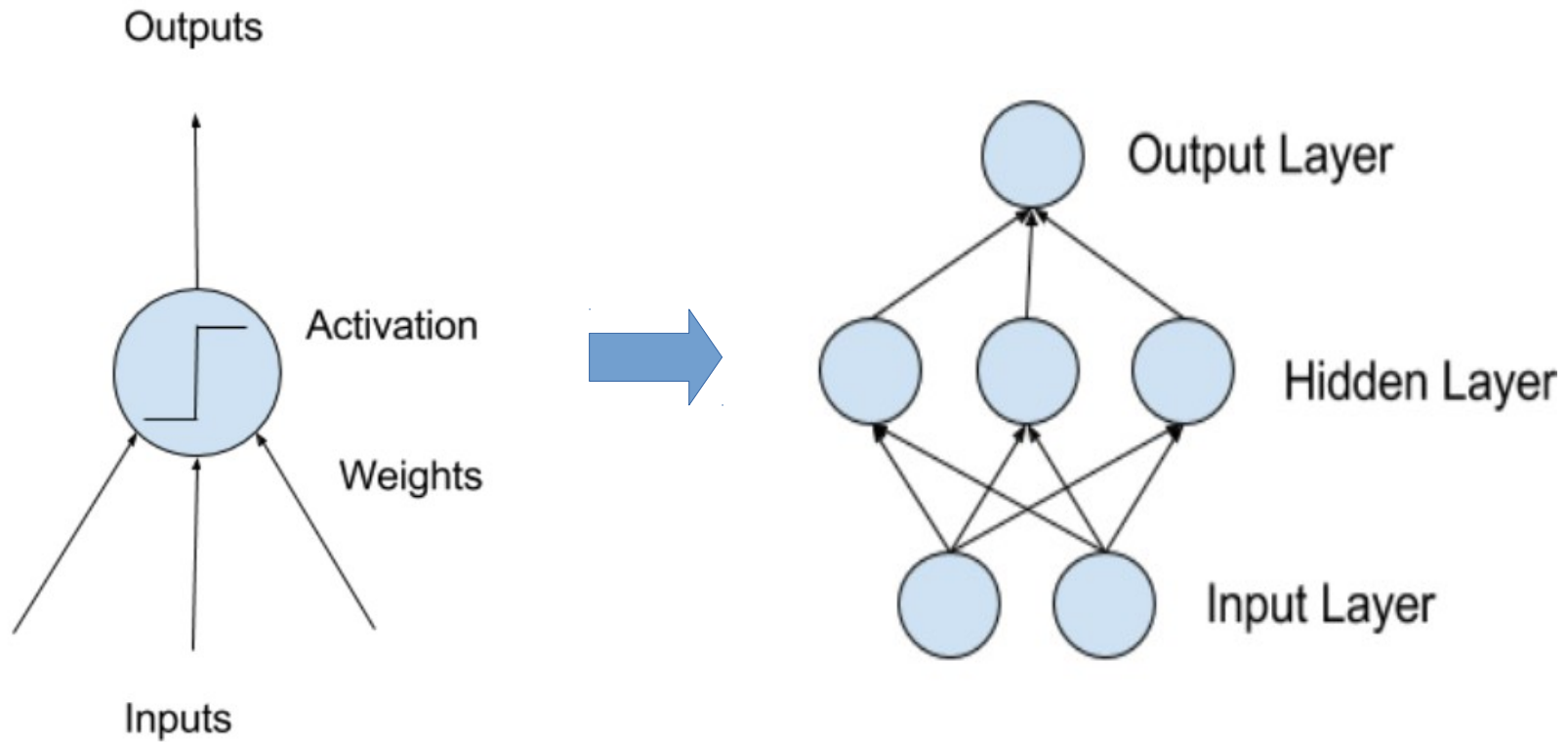
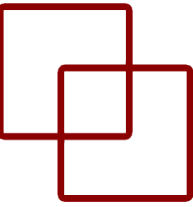


Background

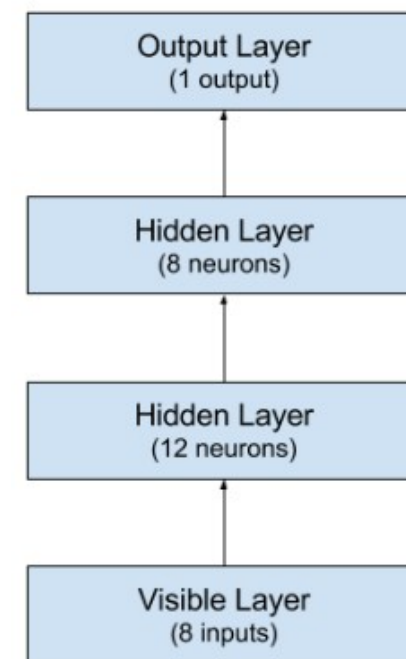
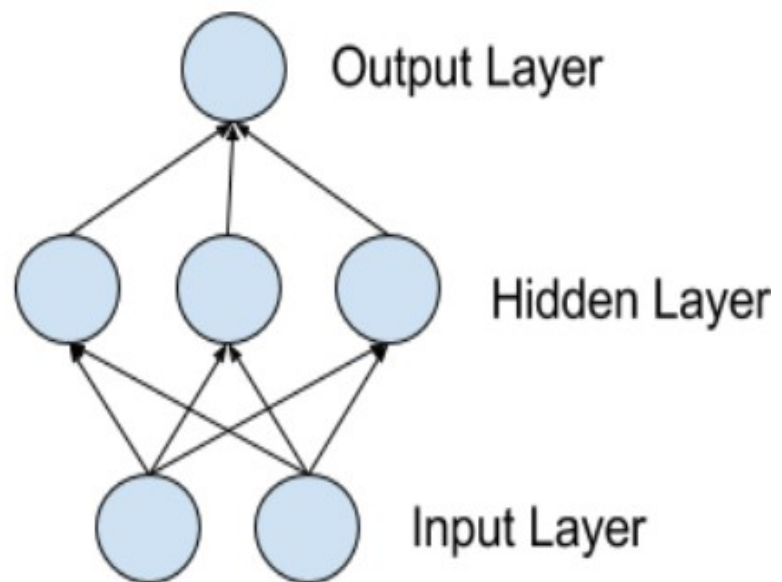
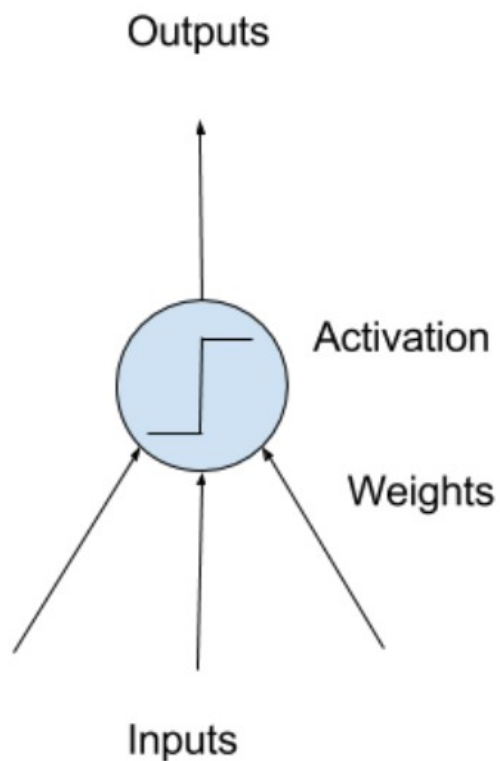
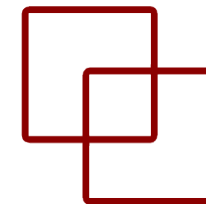
0. Background



0. Background



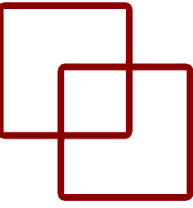
0. Background



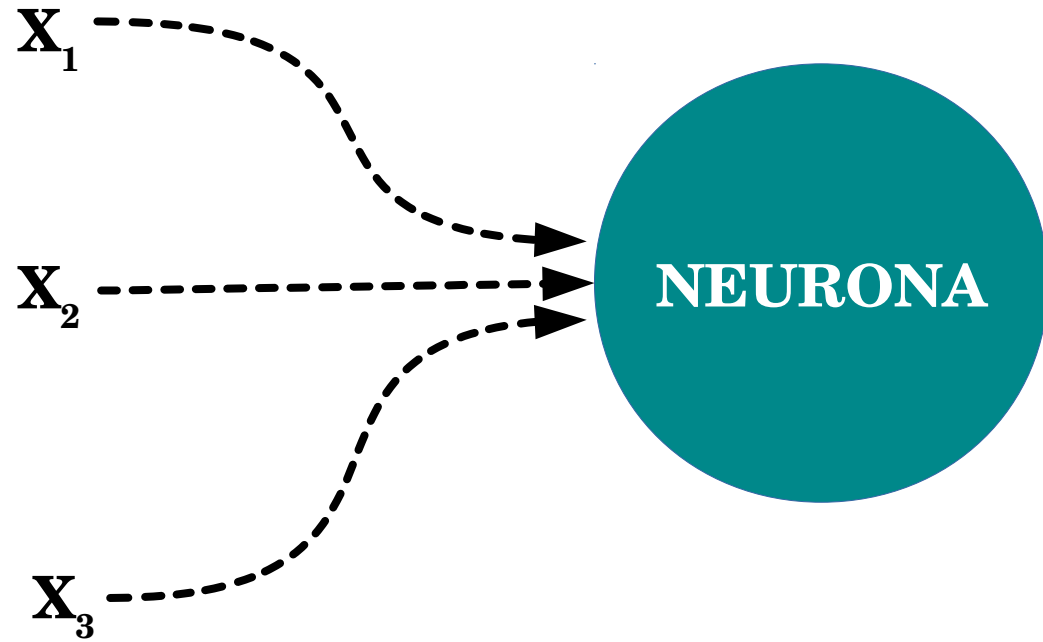
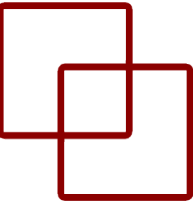


Neurona

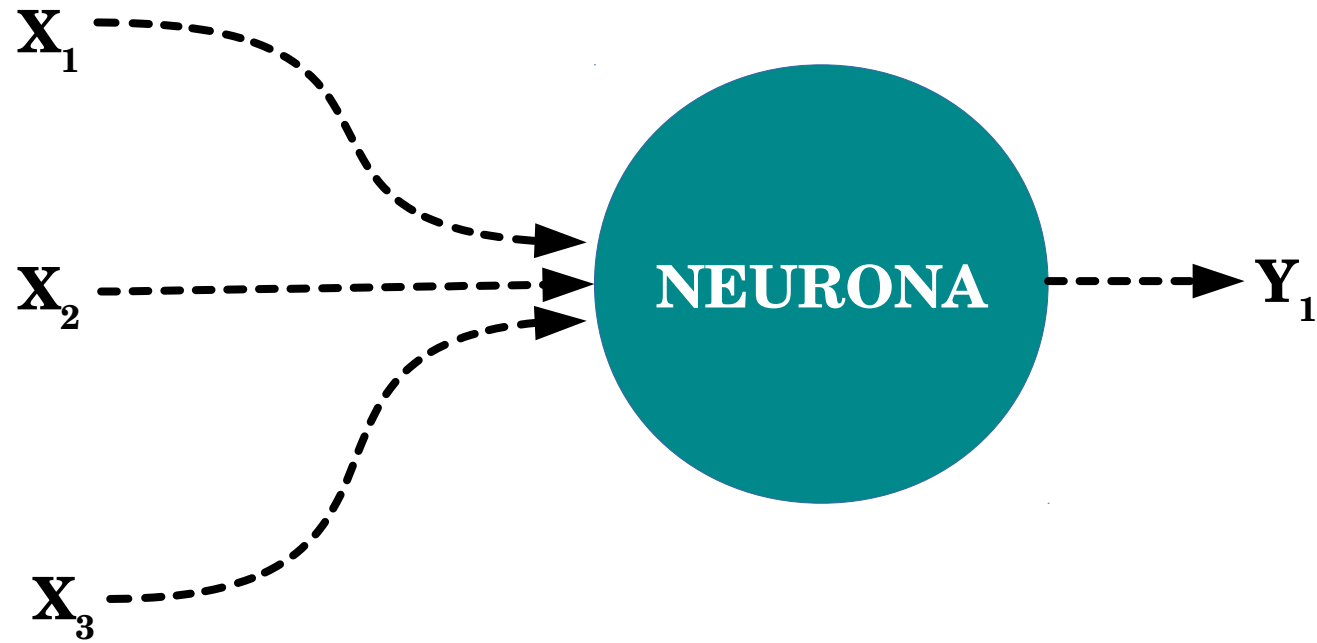
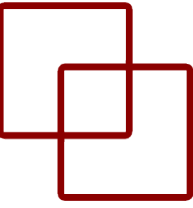
1. Neurona



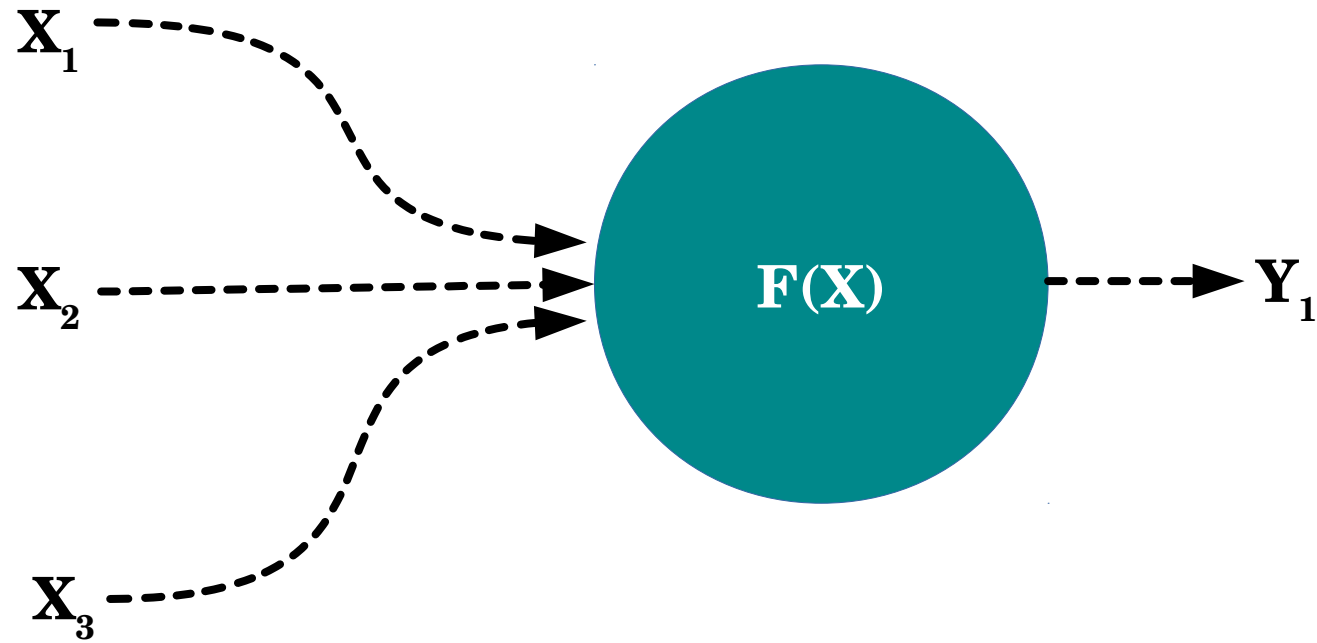
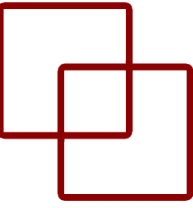
1. Neurona



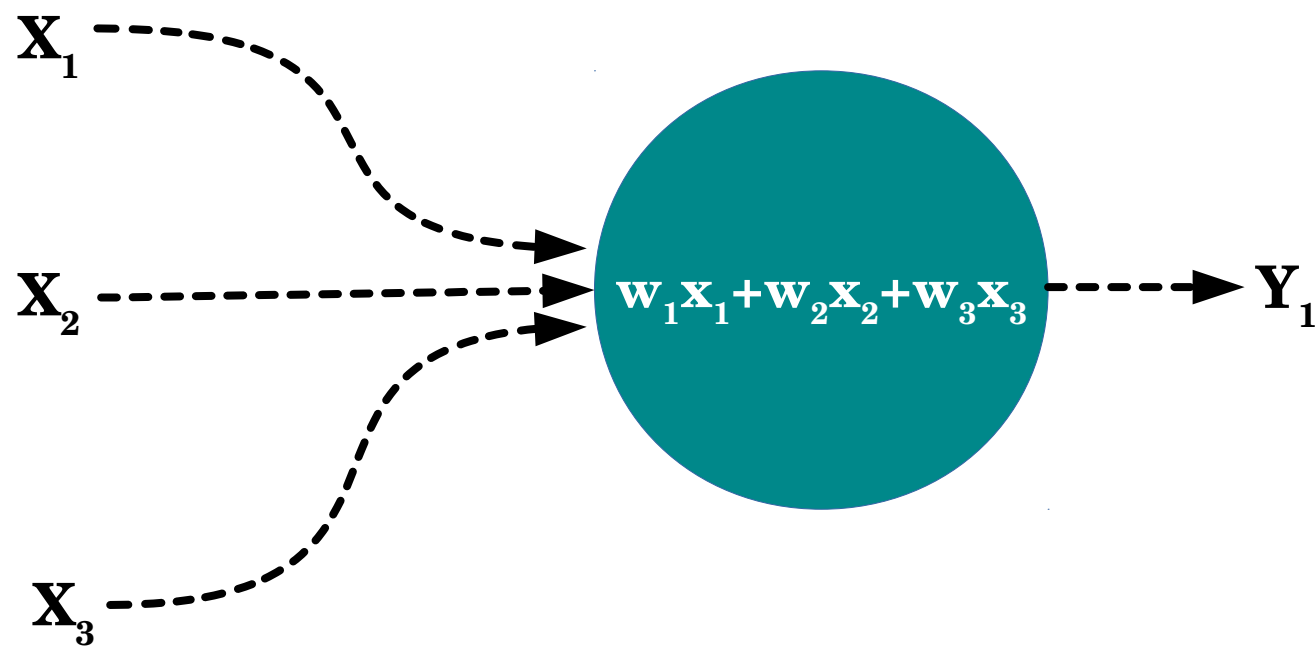
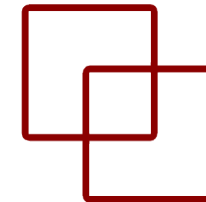
1. Neurona



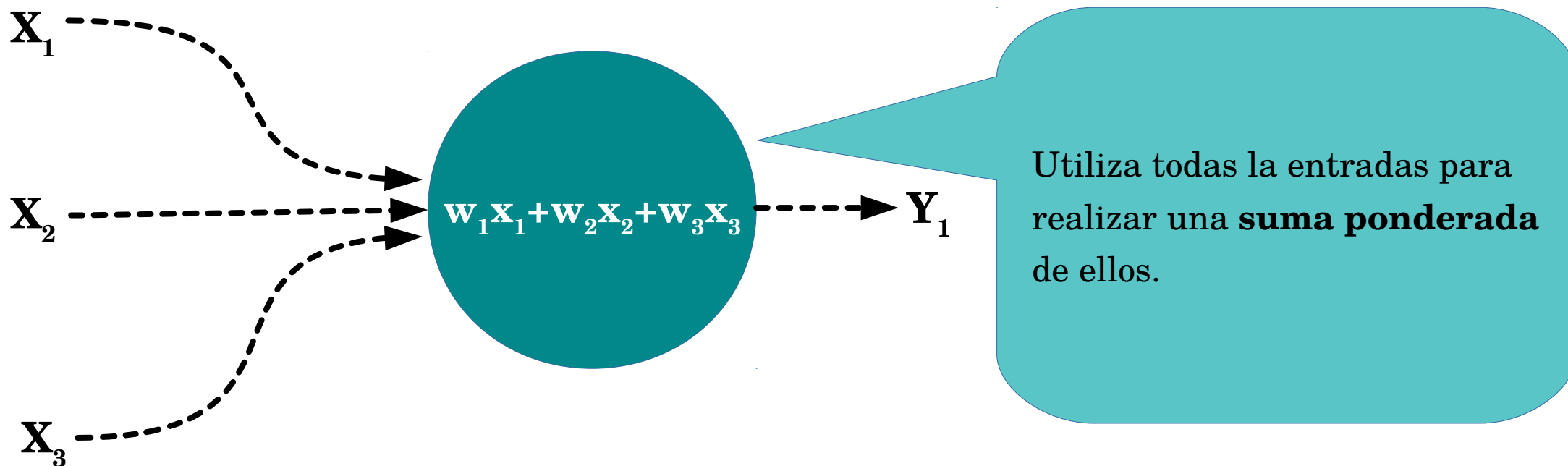
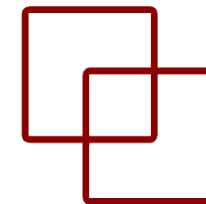
1. Neurona



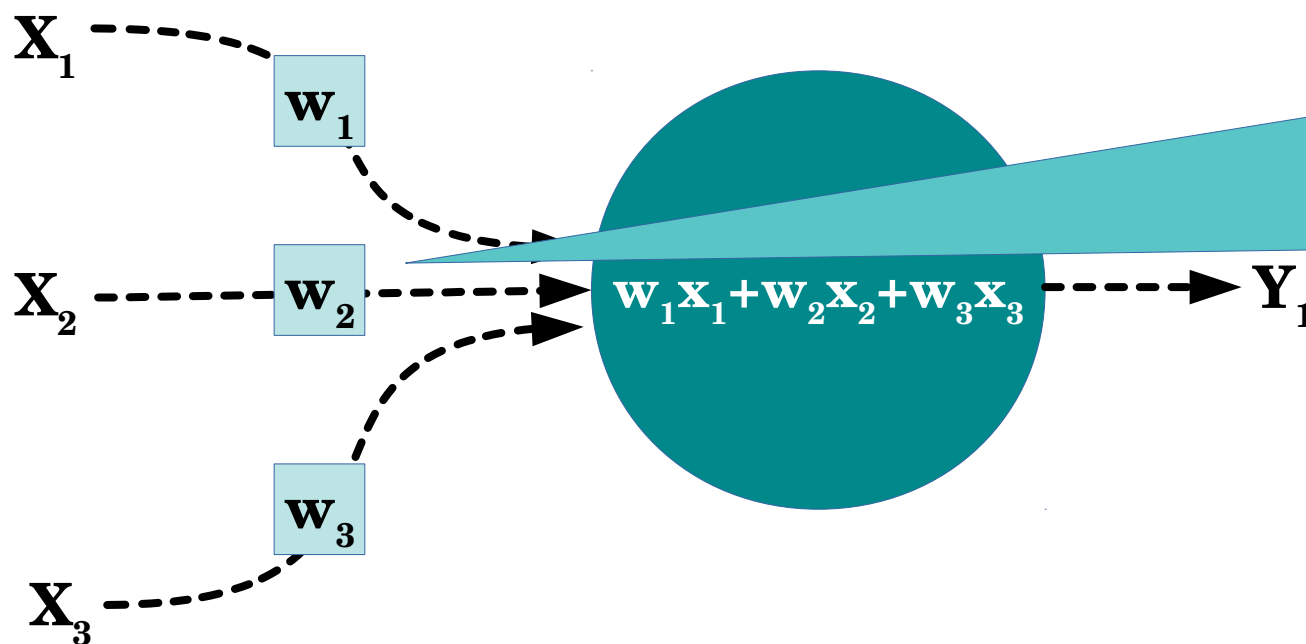
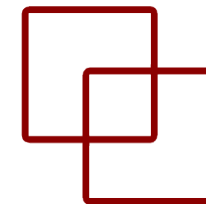
1. Neurona



1. Neurona

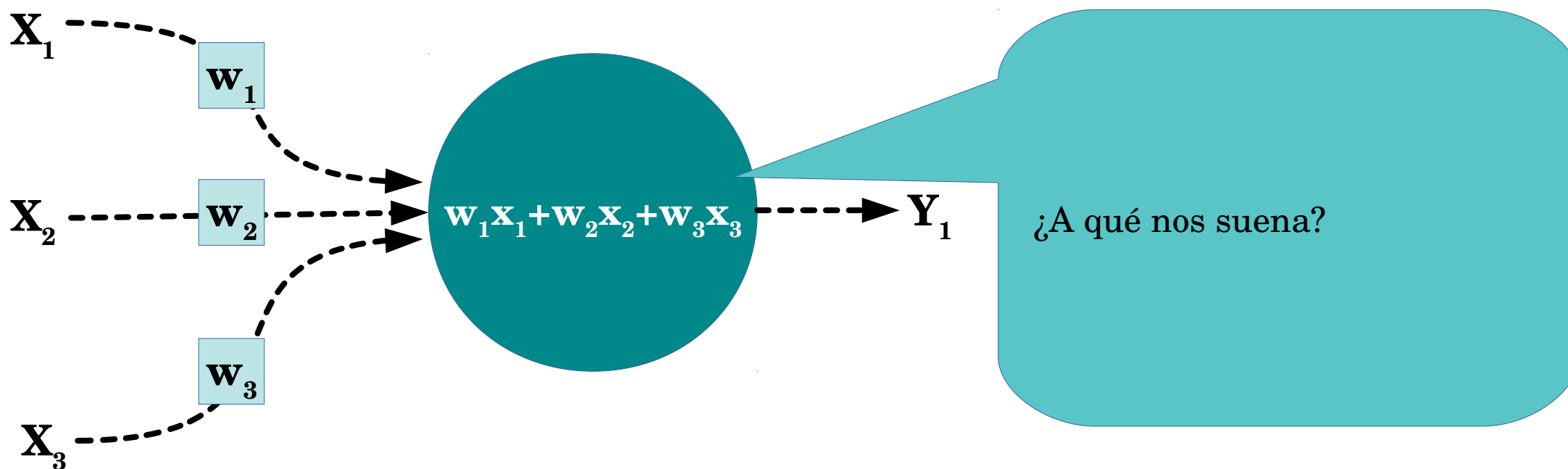
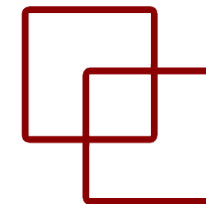


1. Neurona

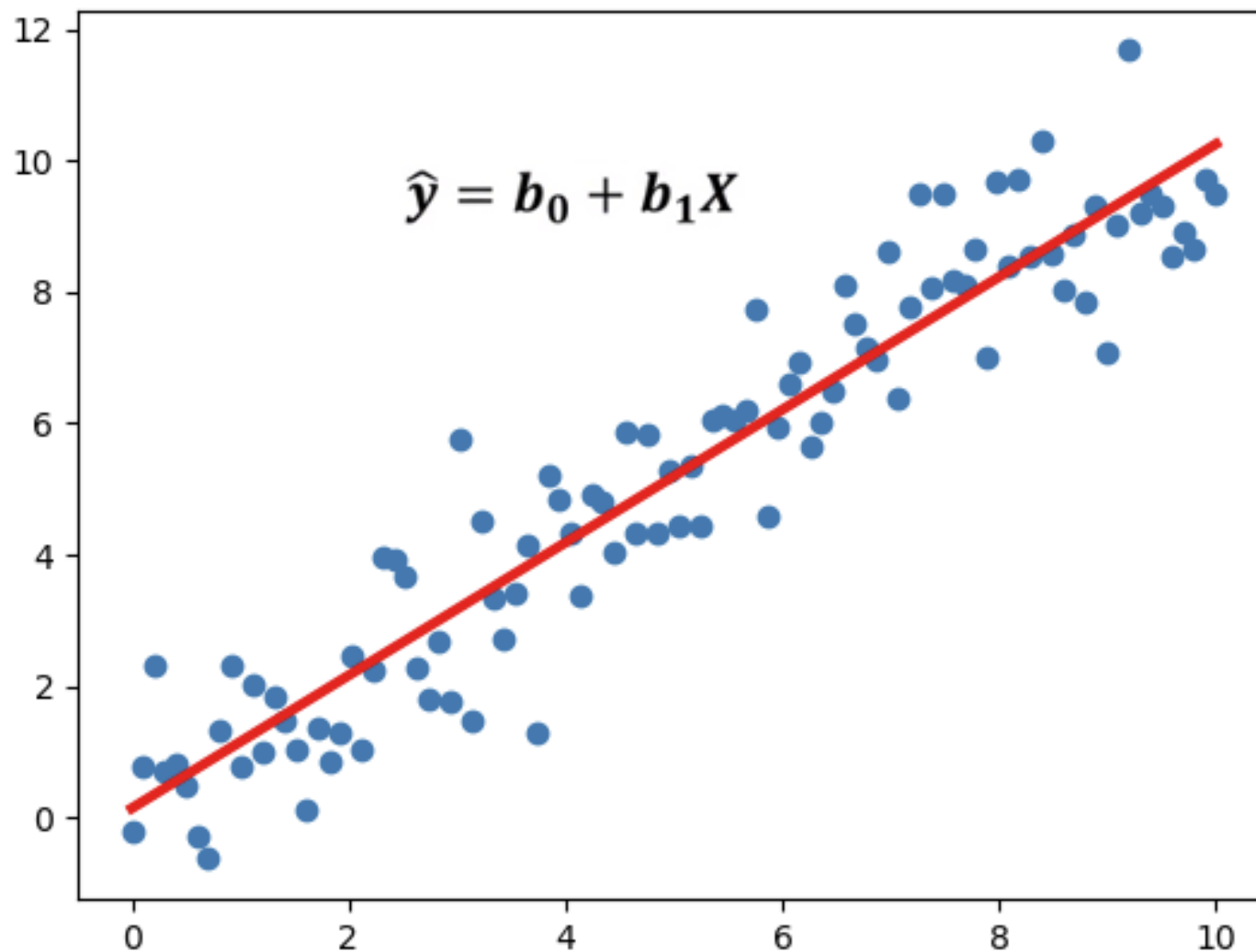
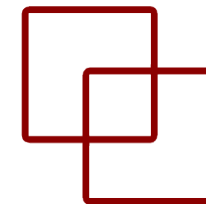


Hay que tener en cuenta que la suma viene dada por el **peso** (w) que se le asigna a cada una de las entradas

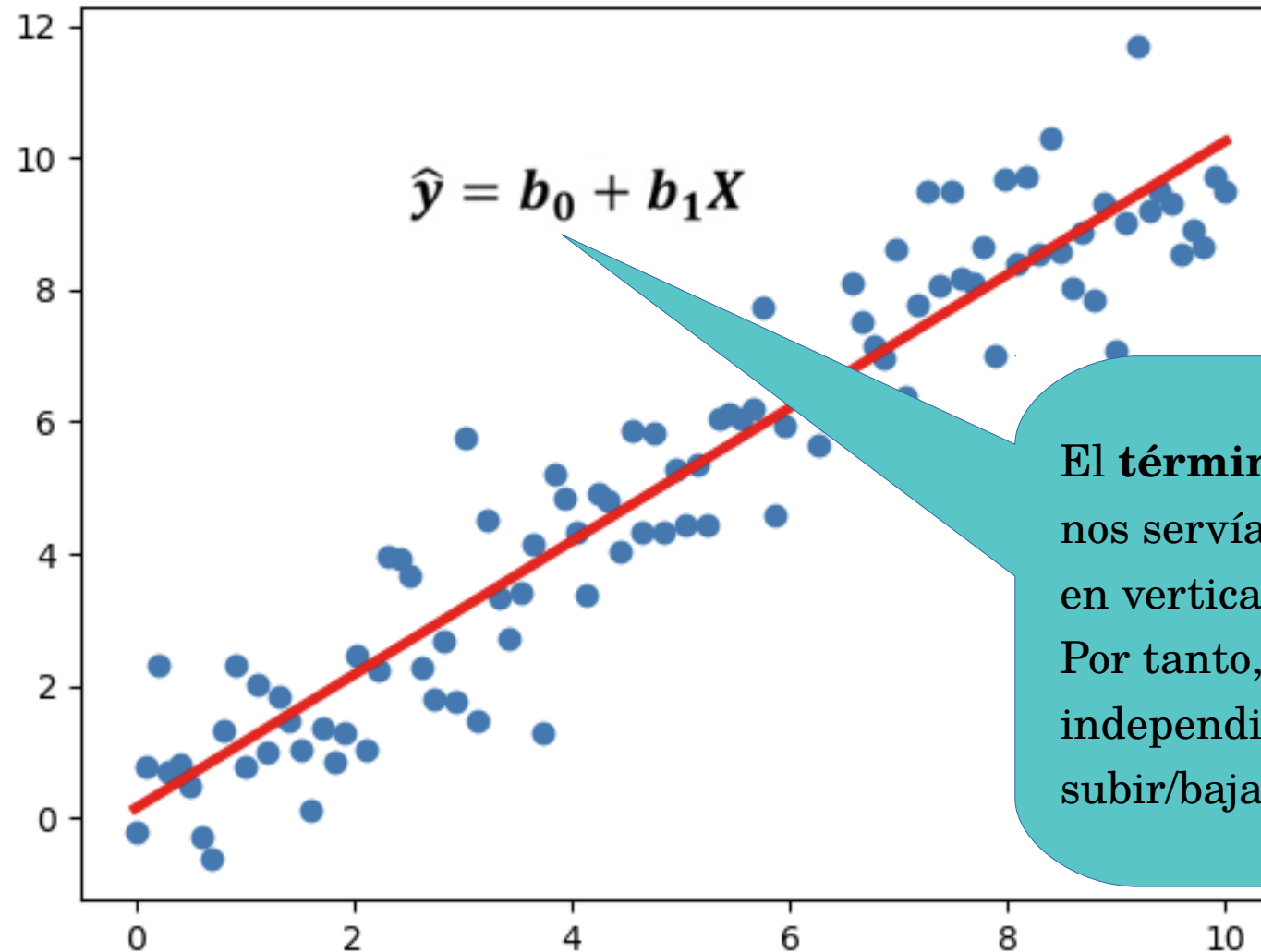
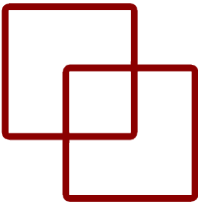
1. Neurona



2. Regresión lineal

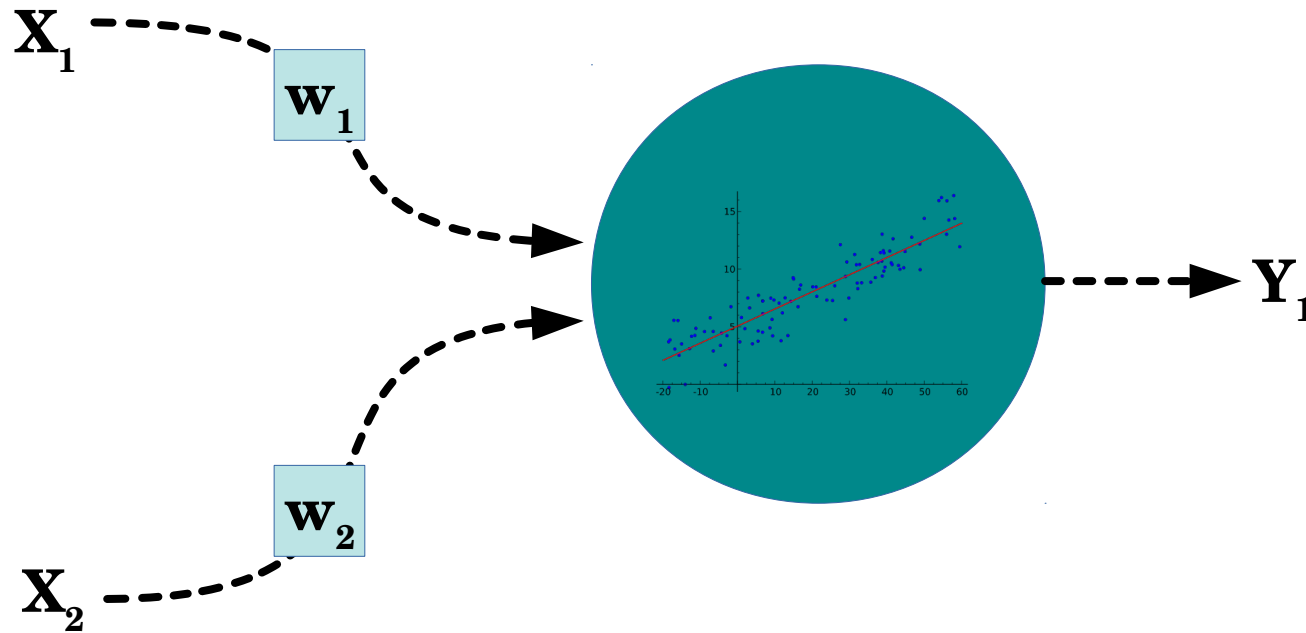
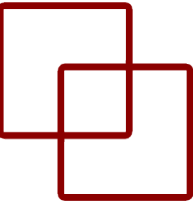


2. Regresión lineal



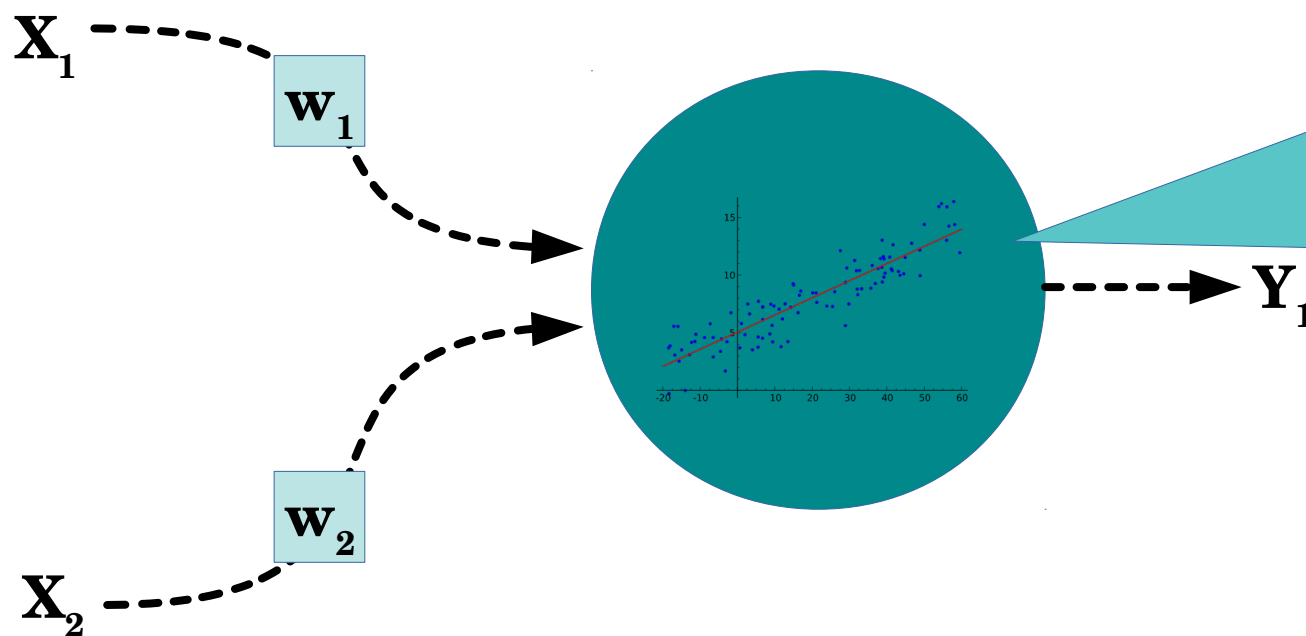
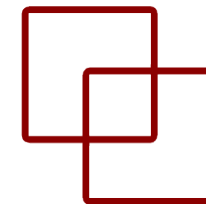
El **término independiente** nos servía para mover la recta en vertical, i.e., en el eje y. Por tanto, el término independiente nos servía para subir/bajar la recta

3. Función de la NN



$$y = w_1 x_1 + w_2 x_2$$

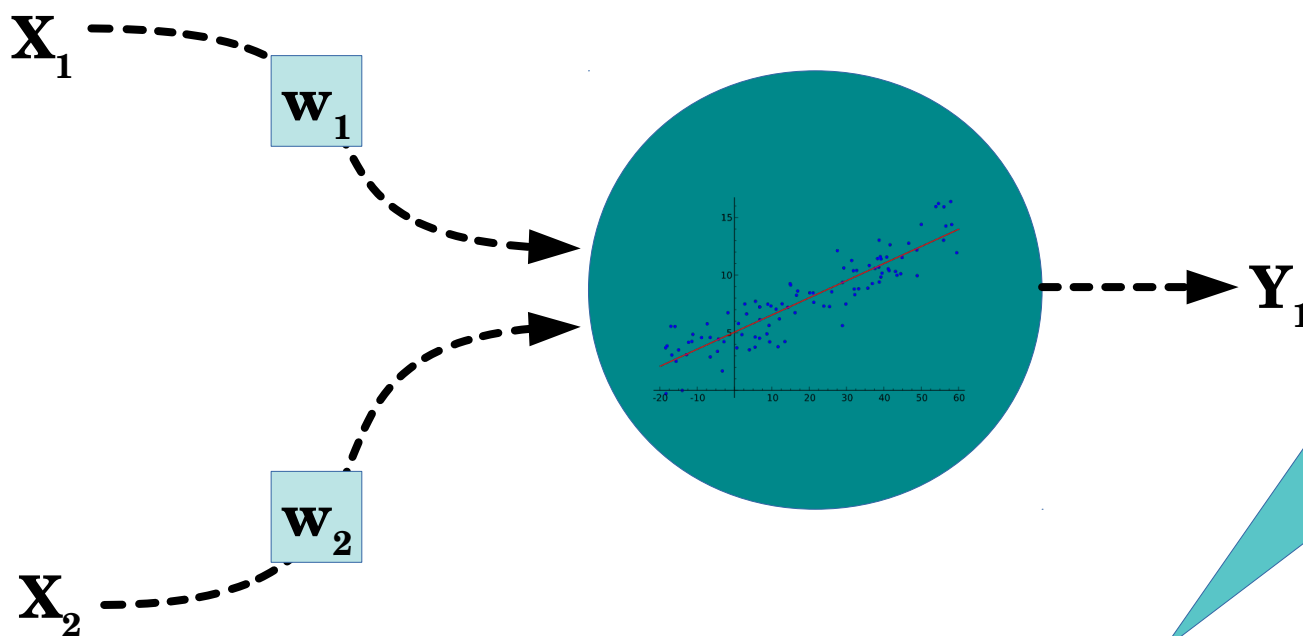
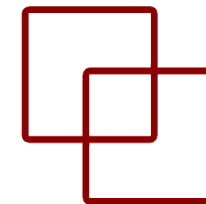
3. Función de la NN



Podemos verlo como una función de regresión lineal en el que una recta o hiperplano se ajusta a nuestros datos de entrada

$$y = w_1x_1 + w_2x_2$$

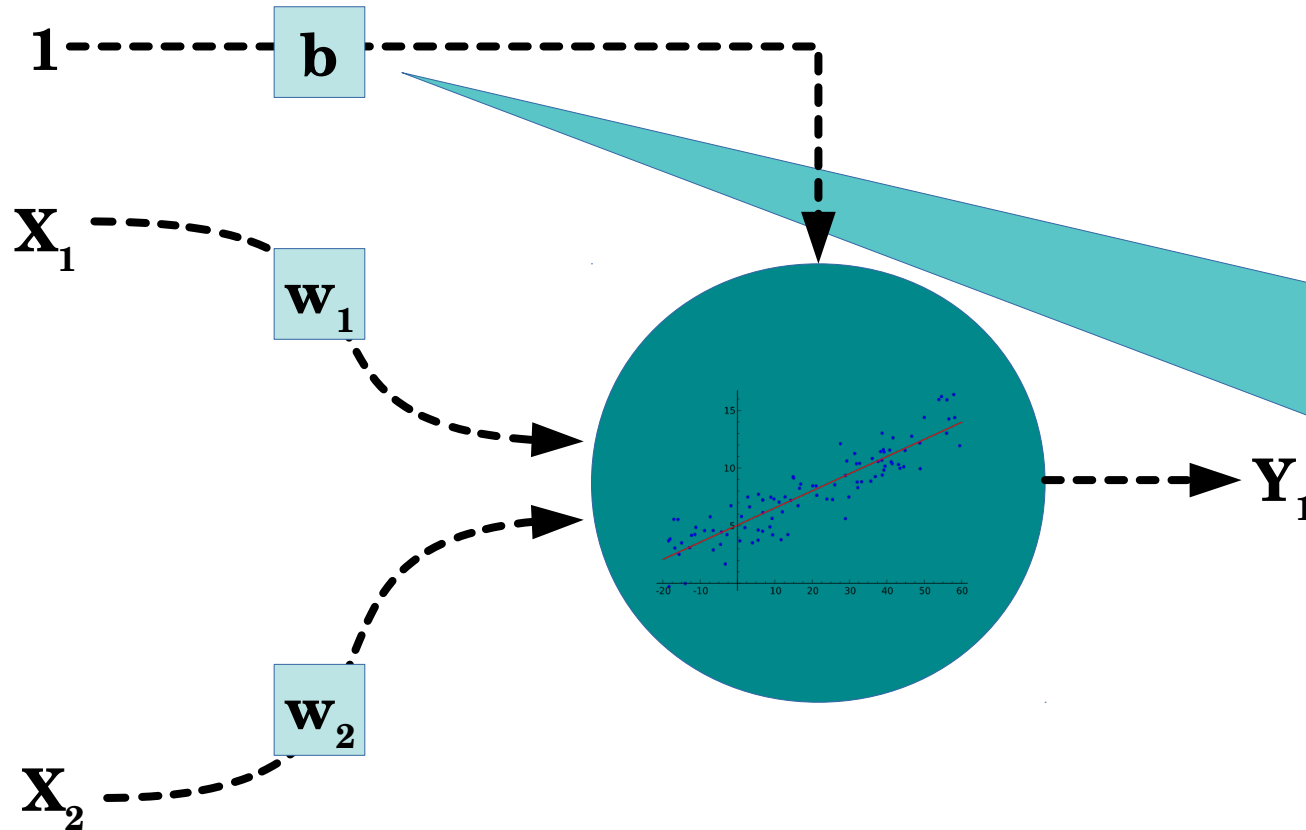
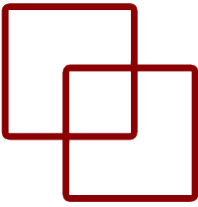
3. Función de la NN



El término independiente, llamado **bias** o **sesgo**, nos permite el desplazamiento de la recta.

$$y = w_1x_1 + w_2x_2 + \mathbf{b}$$

3. Función de la NN



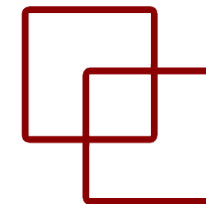
Así nuestra neruona
funcionaría exactamente
como un **modelo de regresión**

$$y = w_1 x_1 + w_2 x_2 + b$$



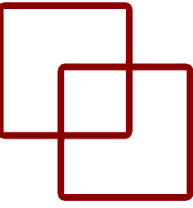
Simulación

1. Ejemplo

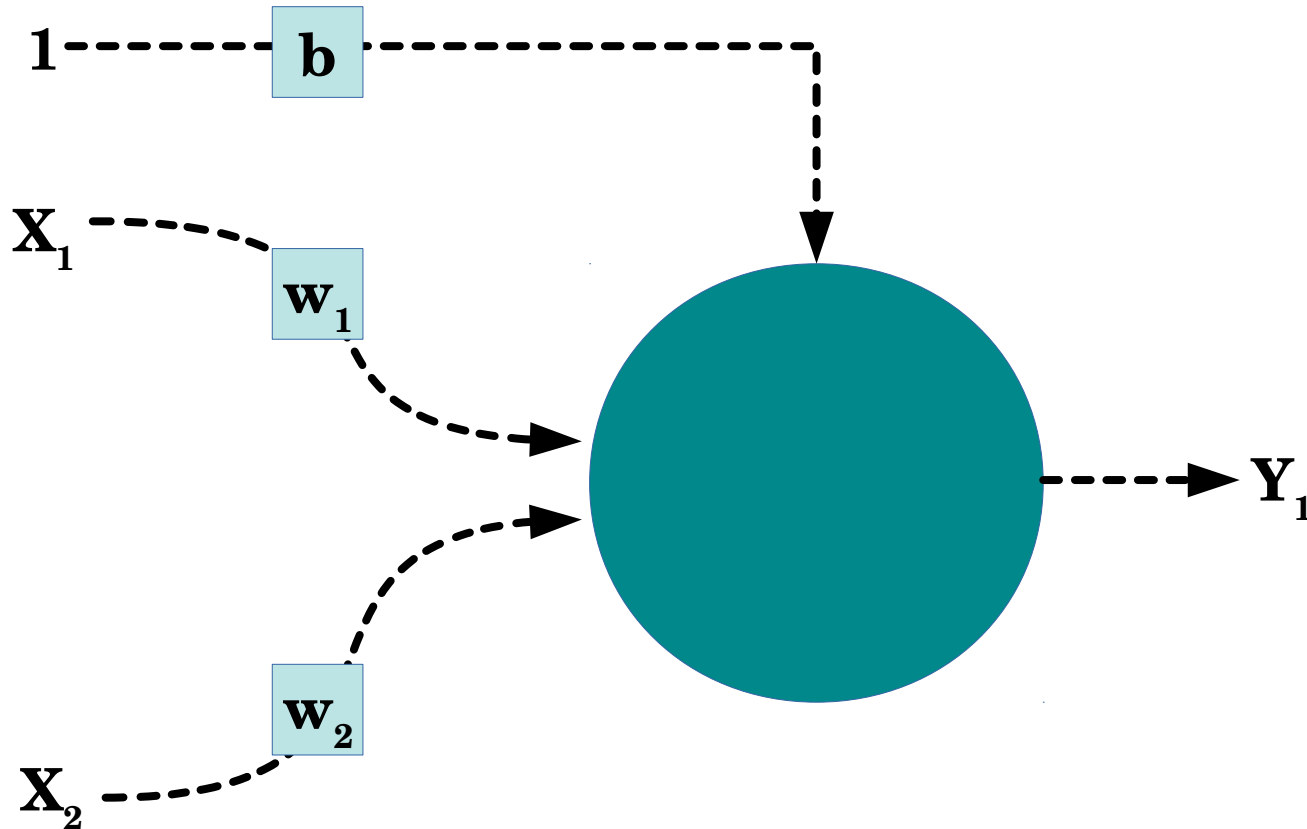


Estudiar + Estar en casa = Aprobar el examen

1. Ejemplo

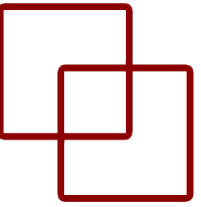


Estudiar + Estar en casa = Aprobar el examen

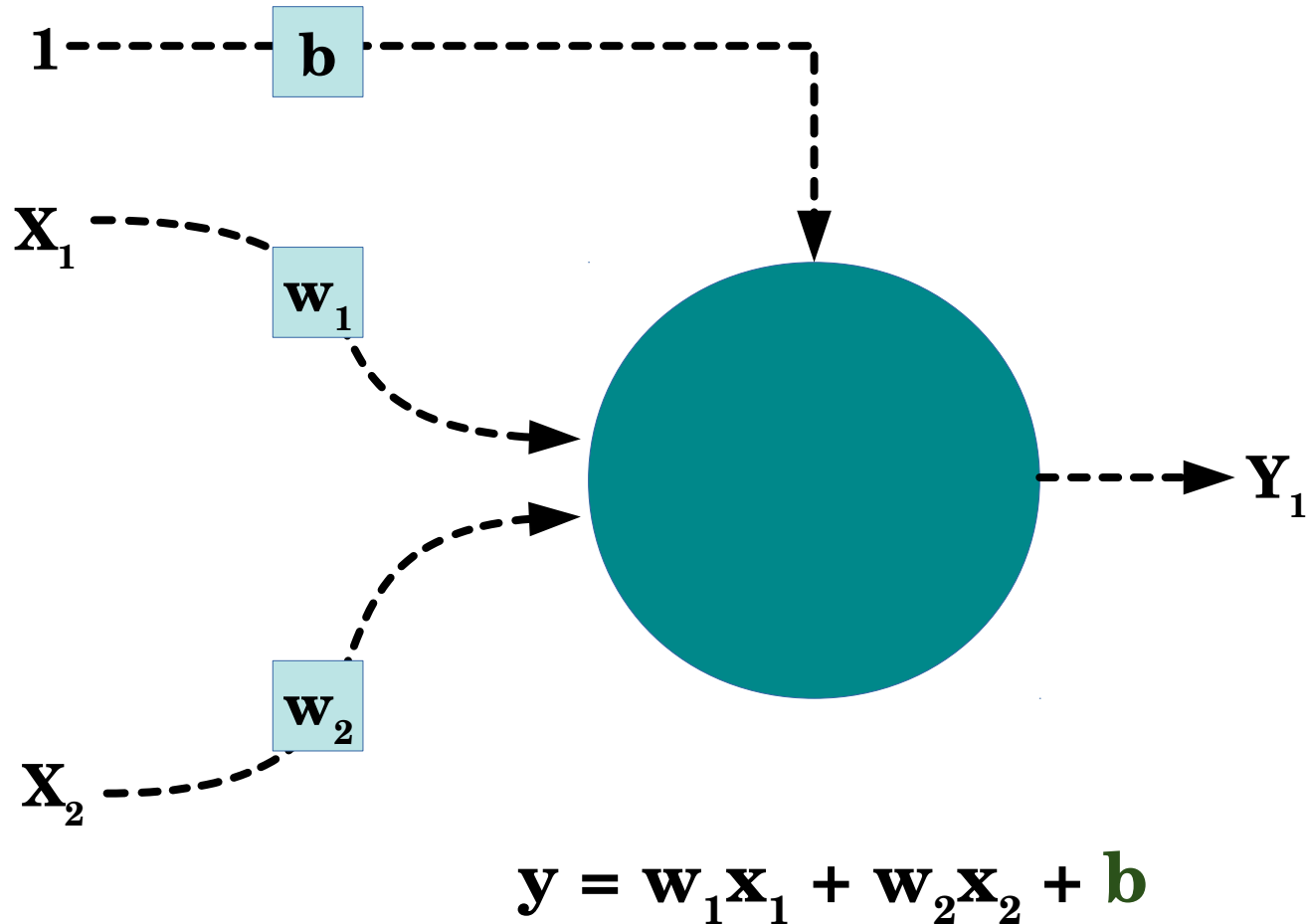


$$y = w_1 x_1 + w_2 x_2 + b$$

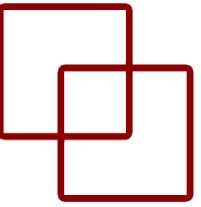
1. Ejemplo



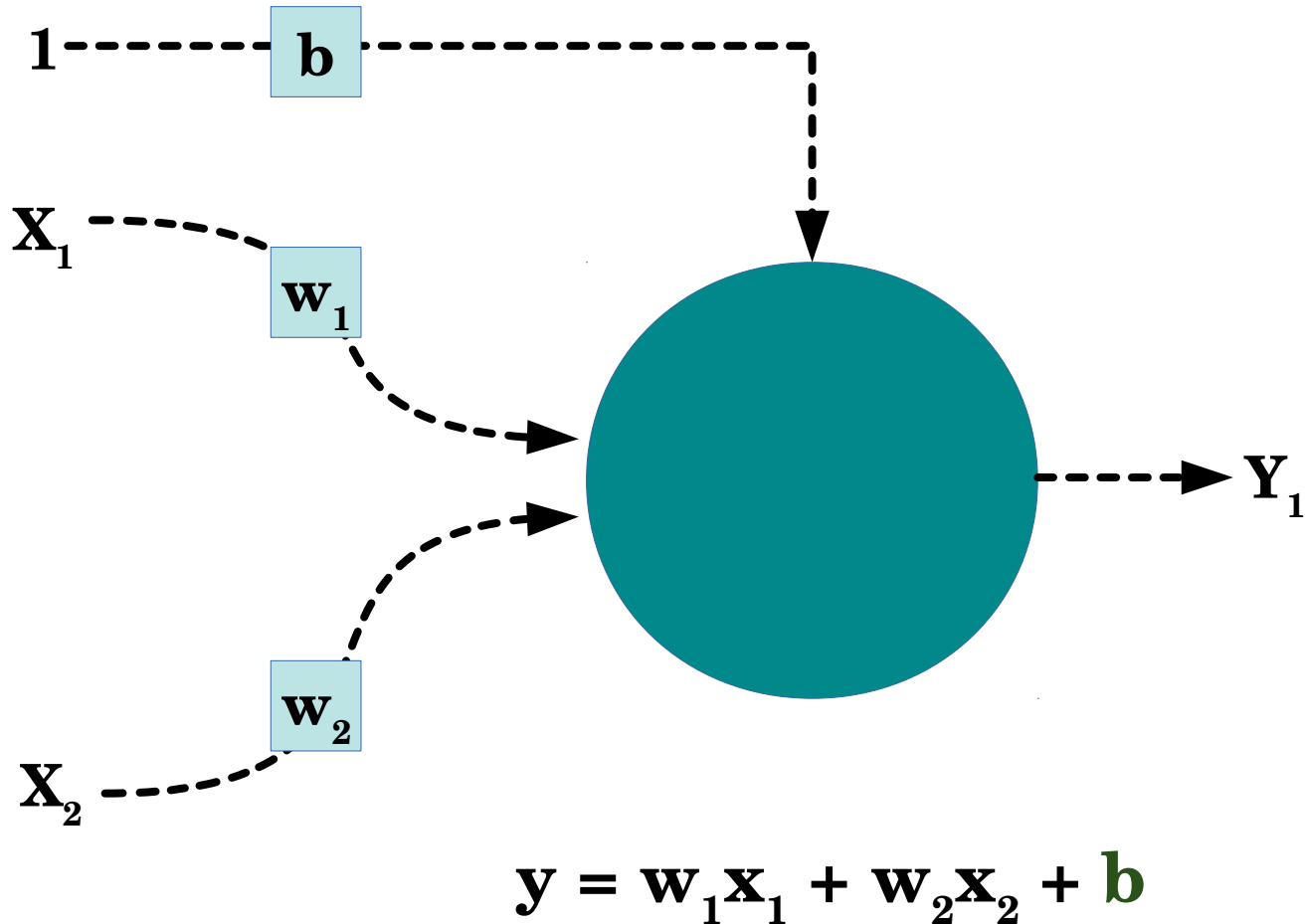
Estudiar + Estar en casa = Aprobar el examen



1. Ejemplo

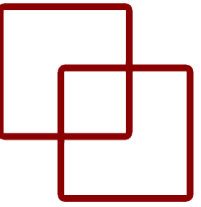


Estudiar + Estar en casa = Aprobar el examen



1. Ejemplo

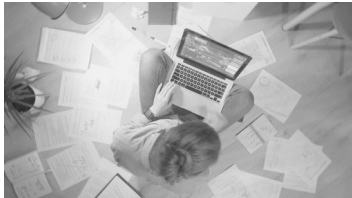
Umbral



Estudiar + Estar en casa = Aprobar el examen

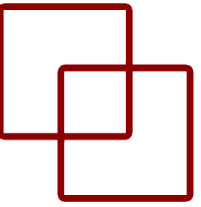
$WX \leq \text{UMBRAL} \rightarrow Y = 0$

$WX > \text{UMBRAL} \rightarrow Y = 1$

	1	0
X_1		
X_2		
.....		
Y_1		

1. Ejemplo

Umbral



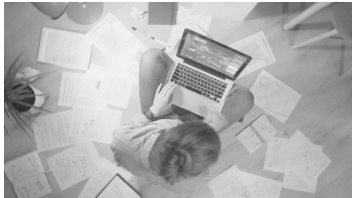
Estudiar + Estar en casa = Aprobar el examen

$$WX \leq \text{UMBRAL} \rightarrow Y = 0$$

$$WX > \text{UMBRAL} \rightarrow Y = 1$$

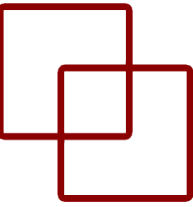
.....

$$\text{BIAS} = -\text{UMBRAL}$$

	1	0
X_1		
X_2		
Y_1		

1. Ejemplo

Umbral



Estudiar + Estar en casa = Aprobar el examen

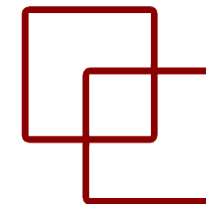
BIAS = -UMBRA

.....

$WX + b \leq 0 \rightarrow Y = 0$

$WX + b > 0 \rightarrow Y = 1$



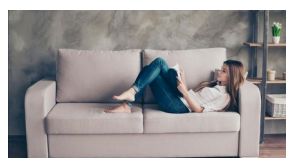
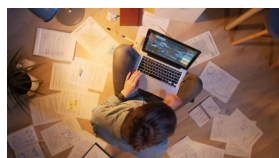
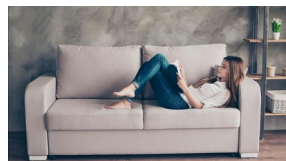
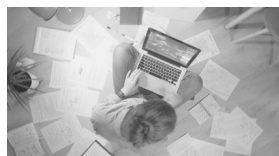
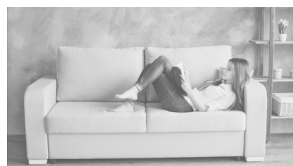
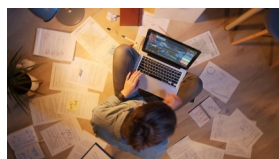
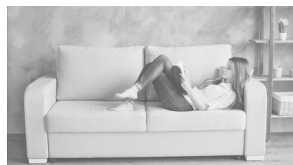
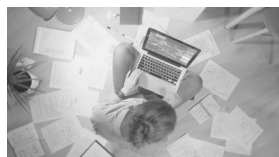


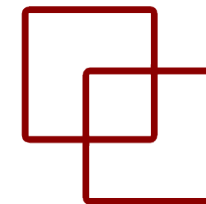
1. Ejemplo

Modelamos el problema

Estudiar + Estar en casa = Aprobar el examen

X_1	X_2	Target	Y
-------	-------	--------	-----



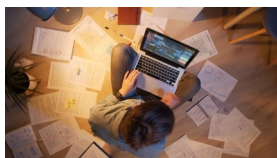
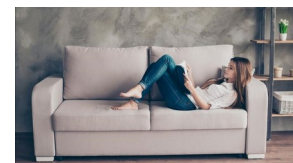
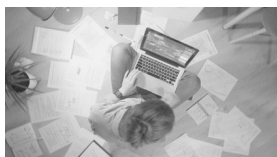
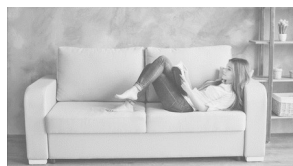
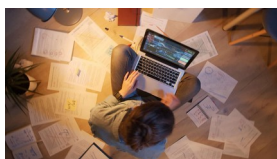
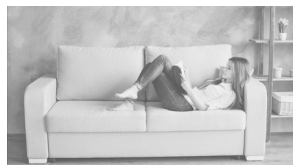
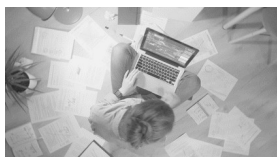


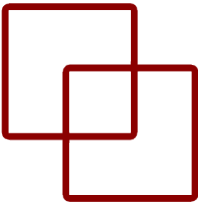
1. Ejemplo

Modelamos el problema

Estudiar + Estar en casa = Aprobar el examen

X_1	X_2	Target	Y
-------	-------	--------	---





1. Ejemplo

Modelamos el problema

Estudiar + Estar en casa = Aprobar el examen

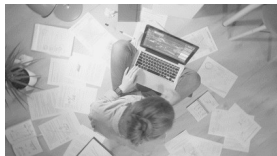
X_1	X_2	Target	Y
-------	-------	--------	---



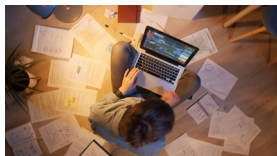
?



?

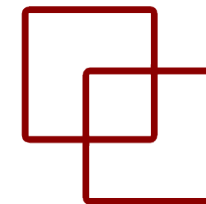


?



?

El resultado de la NN estará condicionado por el valor de los **parámetros**. Por tanto, tenemos que buscar esos pesos y bias.

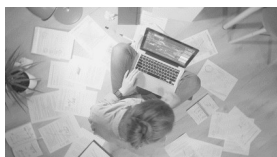


1. Ejemplo

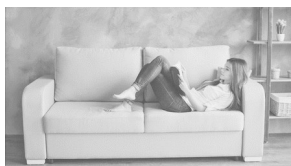
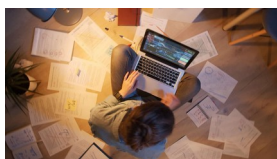
Buscamos los parámetros

Estudiar + **Estar en casa** = **Aprobar el examen**

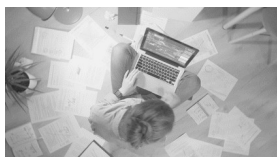
X_1	X_2	Target	Y
-------	-------	--------	-----



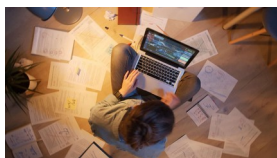
?



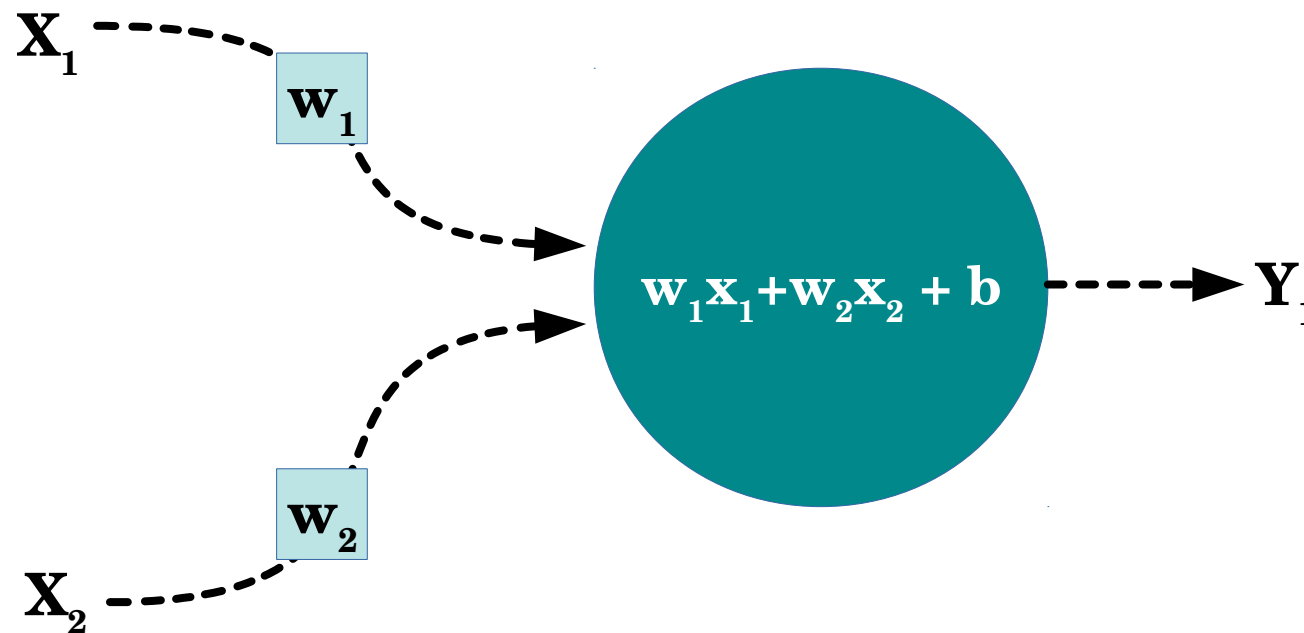
?

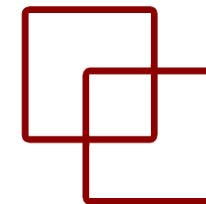


?



?



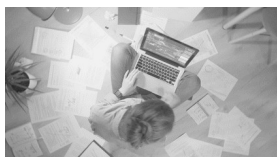


1. Ejemplo

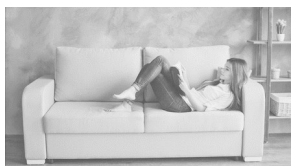
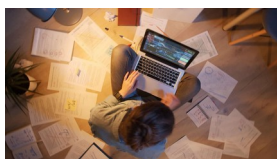
Buscamos los parámetros

Estudiar + Estar en casa = Aprobar el examen

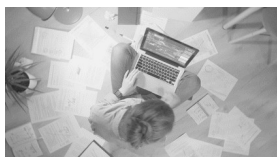
X_1	X_2	Target	Y
-------	-------	--------	-----



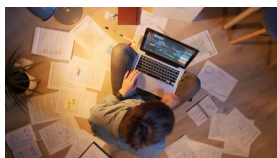
-3



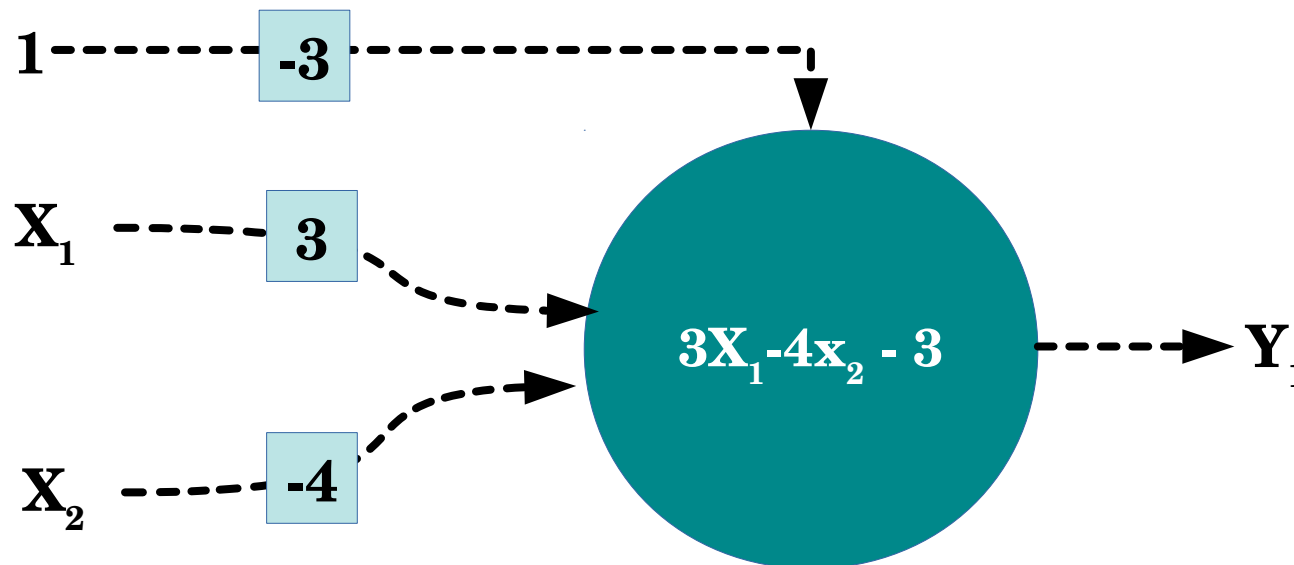
0

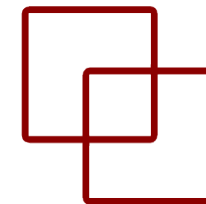


-7



-4



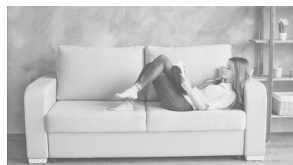
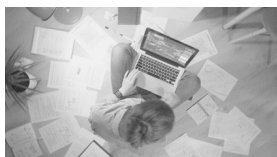


1. Ejemplo

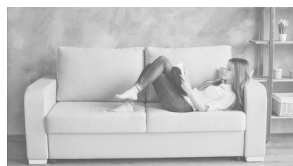
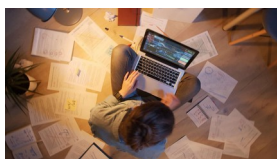
Buscamos los parámetros

Estudiar + Estar en casa = Aprobar el examen

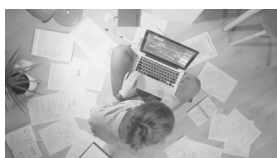
X_1	X_2	Target	Y
-------	-------	--------	-----



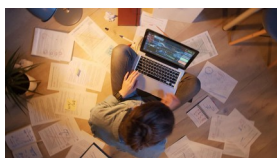
-3



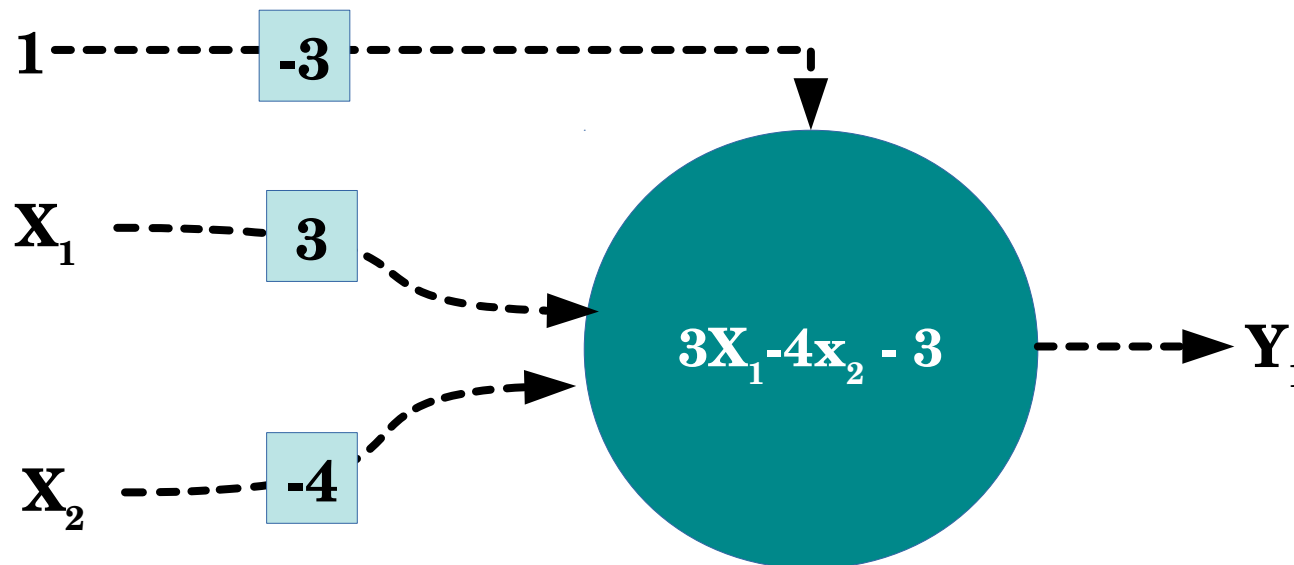
0



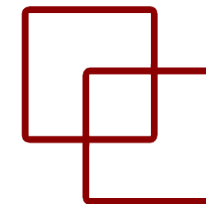
-7



-4



El objetivo es que salga un resultado positivo el último Y y negativo los tres primeros

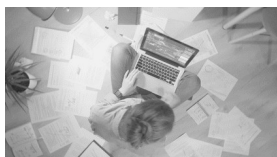


1. Ejemplo

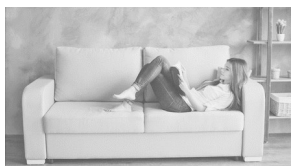
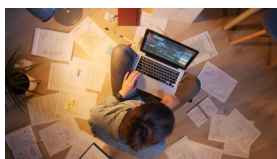
Buscamos los parámetros

Estudiar + Estar en casa = Aprobar el examen

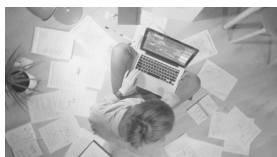
X_1	X_2	Target	Y
-------	-------	--------	-----



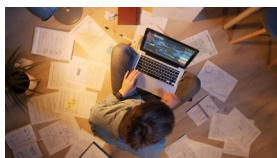
-3



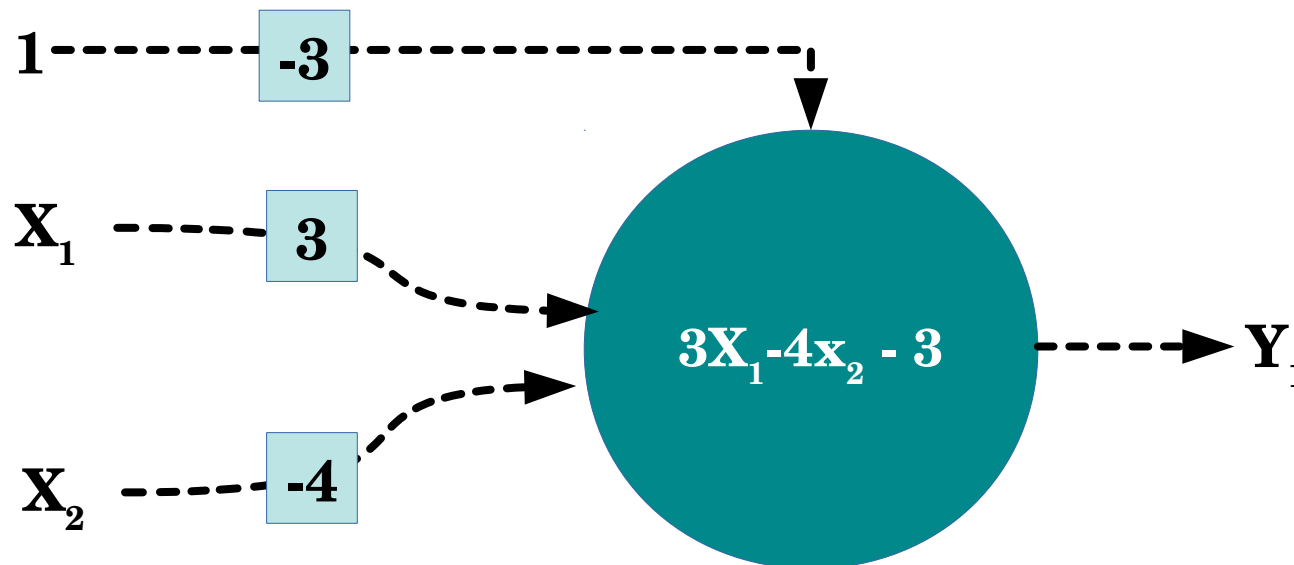
0



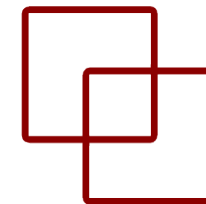
-7



-4



¡No cumple!

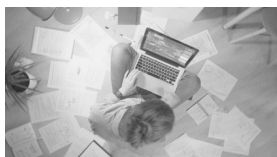


1. Ejemplo

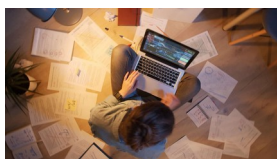
Buscamos los parámetros

Estudiar + Estar en casa = Aprobar el examen

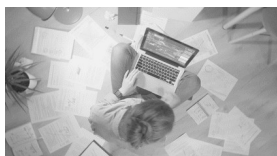
X_1	X_2	Target	Y
-------	-------	--------	-----



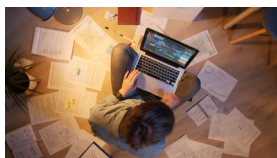
2



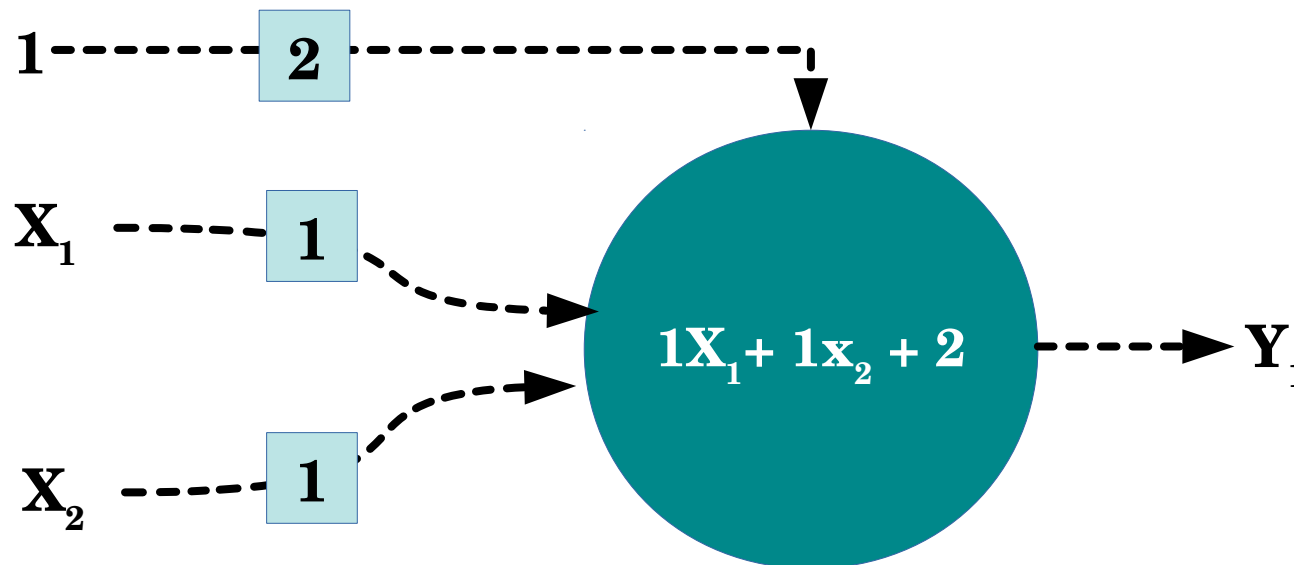
3

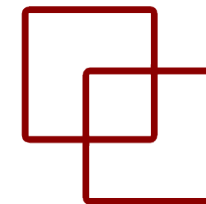


3



4



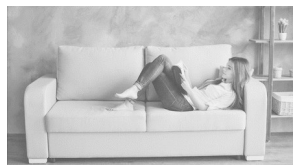
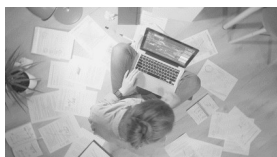


1. Ejemplo

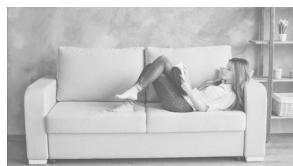
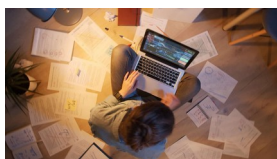
Buscamos los parámetros

Estudiar + Estar en casa = Aprobar el examen

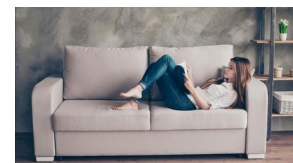
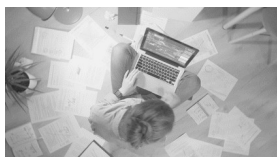
X_1	X_2	Target	Y
-------	-------	--------	-----



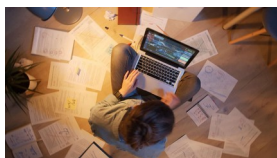
2



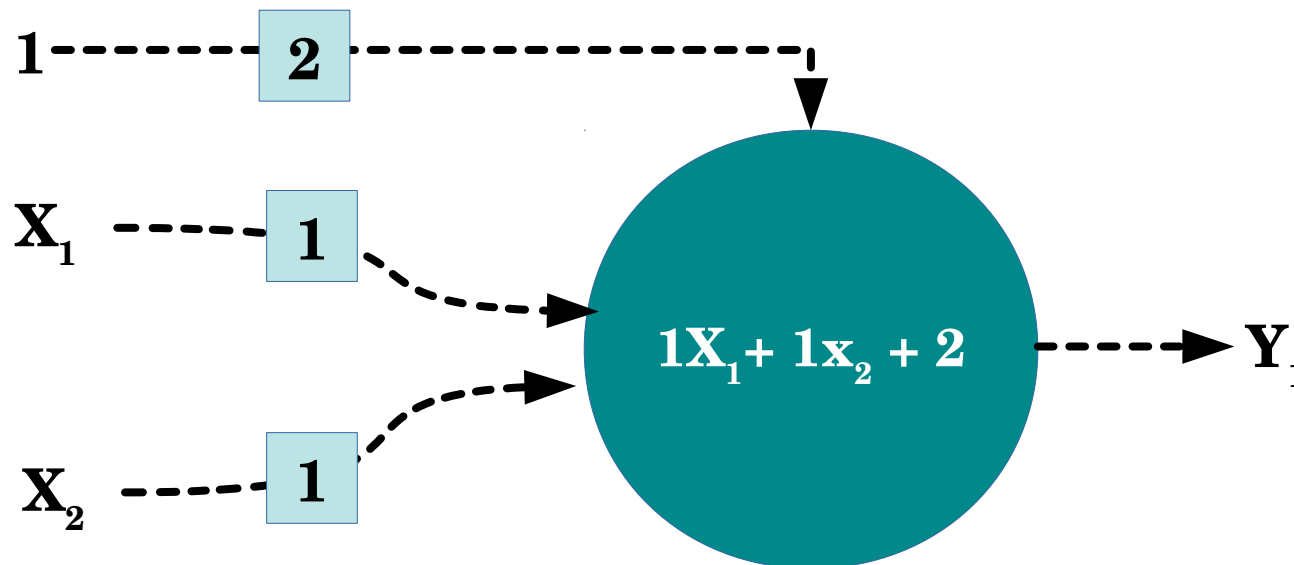
3



3



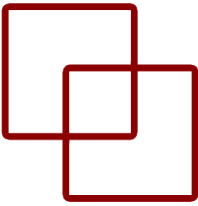
4



¡No cumple!

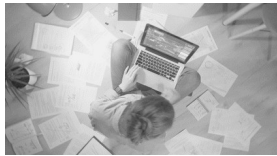
1. Ejemplo

Buscamos los parámetros



Estudiar + Estar en casa = Aprobar el examen

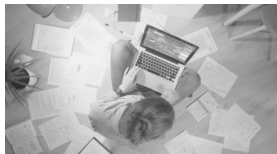
X_1	X_2	Target	Y
-------	-------	--------	-----



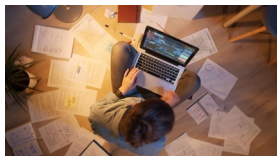
-6



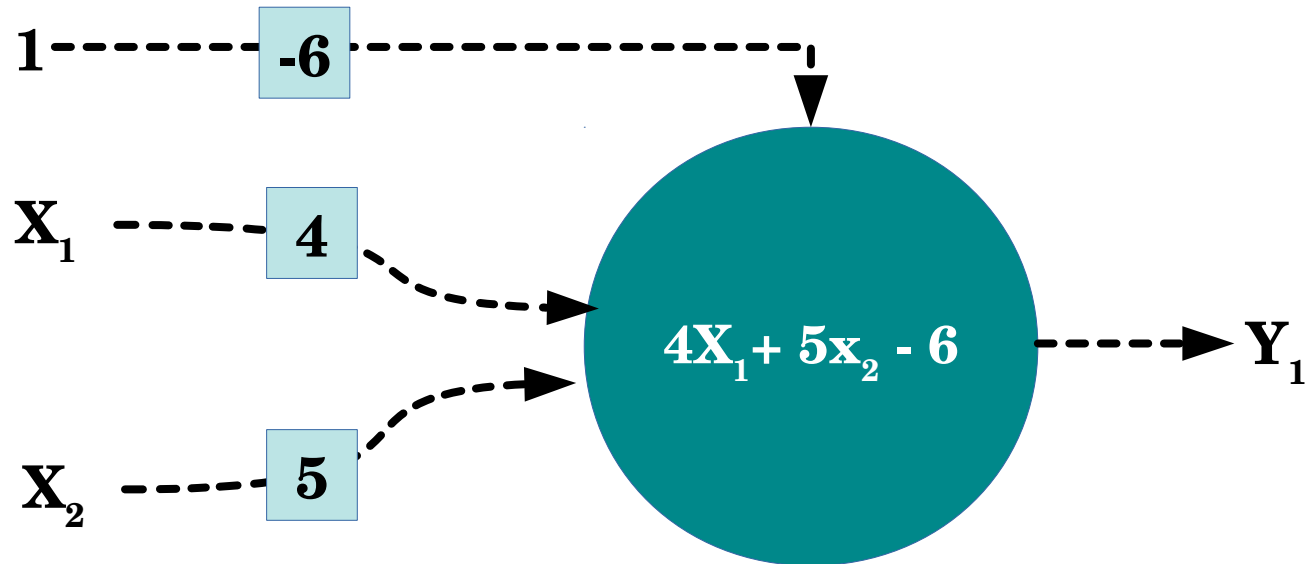
-2

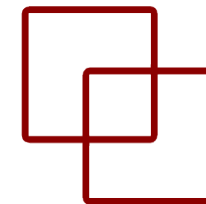


-1



3



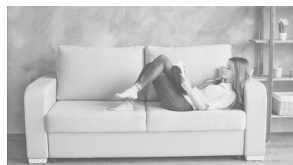
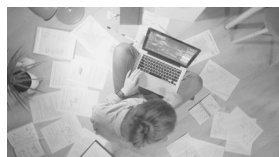


1. Ejemplo

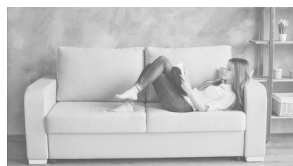
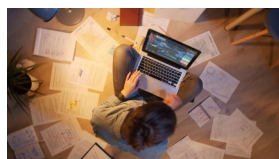
Buscamos los parámetros

Estudiar + Estar en casa = Aprobar el examen

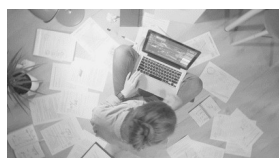
X_1	X_2	Target	Y
-------	-------	--------	-----



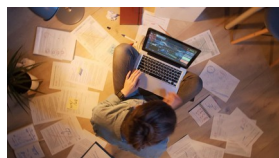
-6



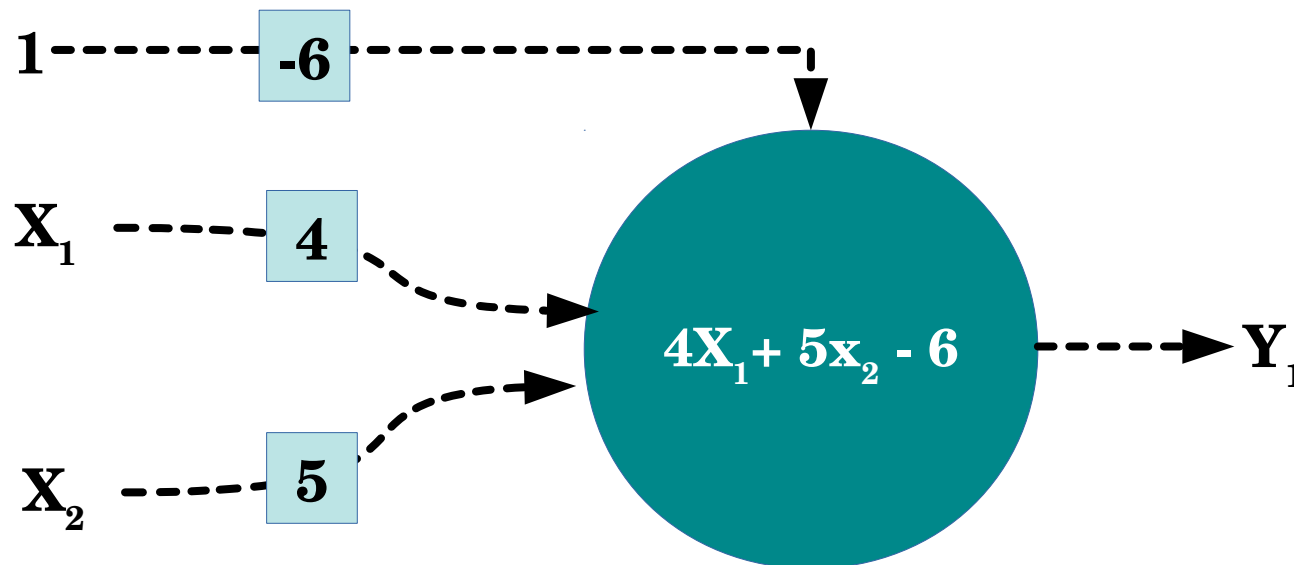
-2



-1



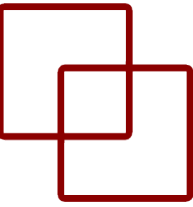
3



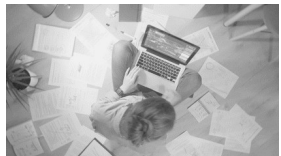
¡SI cumple!

1. Ejemplo

Entender el procedimiento



X_1

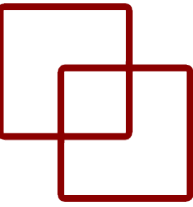


X_2

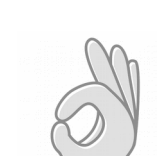
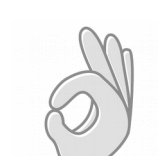
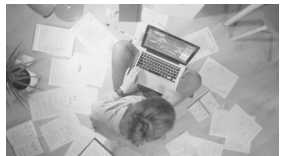


1. Ejemplo

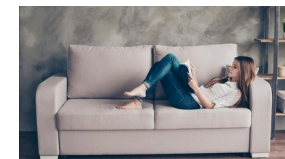
Entender el procedimiento



$\mathbf{x}_1$

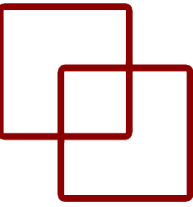


$\mathbf{x}_2$

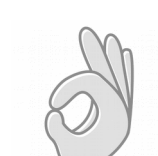
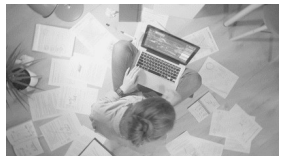


1. Ejemplo

Entender el procedimiento



$\mathbf{x}_1$

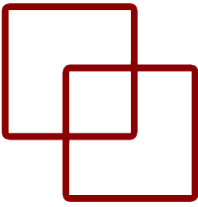


$\mathbf{x}_2$



1. Ejemplo

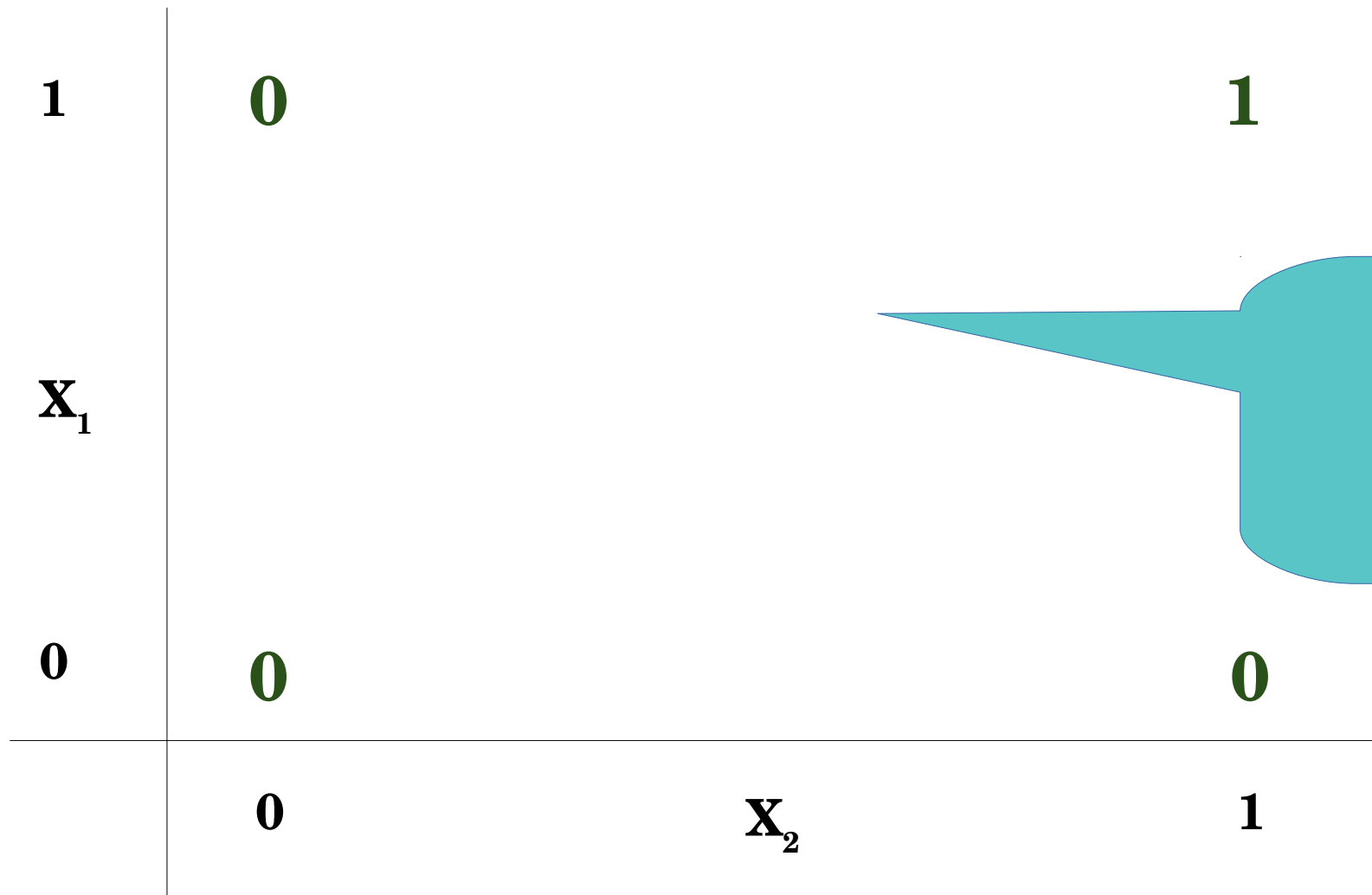
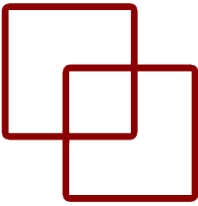
Entender el procedimiento



1	0	1
x_1		
0	0	0
	0	x_2
		1

1. Ejemplo

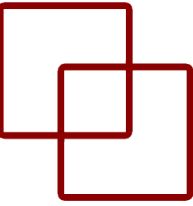
Entender el procedimiento



¿Qué observamos?

1. Ejemplo

Puerta lógica AND

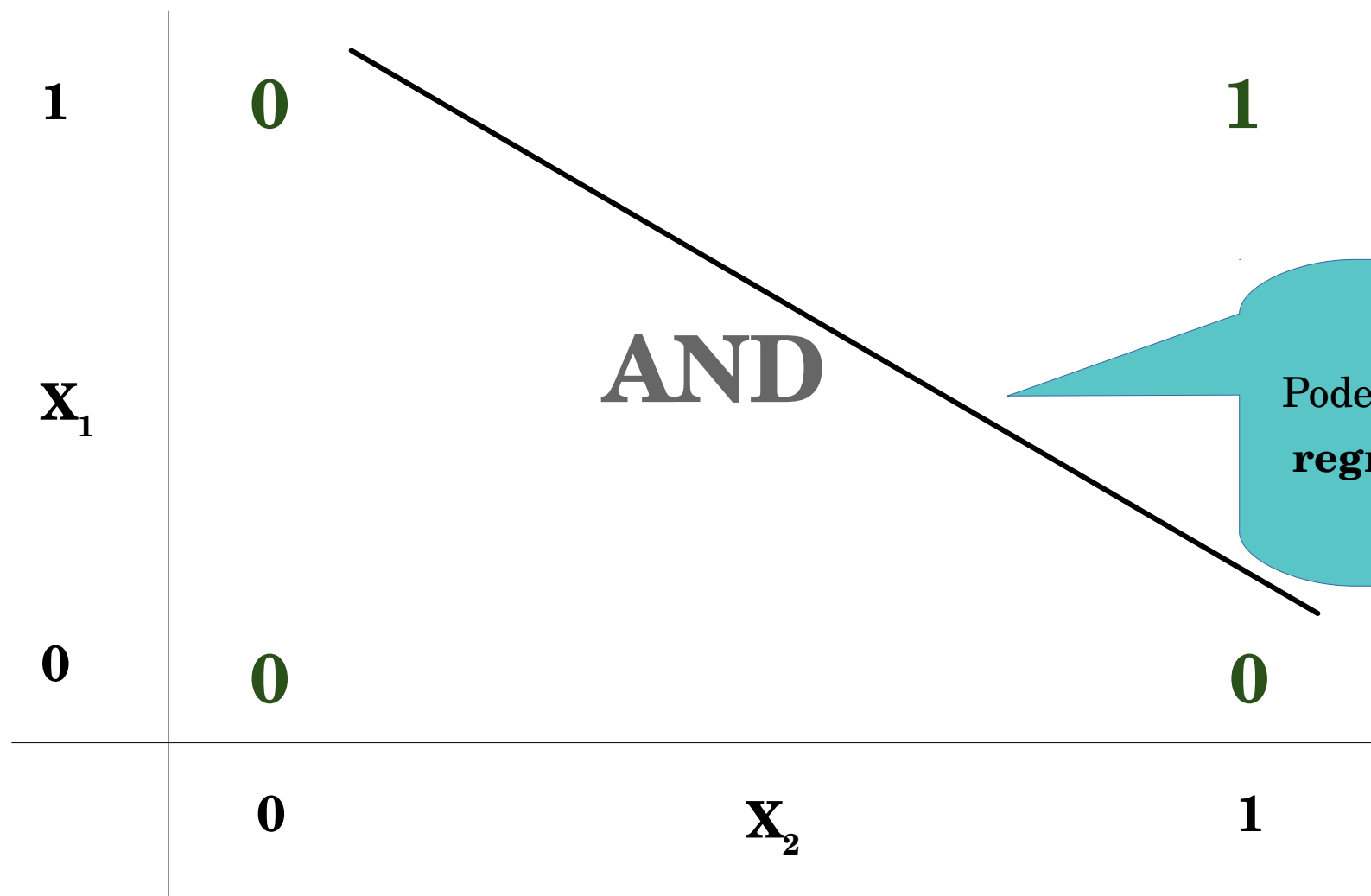
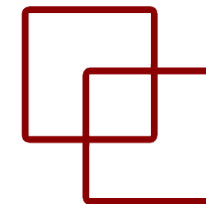


1	0	1
x_1		
0	0	0
	0	1
	x_2	

¿Qué observamos?
Que tenemos una **puerta**
lógica AND

1. Ejemplo

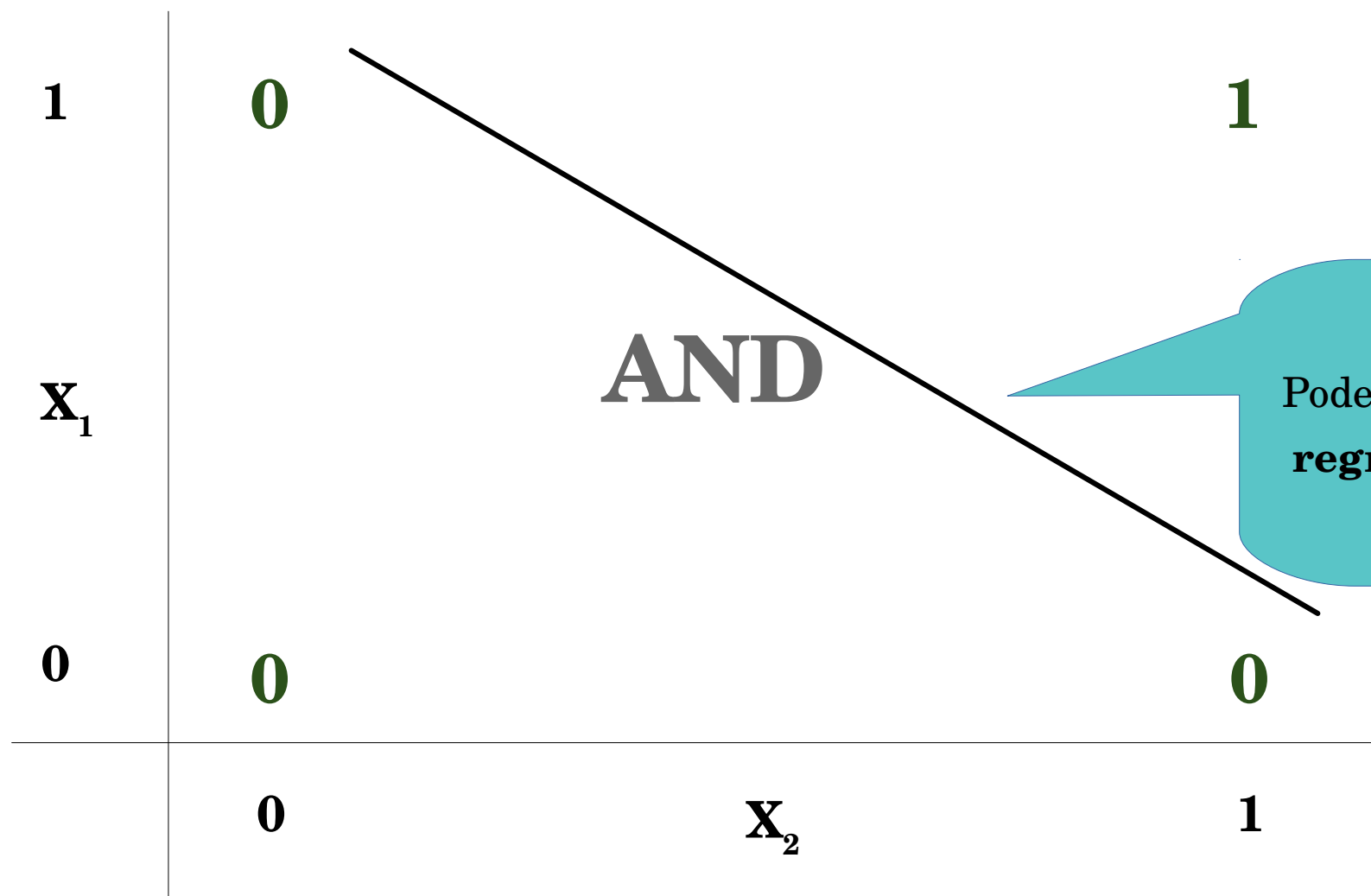
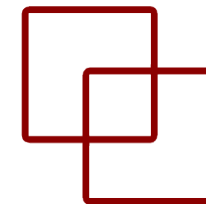
Puerta lógica AND



Podemos establecer la **recta de regresión** que hemos hablado

1. Ejemplo

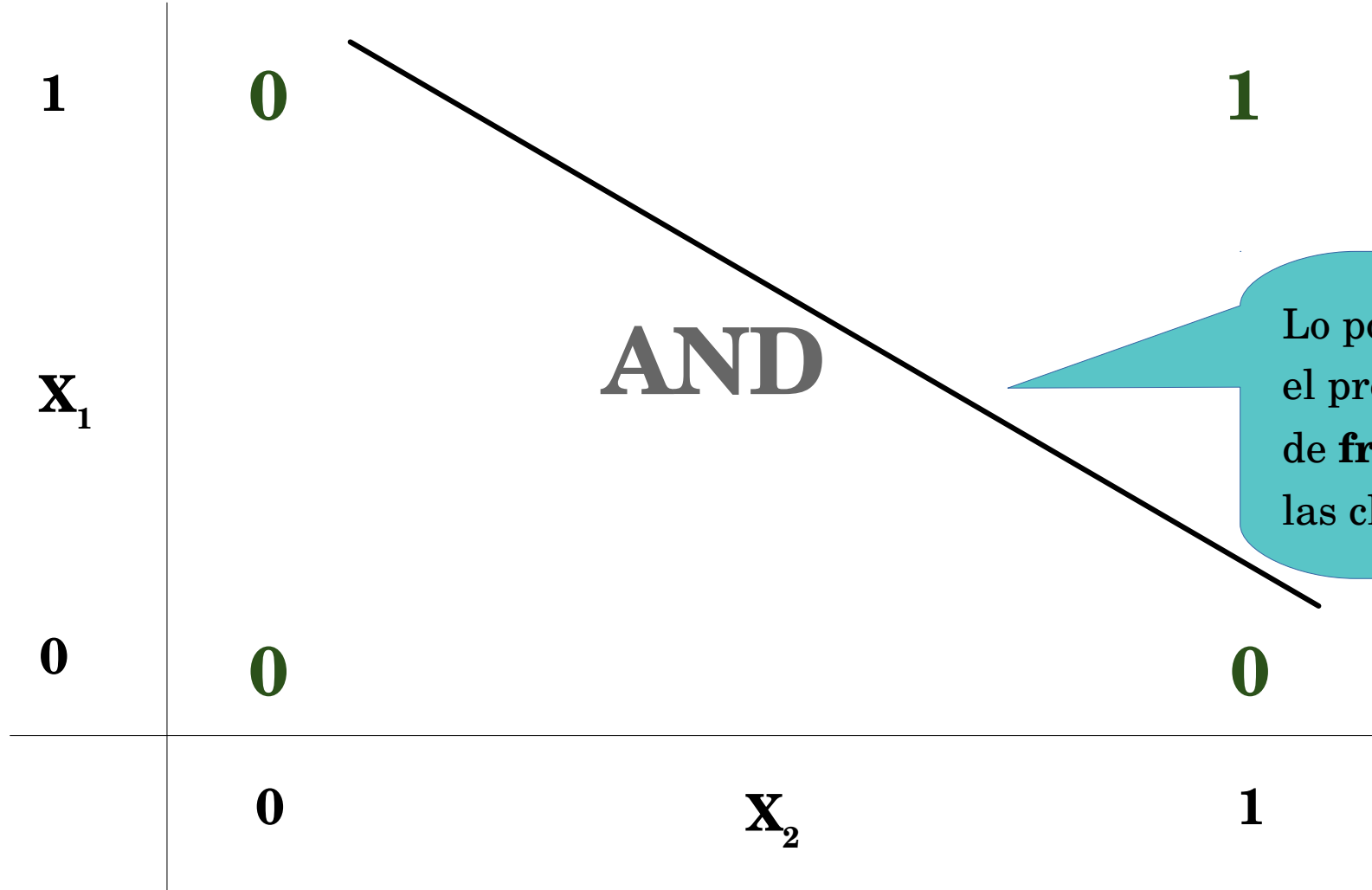
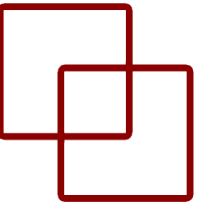
Puerta lógica AND



Podemos establecer la **recta de regresión** que hemos hablado

1. Ejemplo

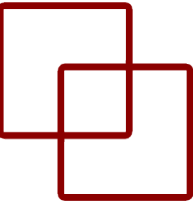
Puerta lógica AND



Lo podemos entender como el problema de trazar una línea de **frontera** en la que separar las clases

1. Ejemplo

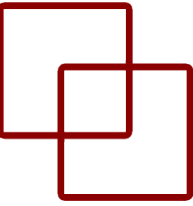
Puerta lógica OR



1	1	1
x_1	OR	
0	0	1
	0	1
	x_2	

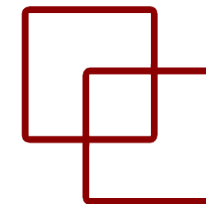
1. Ejemplo

Puerta lógica OR



1	1	1
x_1	OR	
0	0	1
	0	1
	x_2	

Tendríamos que ajustar los parámetros de la neurona hasta ajustar los resultados esperados

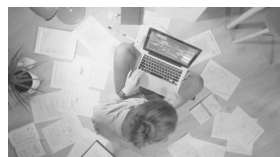


1. Ejemplo

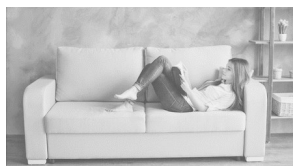
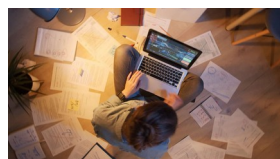
Puerta lógica OR – Buscar parámetros

Estudiar + Estar en casa = Aprobar el examen

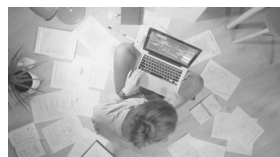
X_1	X_2	Target	Y
-------	-------	--------	-----



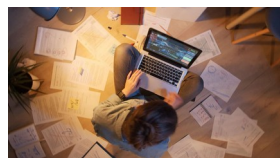
-2



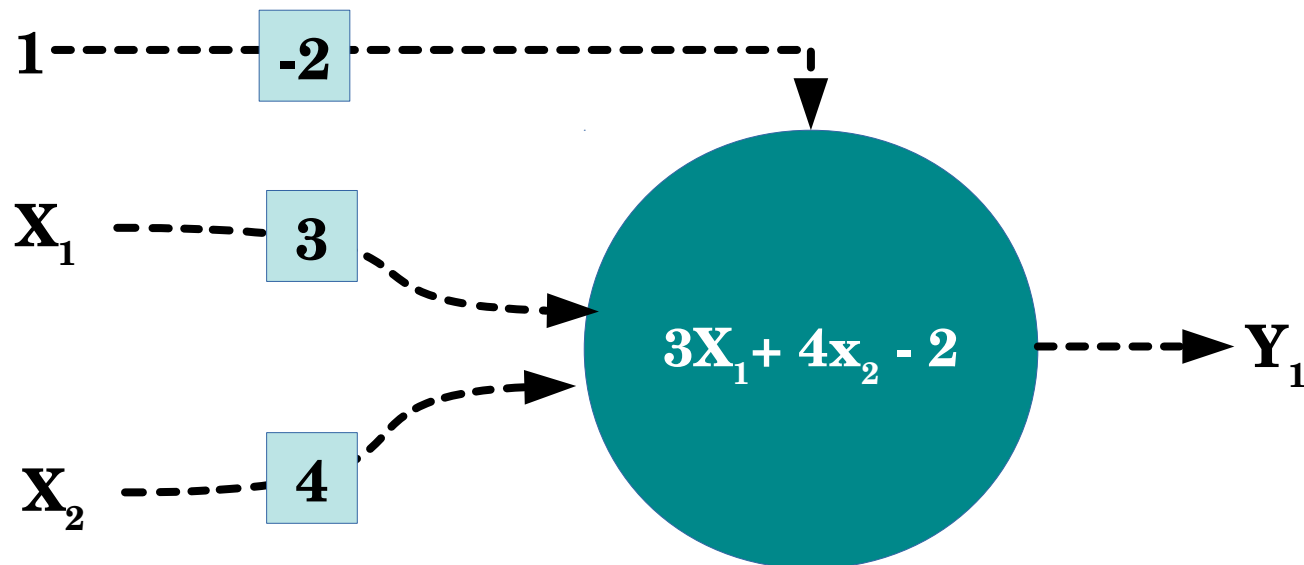
1

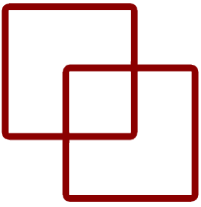


2



5



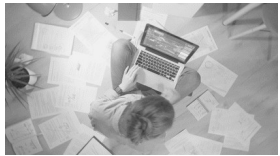


1. Ejemplo

Puerta lógica OR – Buscar parámetros

Estudiar + Estar en casa = Aprobar el examen

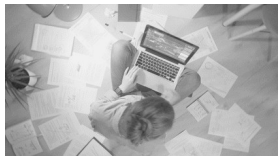
X_1	X_2	Target	Y
-------	-------	--------	-----



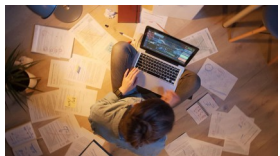
-2



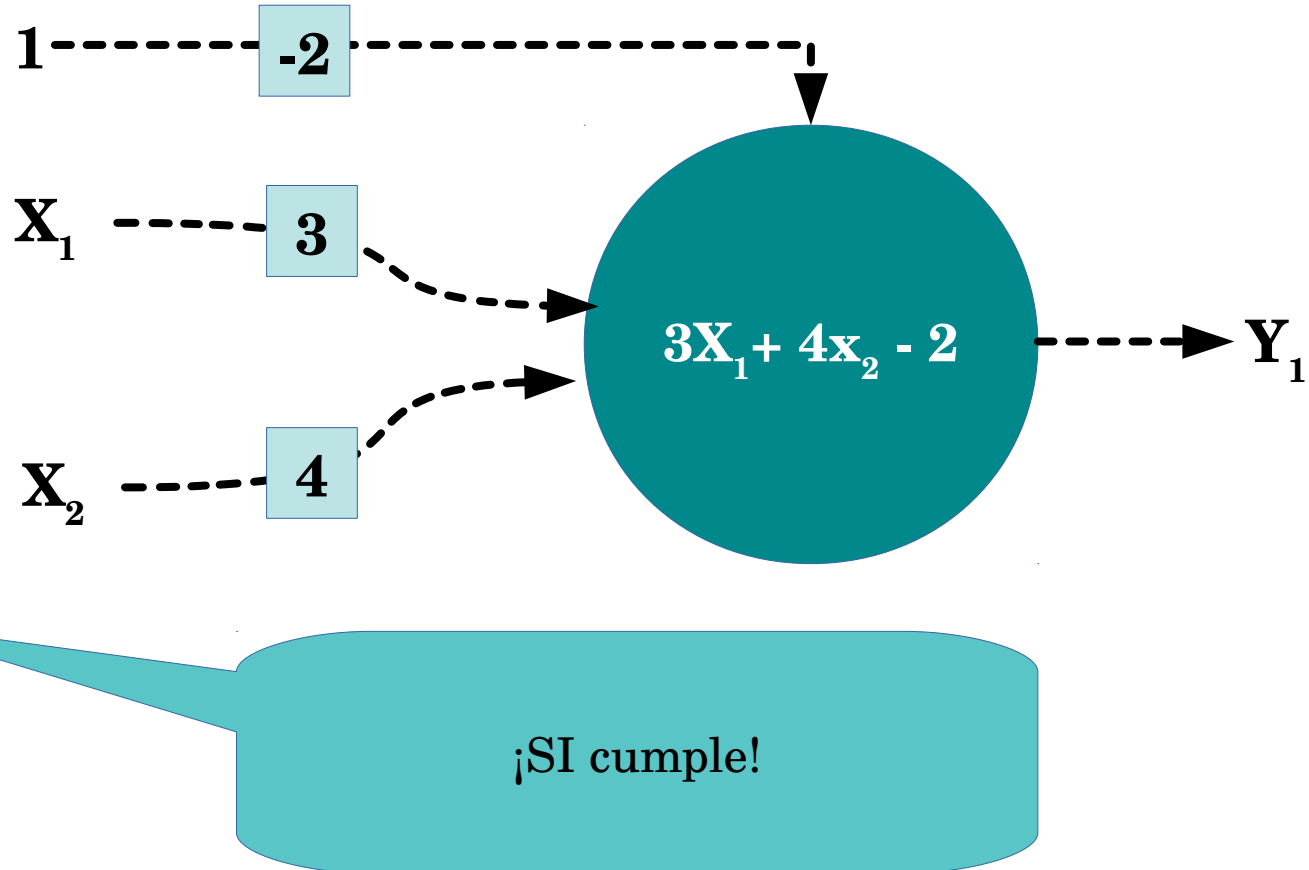
1



2

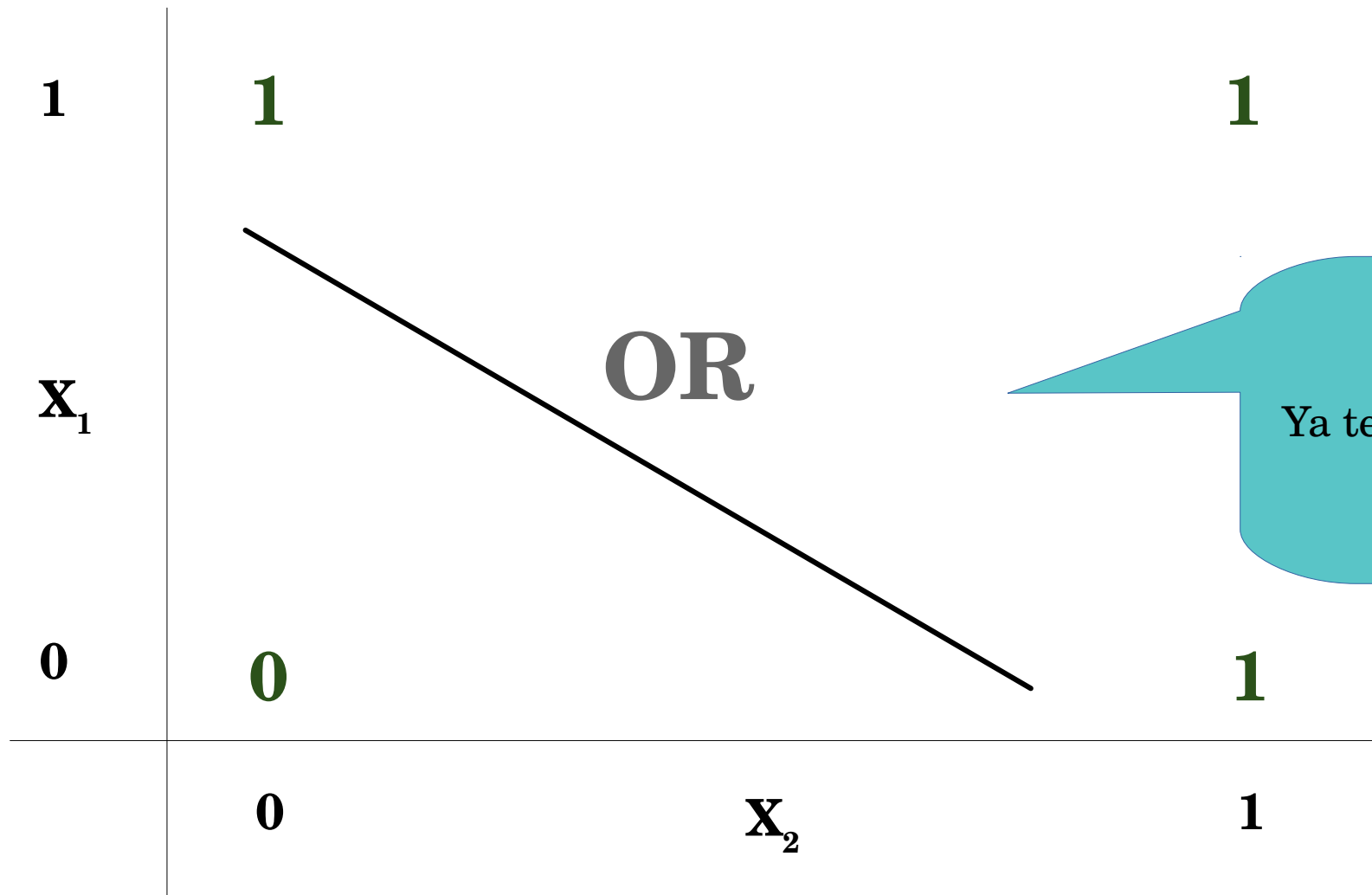
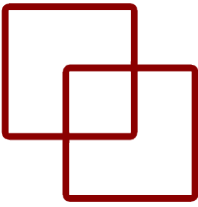


5



1. Ejemplo

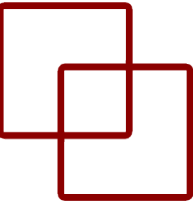
Puerta lógica OR



Ya tenemos la línea de frontera

1. Ejemplo

Puerta lógica XOR

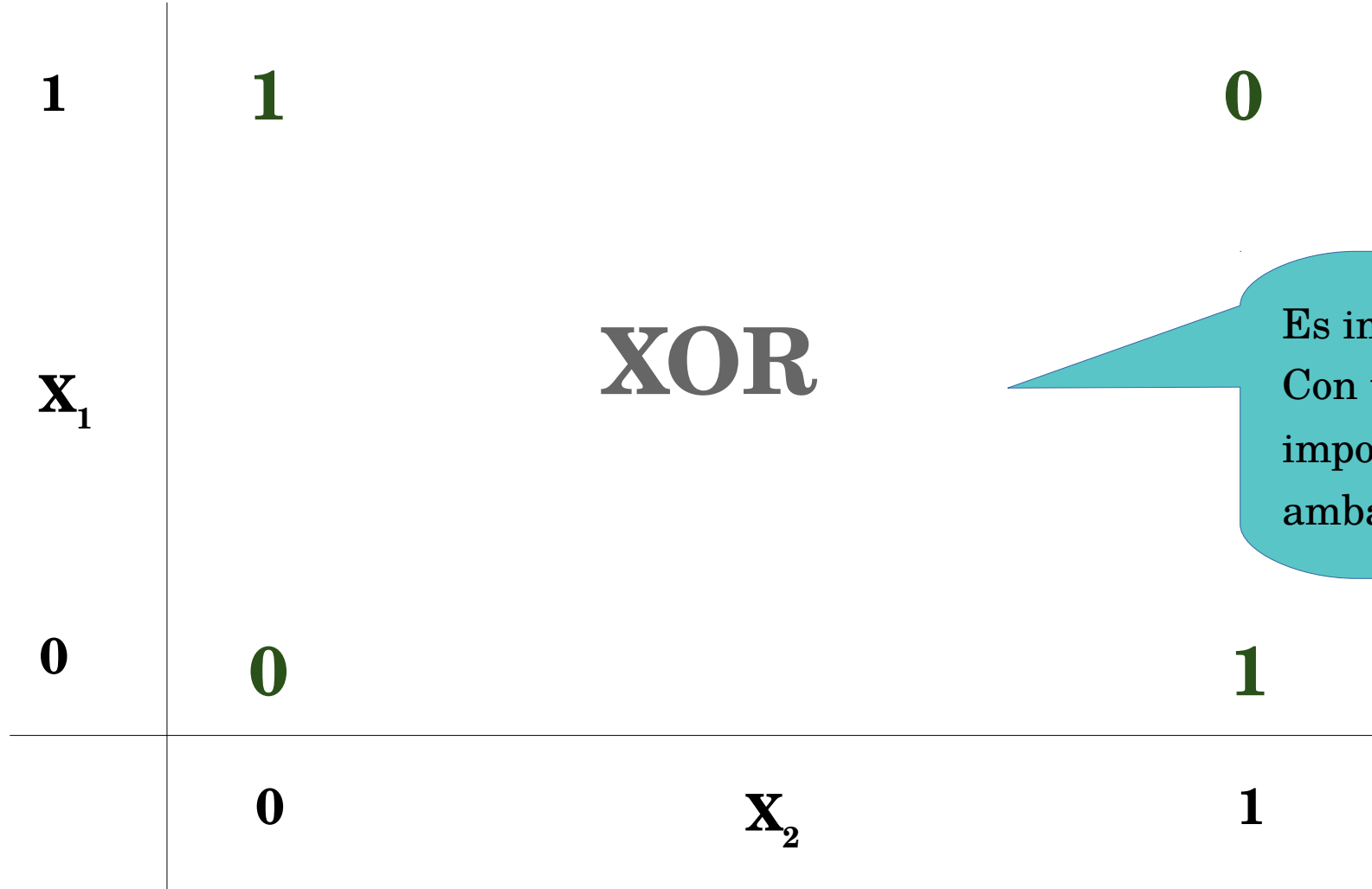
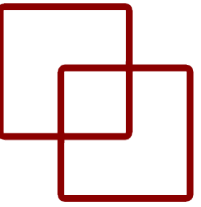


1	1	0
x_1	XOR	
0	0	1
	0	1
	x_2	

¿Como sería con XOR?
¿Recordáis la sesión anterior?

1. Ejemplo

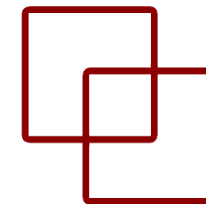
Puerta lógica XOR



Es imposible modelar el problema
Con una única neurona, i.e., es
imposible **separar linealmente**
ambas clases

1. Ejemplo

Puerta lógica XOR



1	1	0
x_1	XOR	
0	0	1
	0	1
	x_2	

¿Como lo solucionamos?

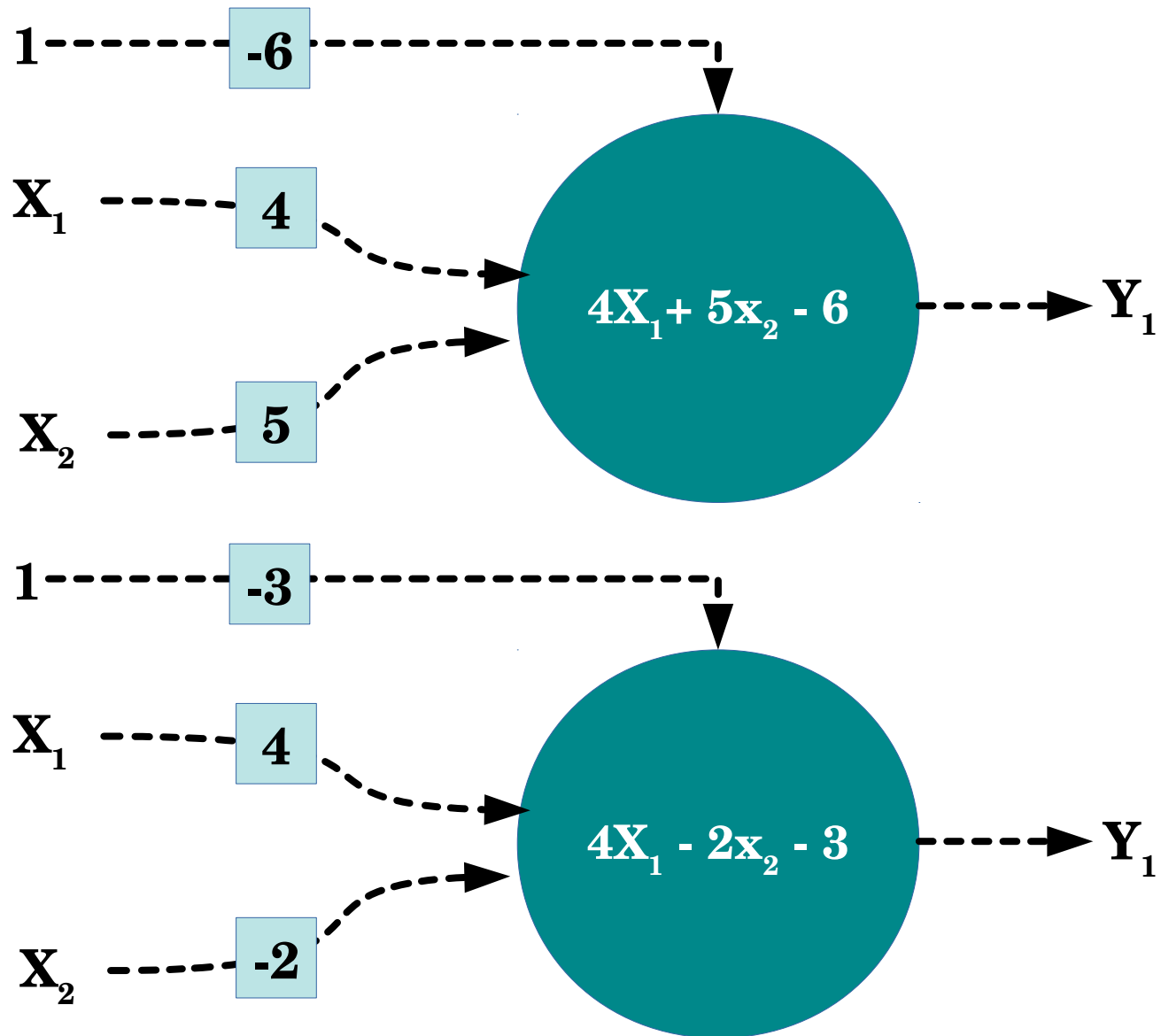
1. Ejemplo

Puerta lógica XOR

		X	
		0	1
1	1	0	
0	0	1	

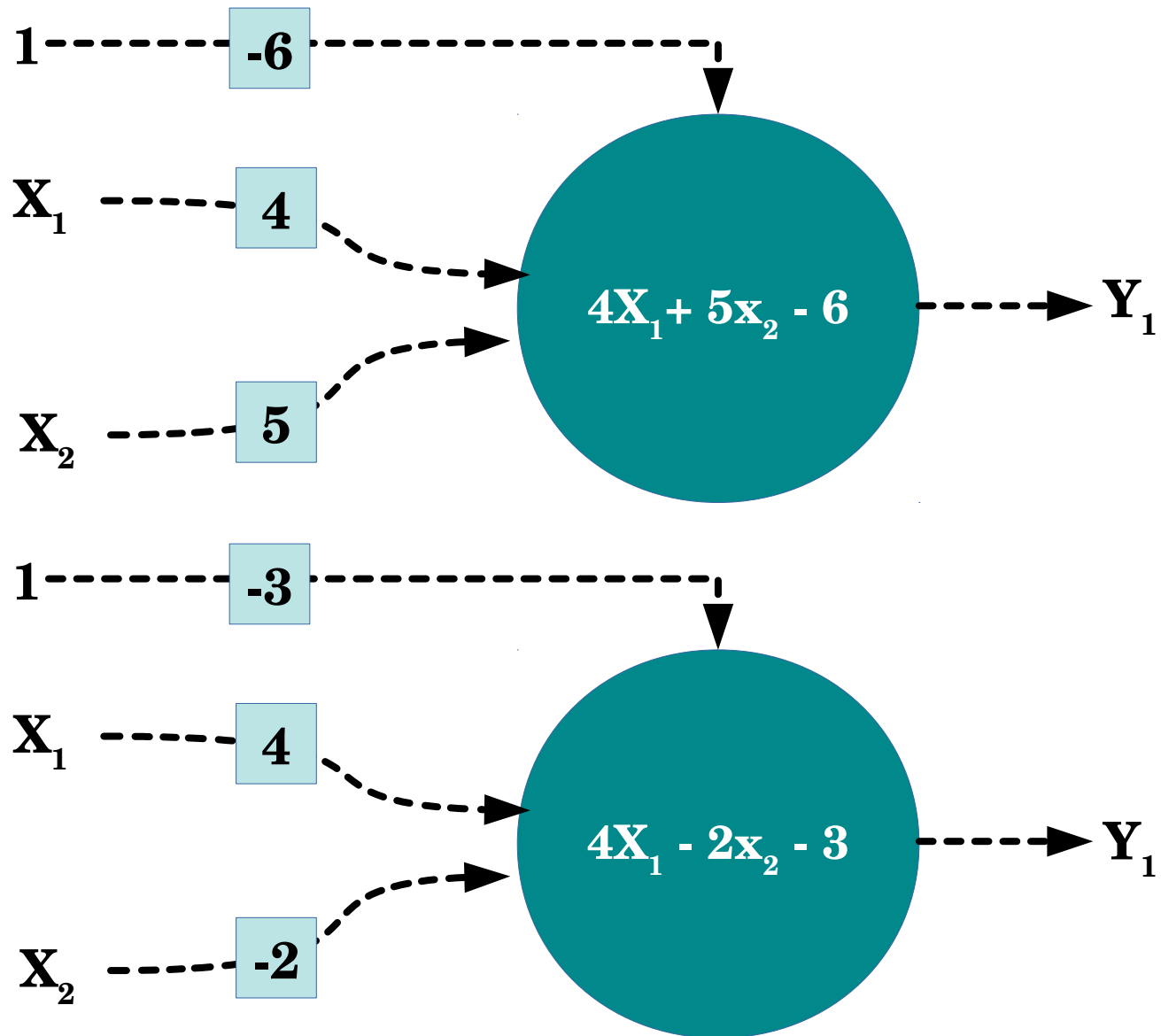
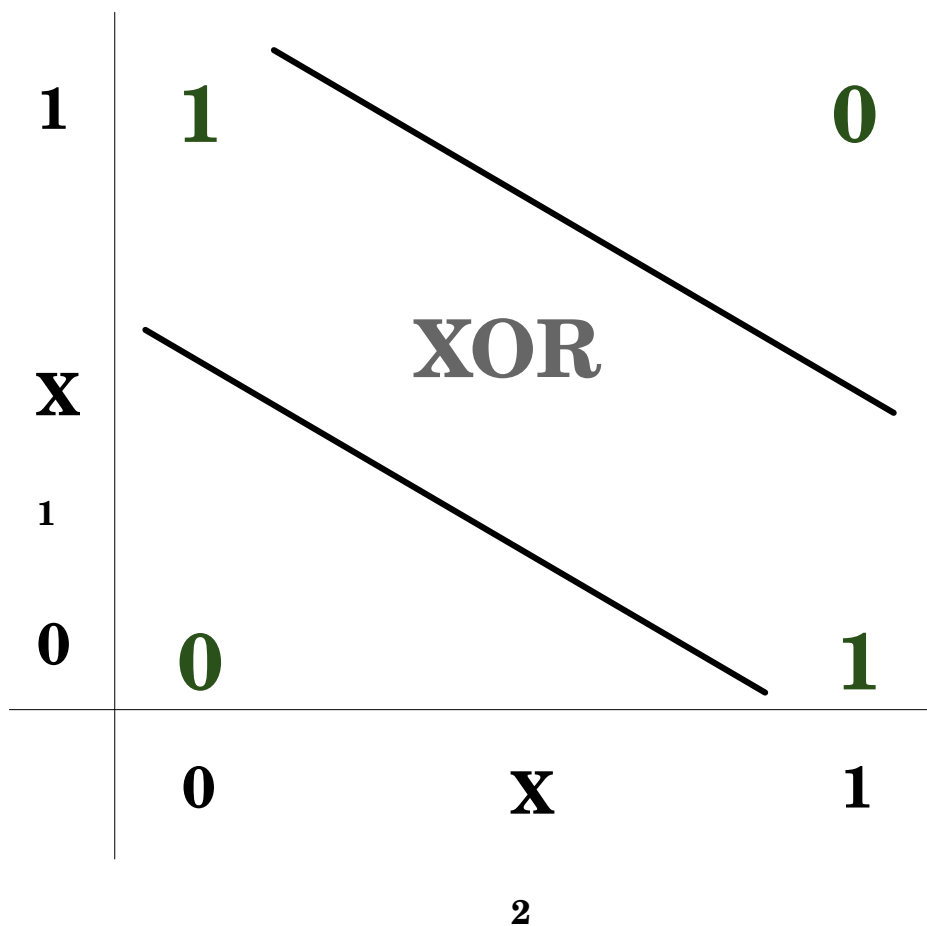
XOR

2



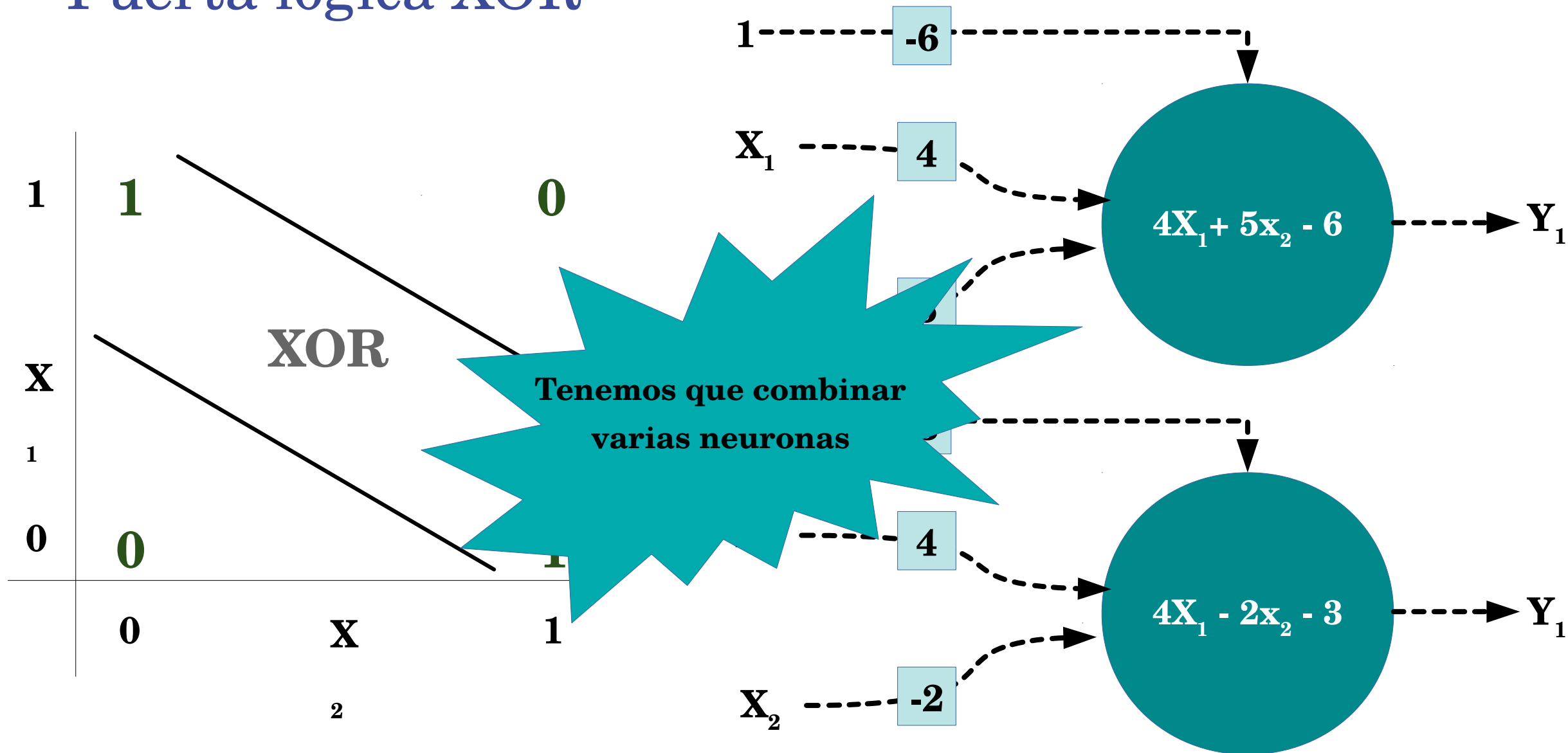
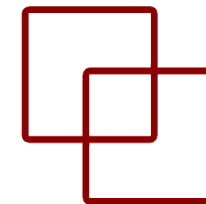
1. Ejemplo

Puerta lógica XOR



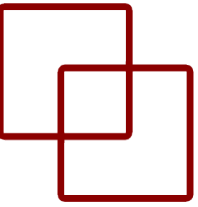
1. Ejemplo

Puerta lógica XOR

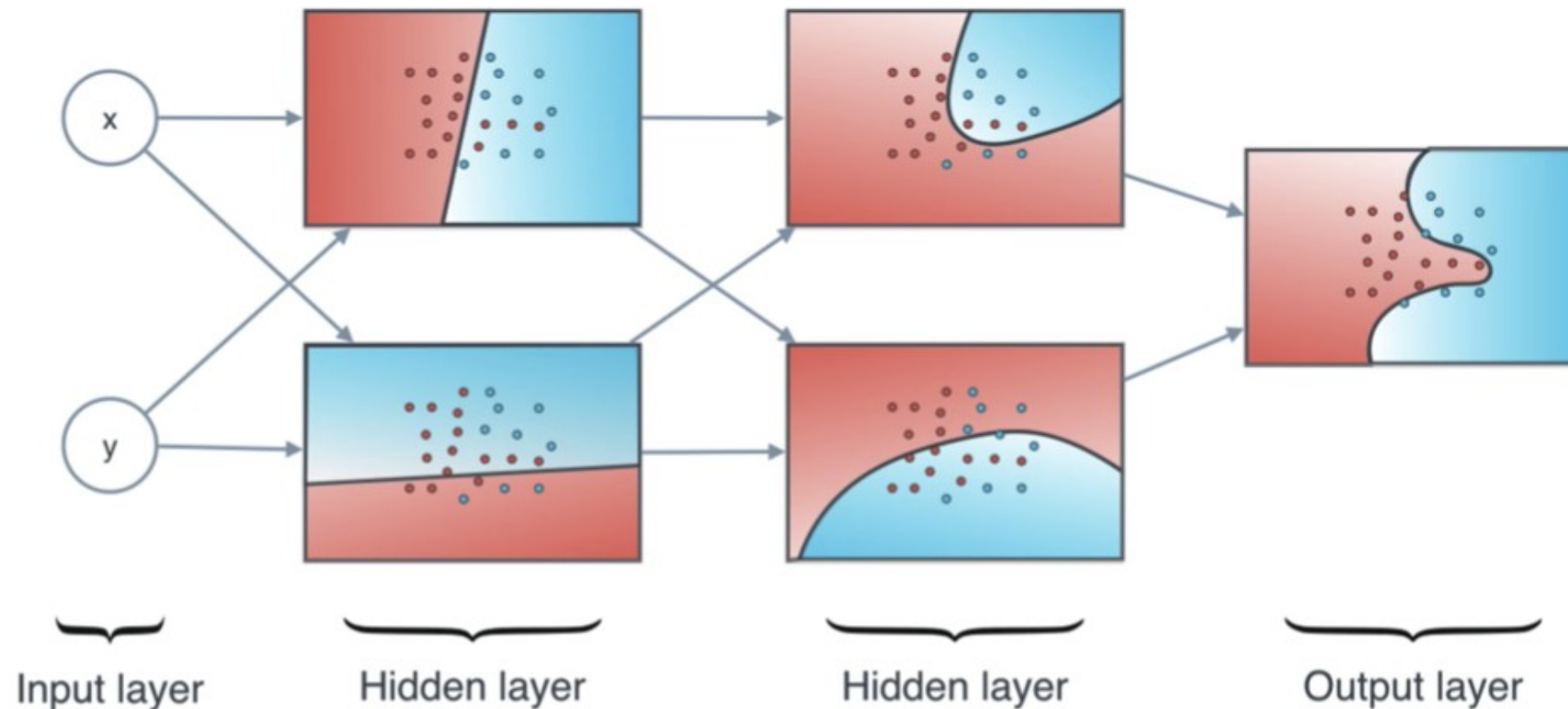


1. Ejemplo

Arquitectura de redes neuronales

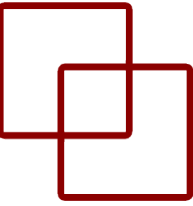


- Son la base de las **redes profundas**, actualmente muy utilizadas con mucho éxito.
- Cada **capa extrae características** cada vez más complejas





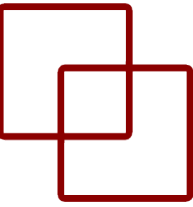
Perceptrón multicapa



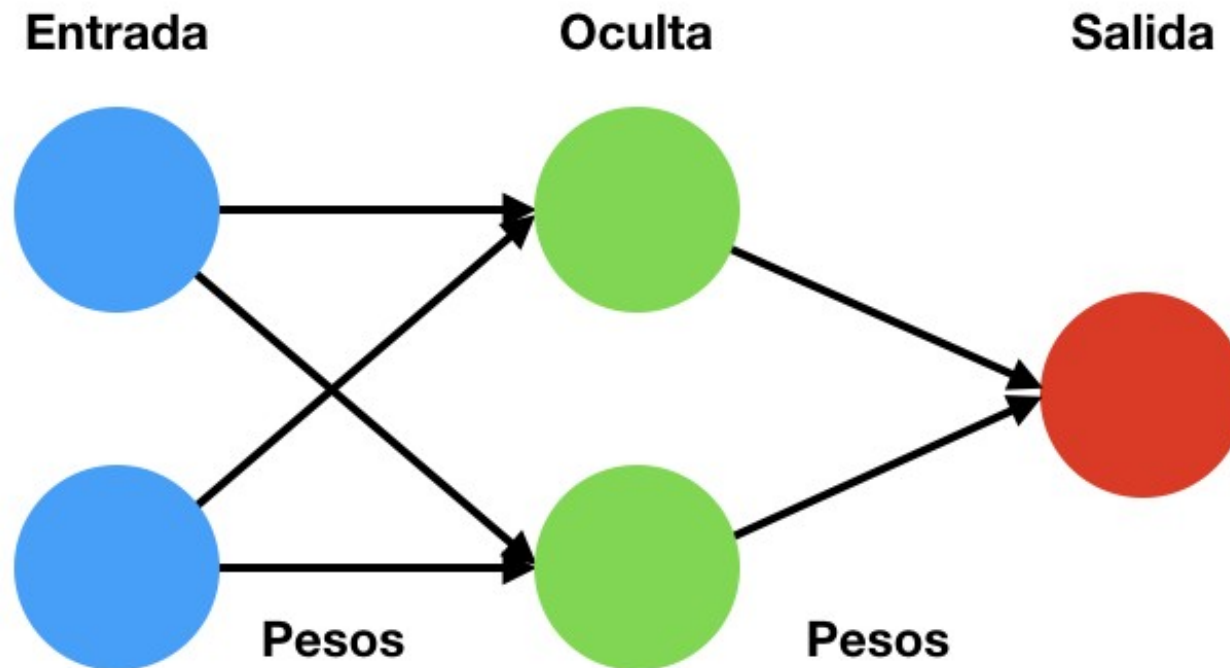
1. Definición

- Una neurona no puede representar toda la información
 - Problema de linealidad como XOR
- Necesitamos una agrupación de neuronas

1. Definición

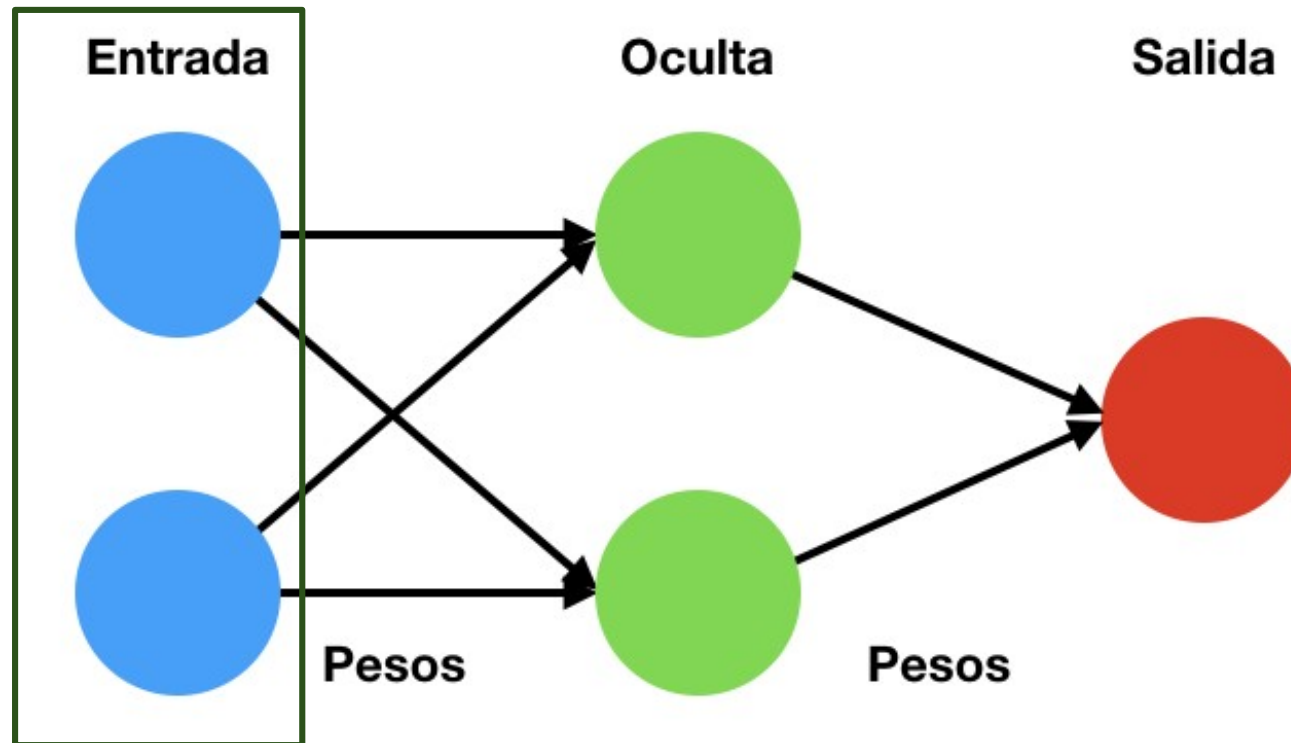


- Una neurona no puede representar toda la información
 - Problema de linealidad como XOR
- Necesitamos una agrupación de neuronas



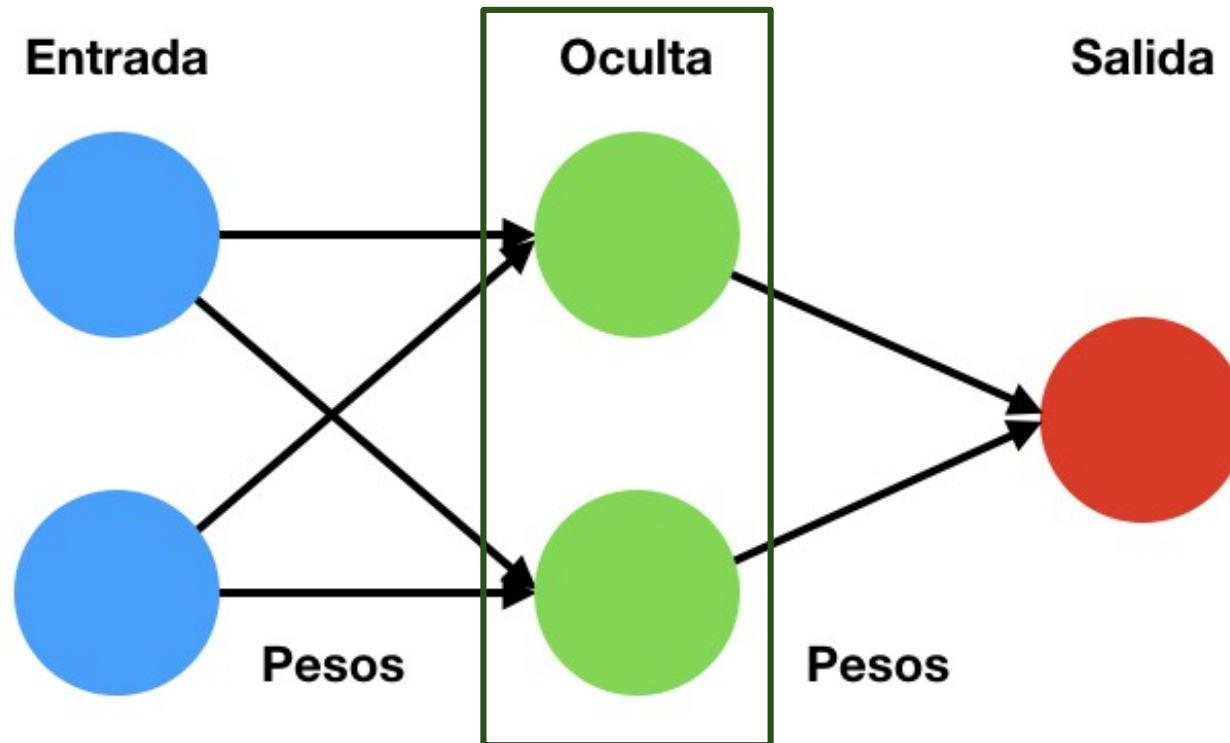
1. Definición

- Una neurona no puede representar toda la información
 - Problema de linealidad como XOR
- Necesitamos una agrupación de neuronas



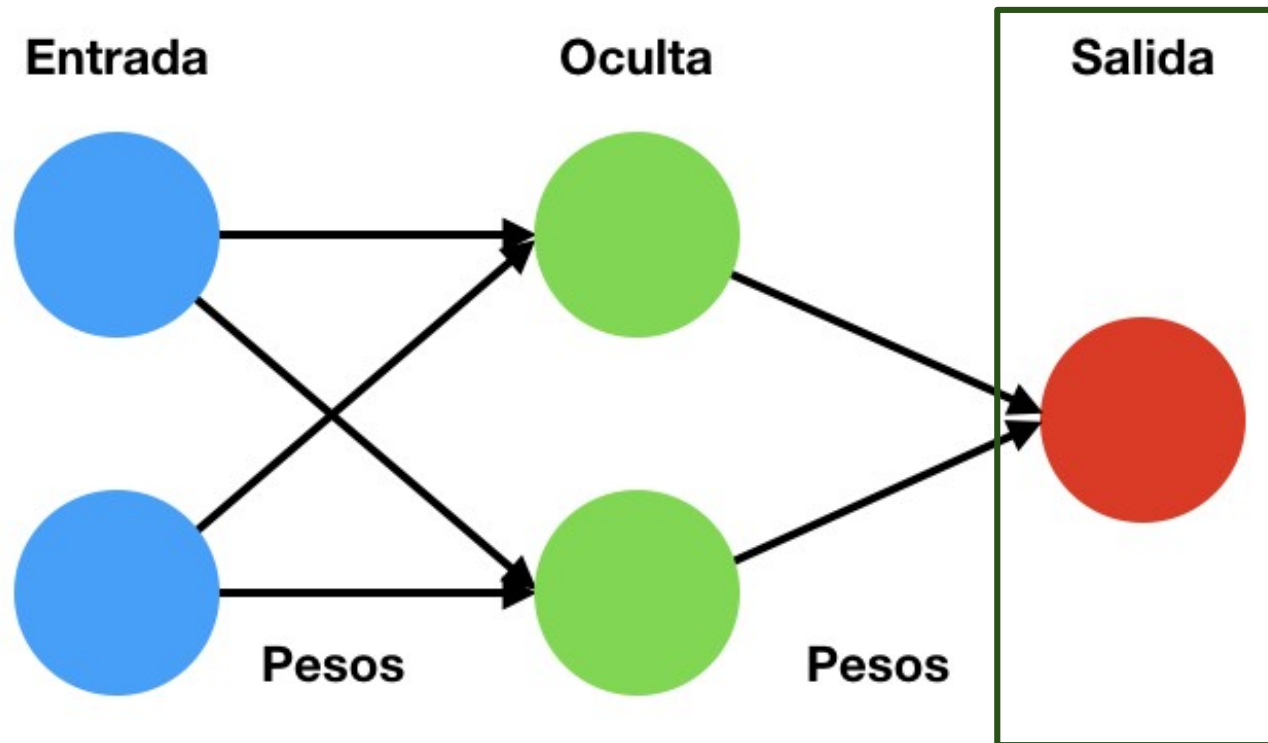
1. Definición

- Una neurona no puede representar toda la información
 - Problema de linealidad como XOR
- Necesitamos una agrupación de neuronas

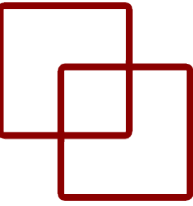


1. Definición

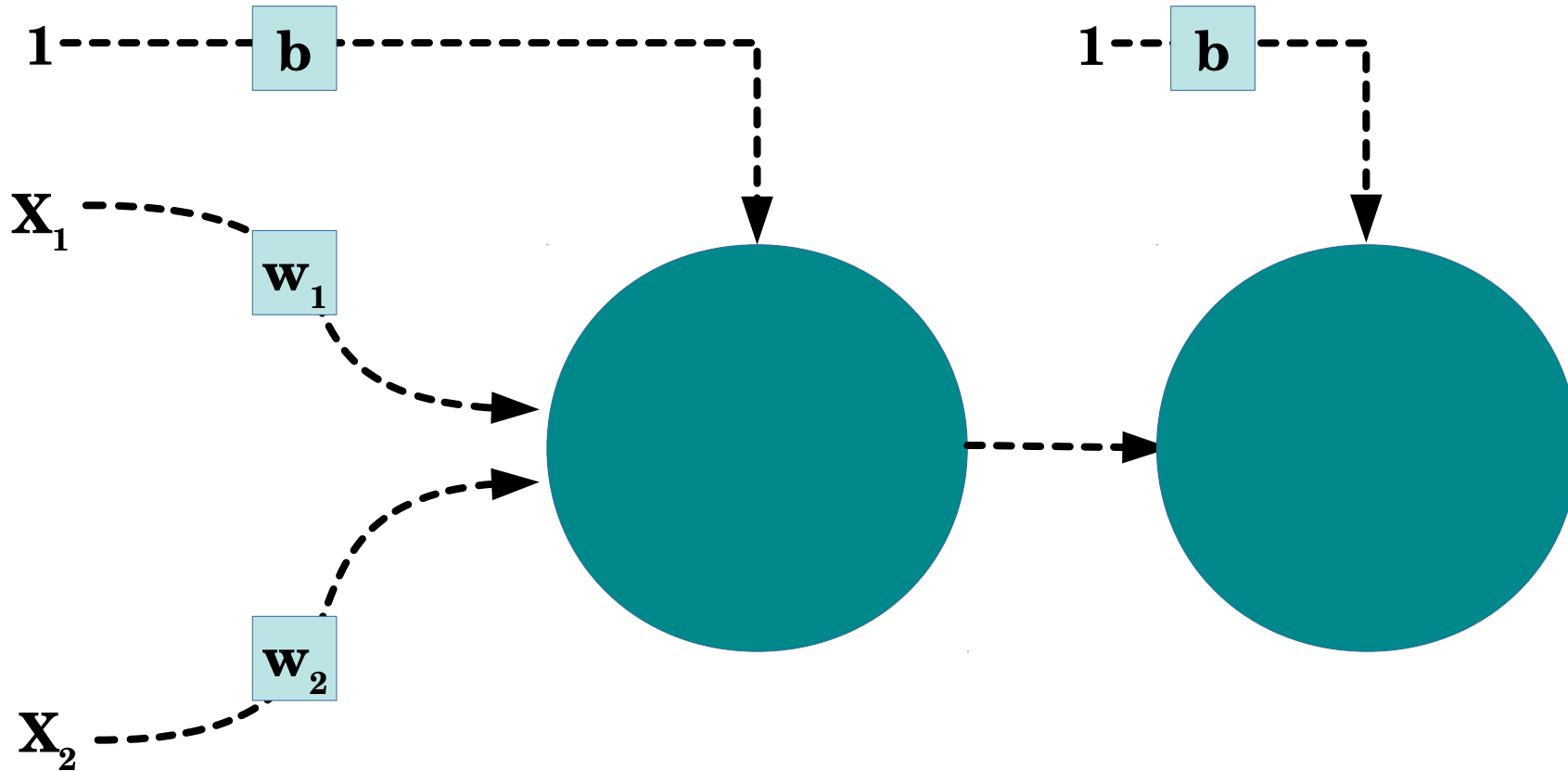
- Una neurona no puede representar toda la información
 - Problema de linealidad como XOR
- Necesitamos una agrupación de neuronas



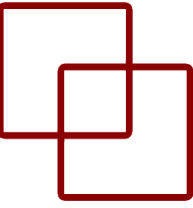
2. Secuencialidad de neuronas



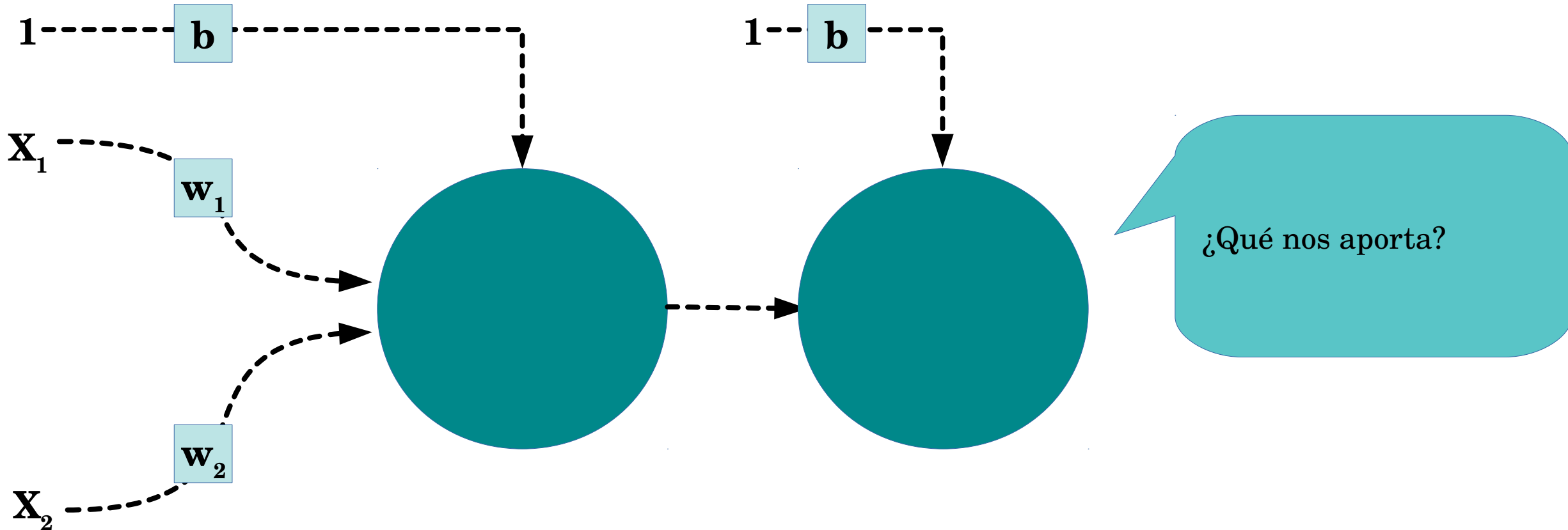
- Si ponemos neuronas de forma secuencial



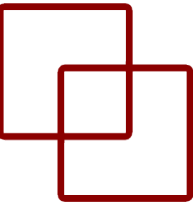
2. Secuencialidad de neuronas



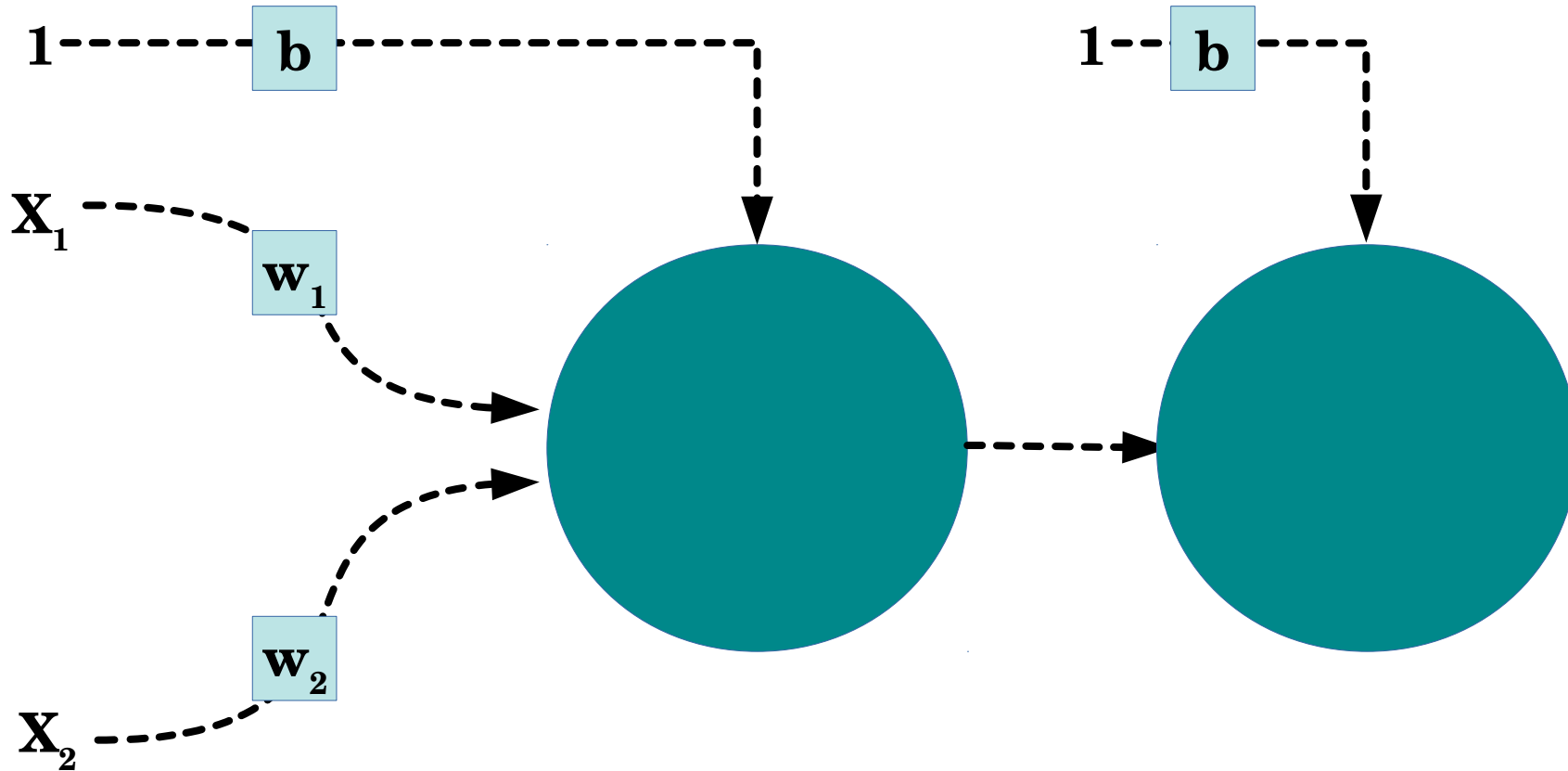
- Si ponemos neuronas de forma secuencial



2. Secuencialidad de neuronas

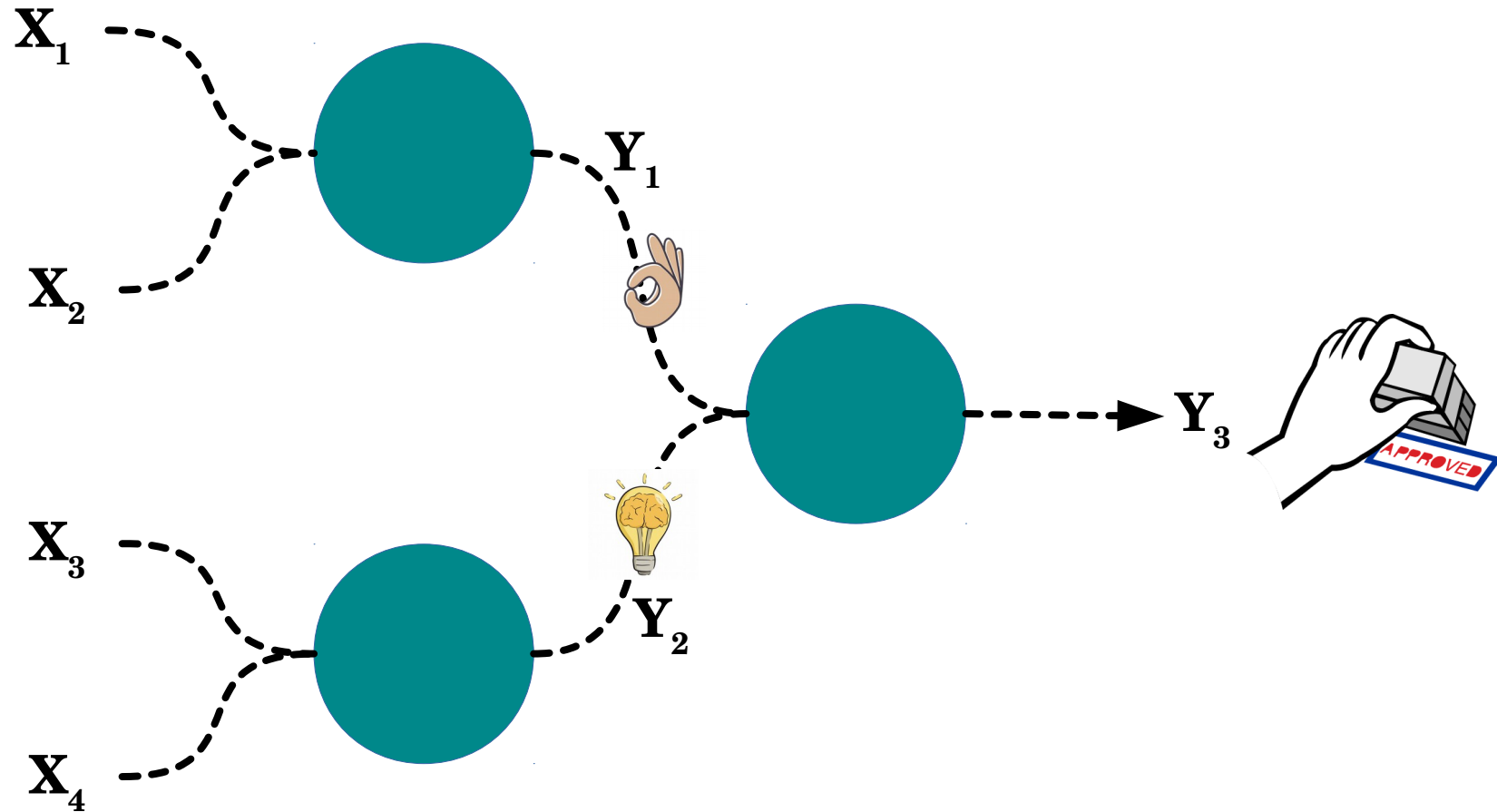
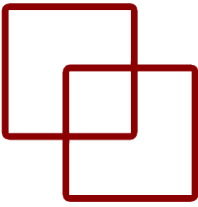


- Si ponemos neuronas de forma secuencial

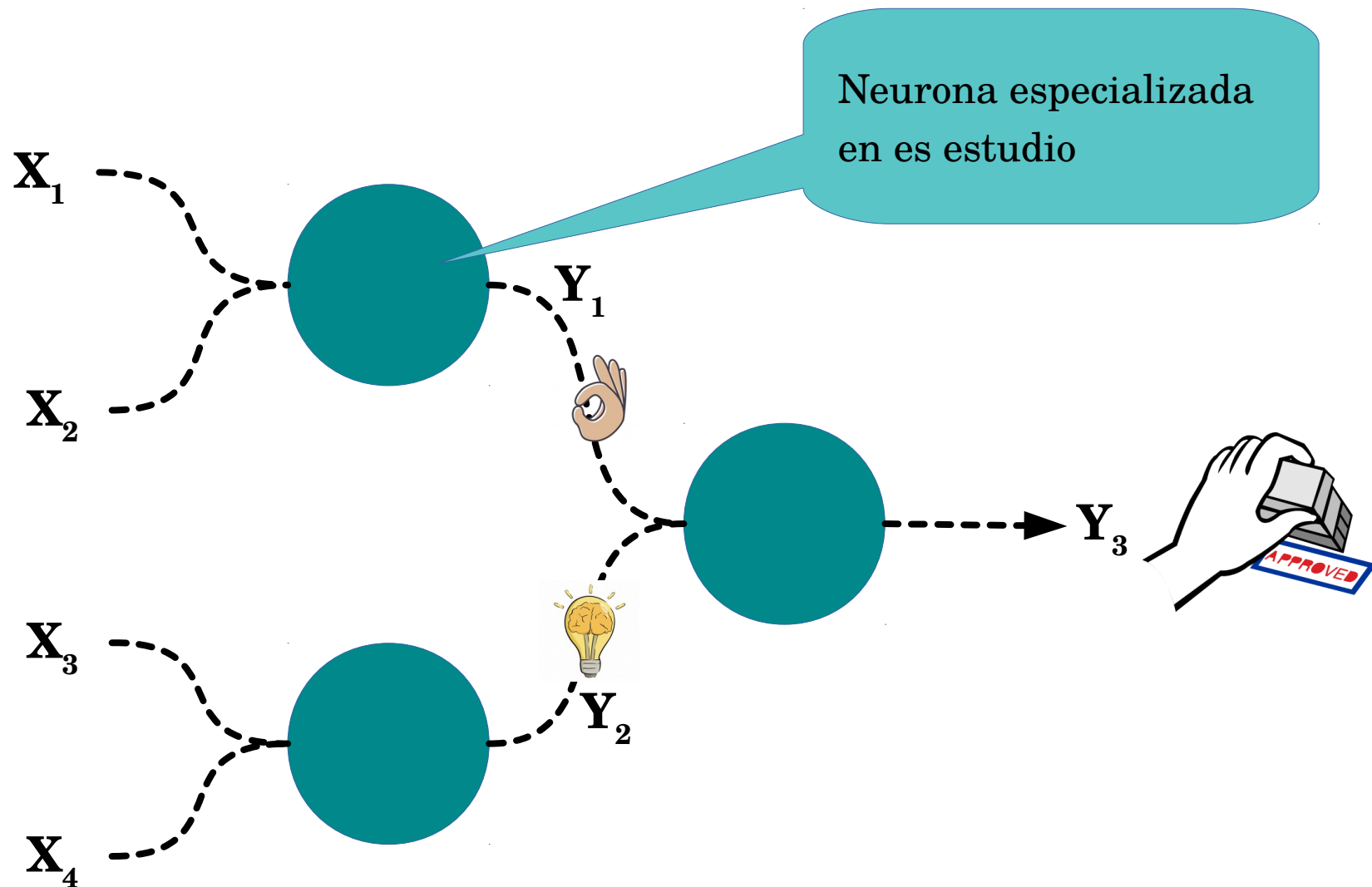
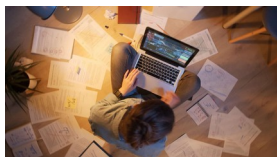
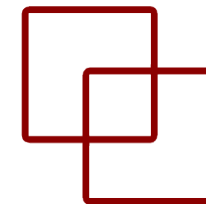


¿Qué nos aporta?
**Conocimiento
jerarquizado**

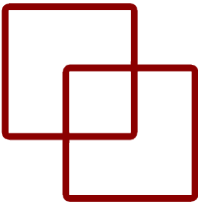
3. Conocimiento Jerarquizado



3. Conocimiento Jerarquizado



3. Conocimiento Jerarquizado

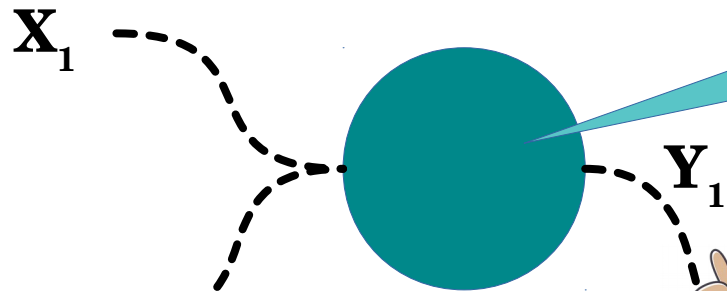


X_1

X_2

X_3

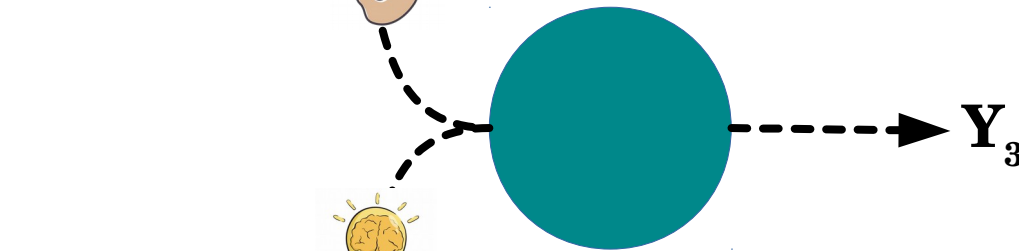
X_4



Neurona especializada
en el estudio



Y_2

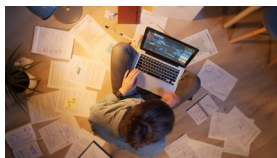
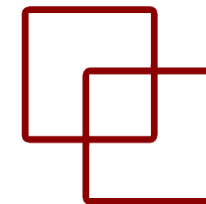


Y_3



Neurona especializada
en la motivación

3. Conocimiento Jerarquizado



X_1



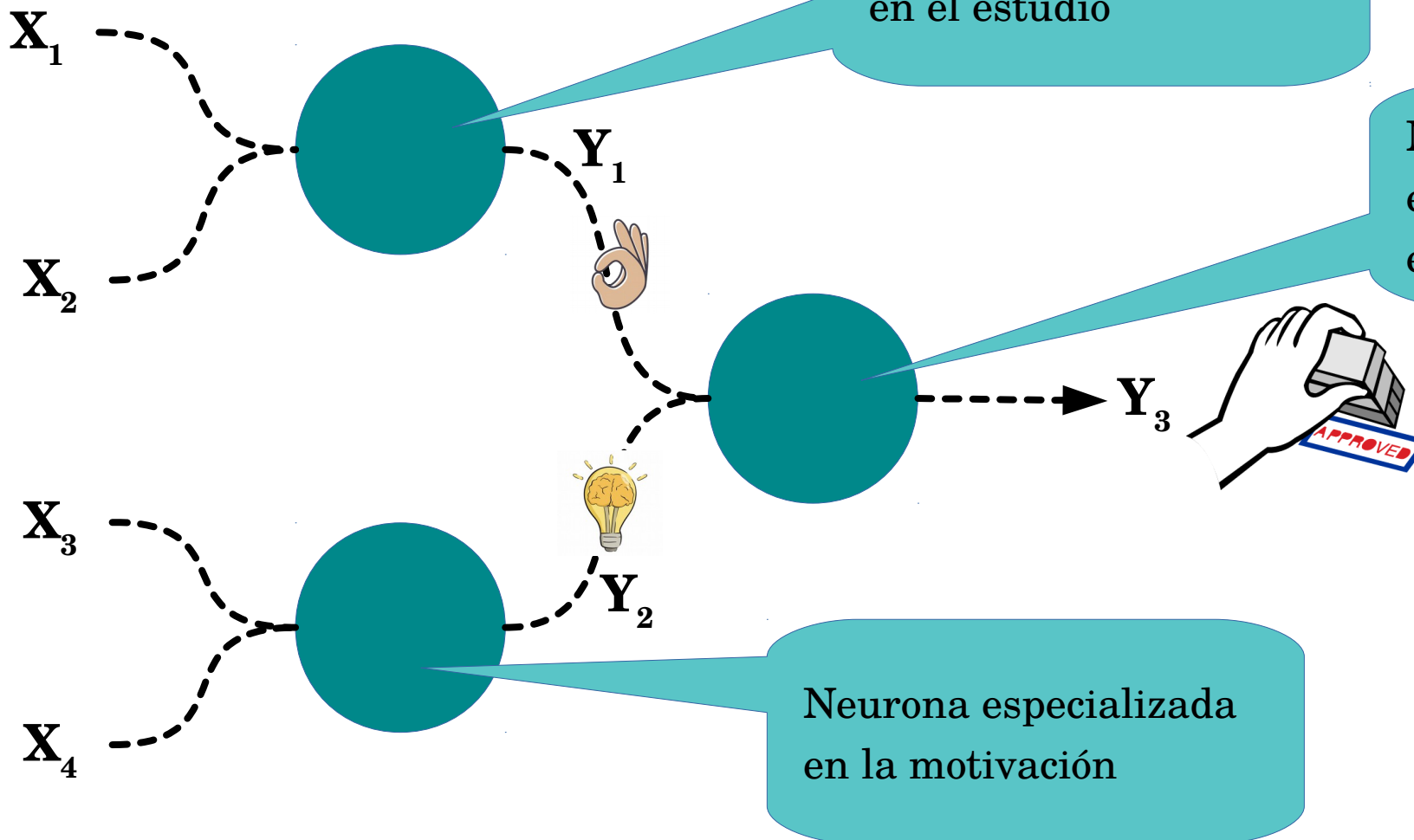
X_2



X_3

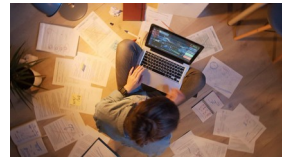
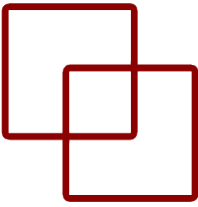


X_4



3. Conocimiento Jerarquizado

Ejemplo



X_1



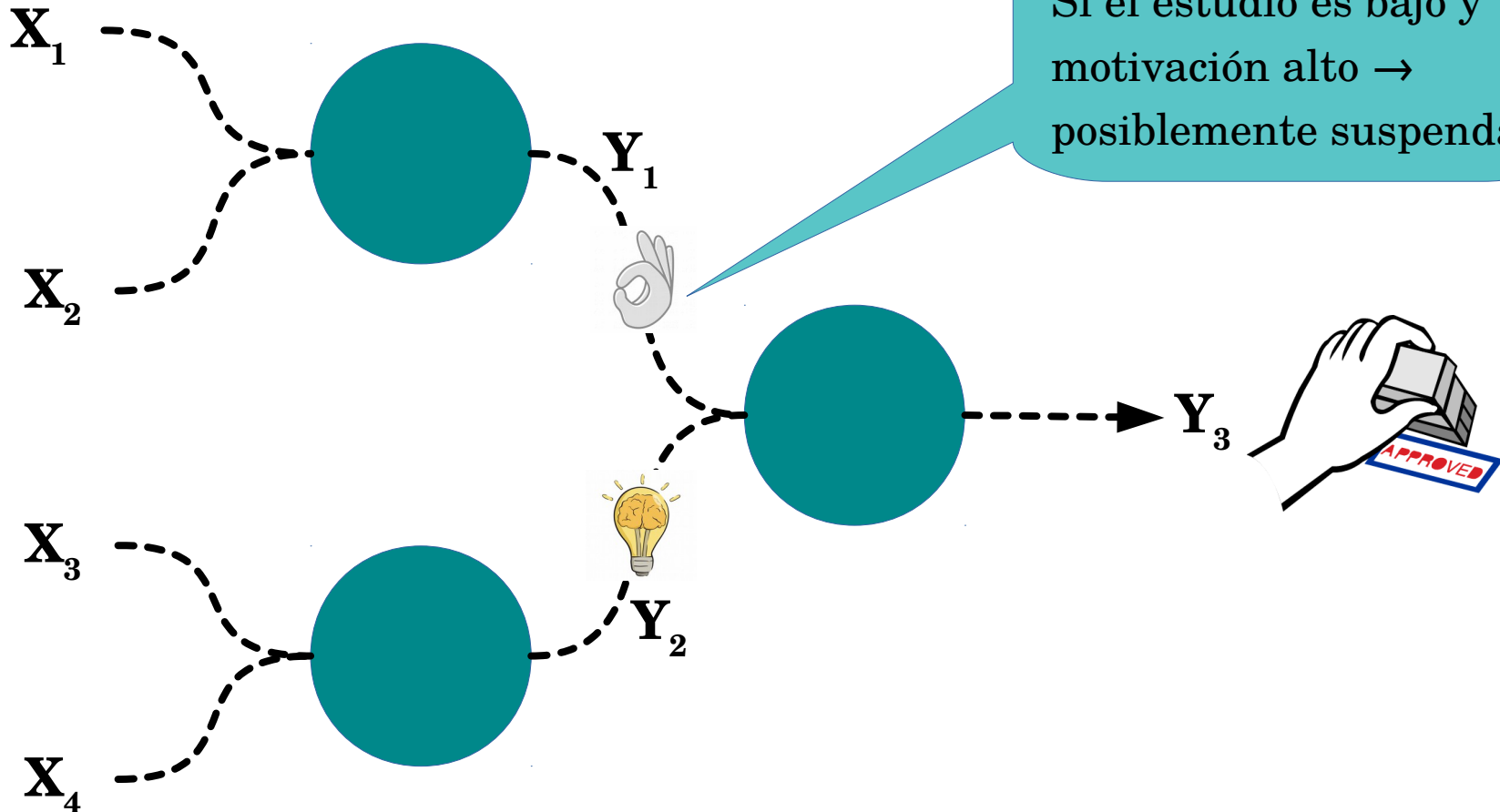
X_2



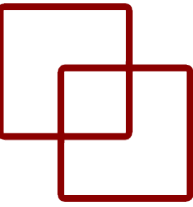
X_3



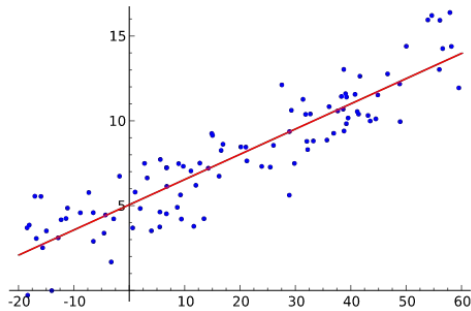
X_4



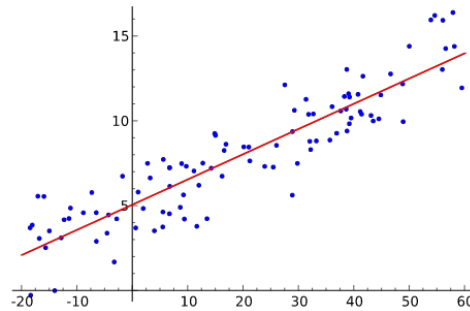
4. Problema



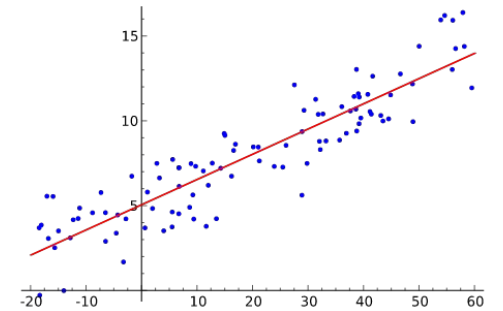
- El problema es que la concatenación de regresiones lineales es igual a una regresión lineal



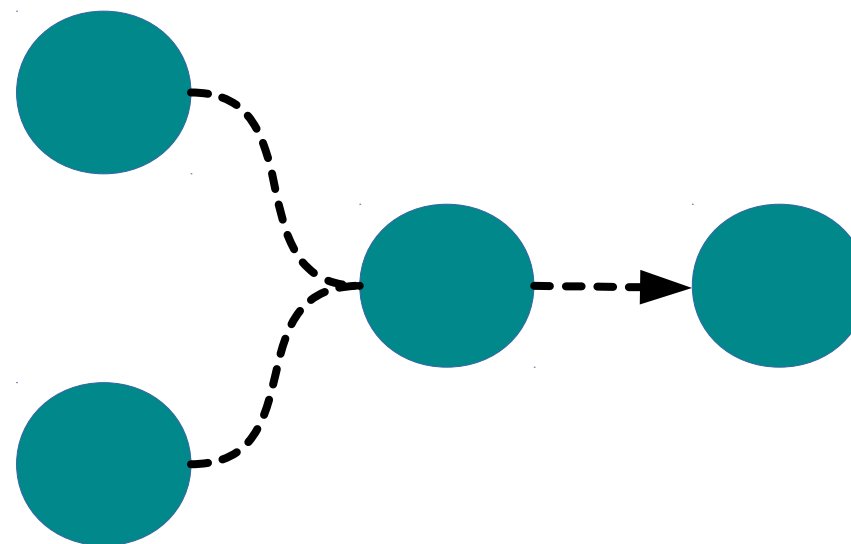
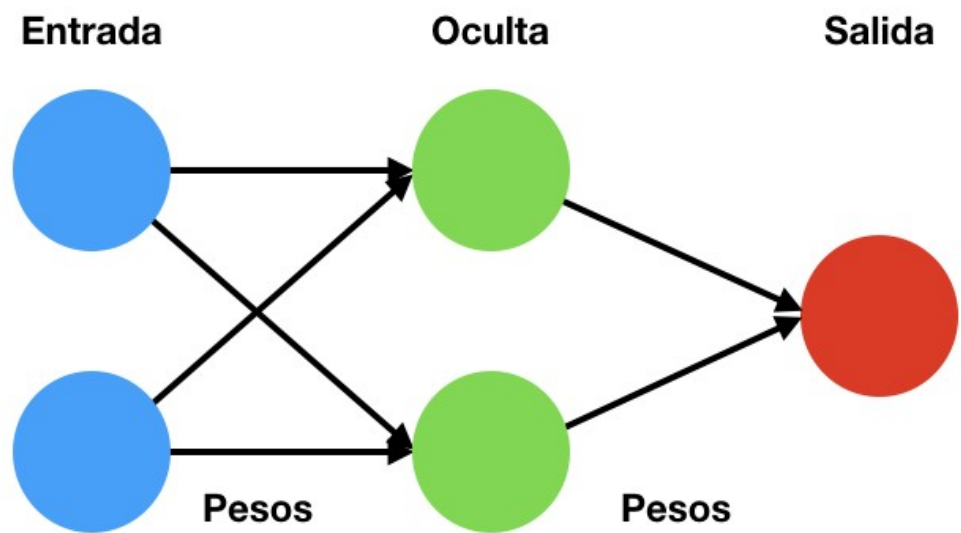
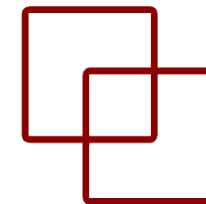
+ ... +



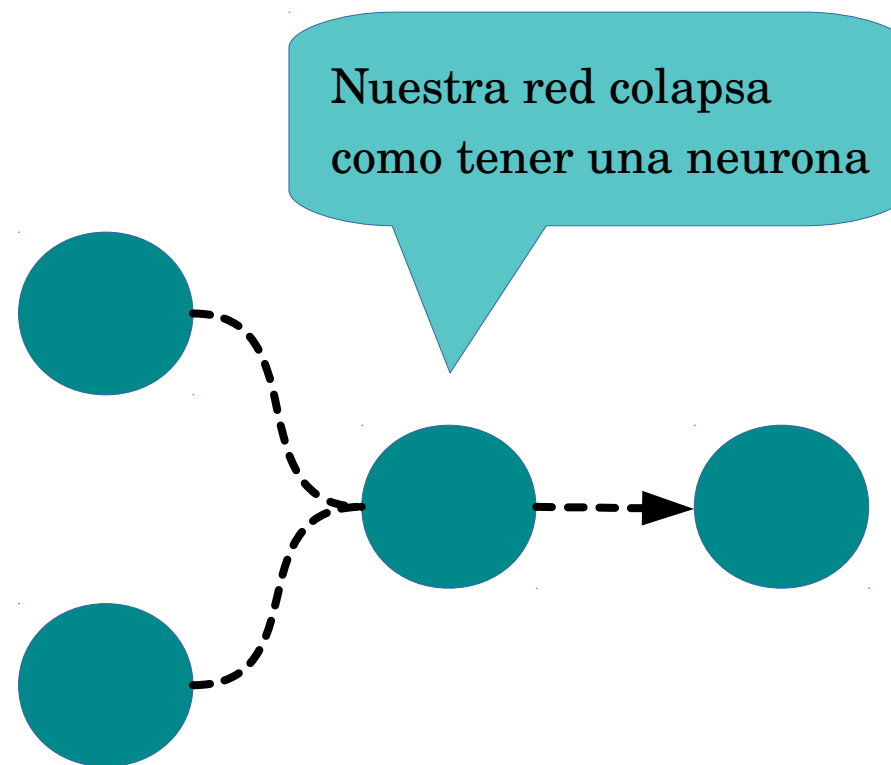
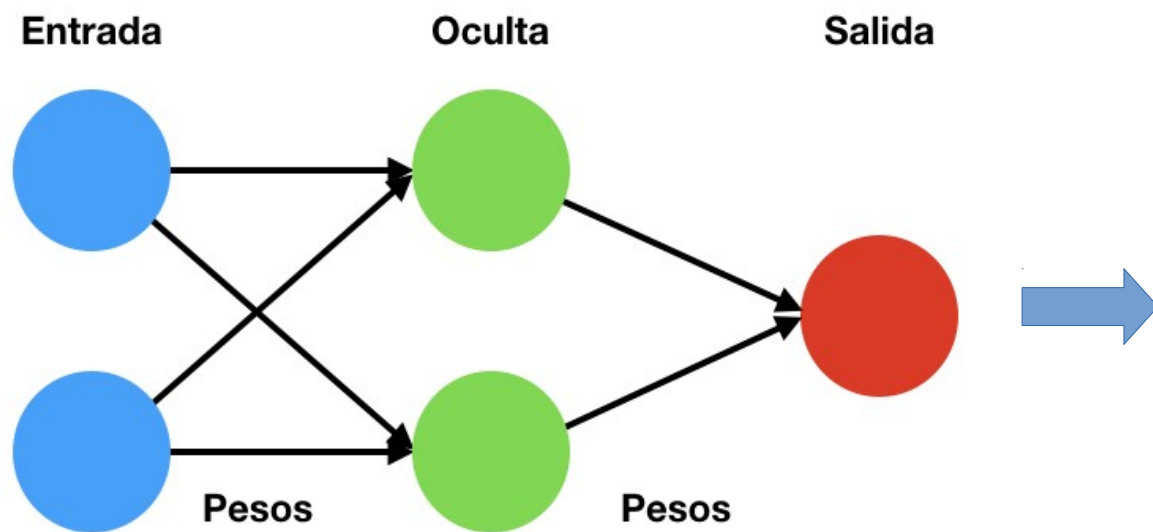
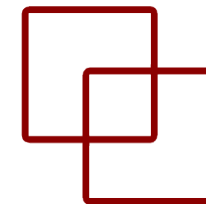
=



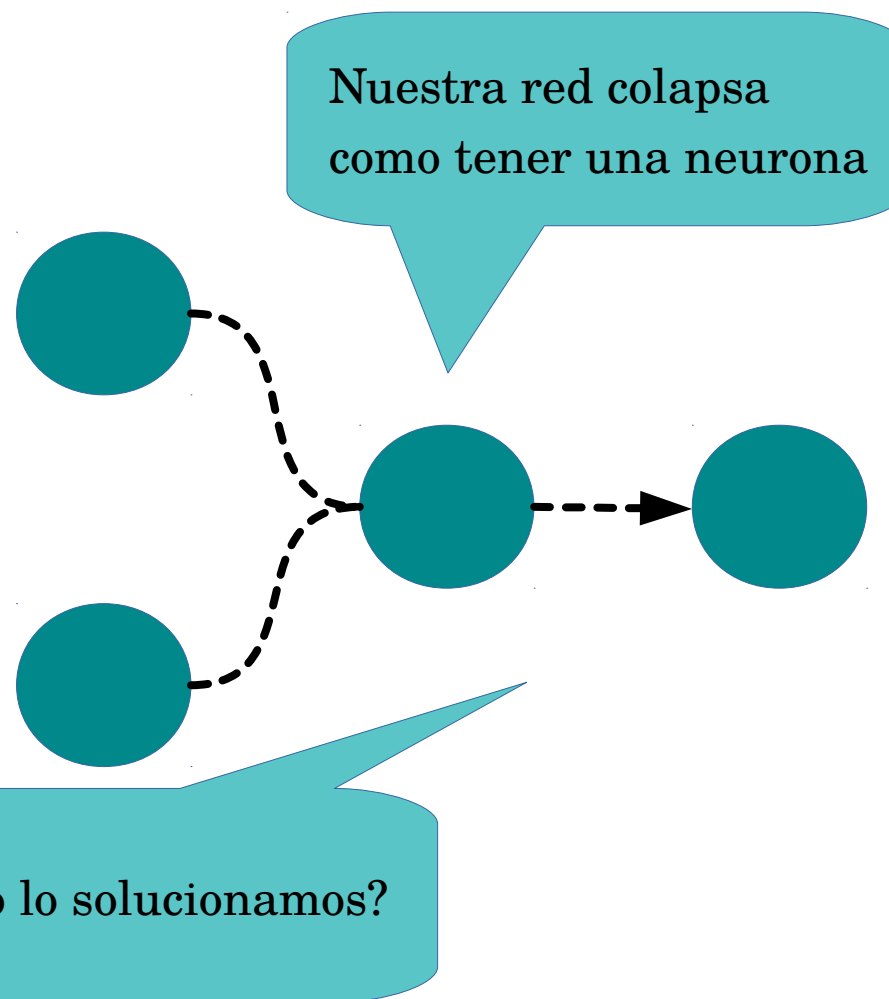
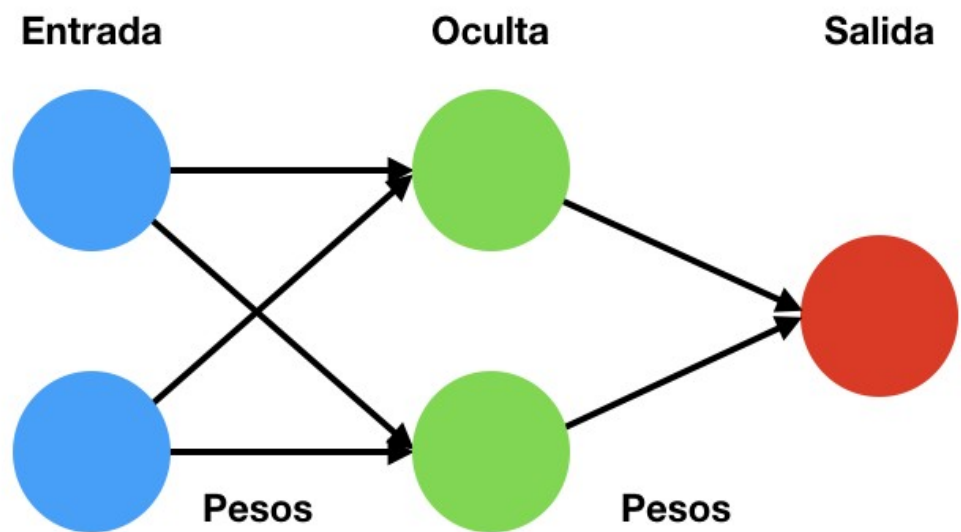
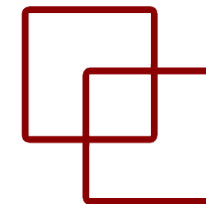
4. Problema



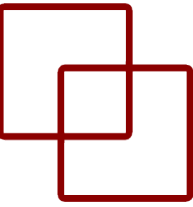
4. Problema



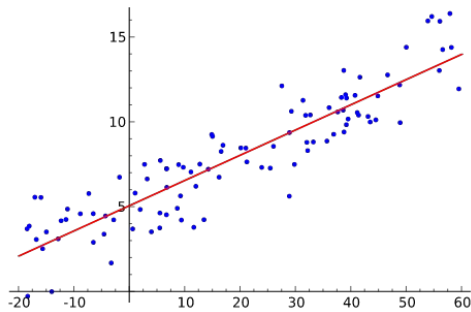
4. Problema



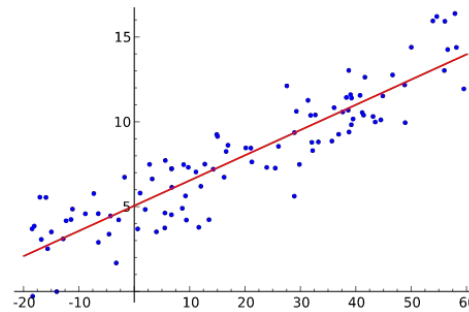
4. Problema



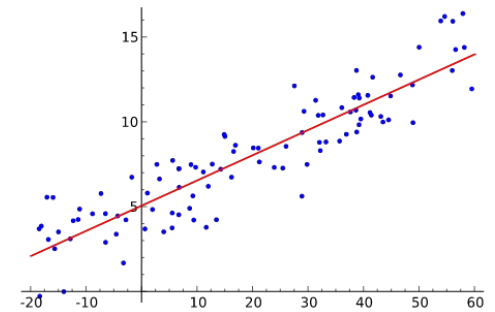
- Que nuestras funciones sufran una distorsión para romper la linealidad



+ ... +

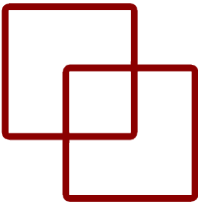


=

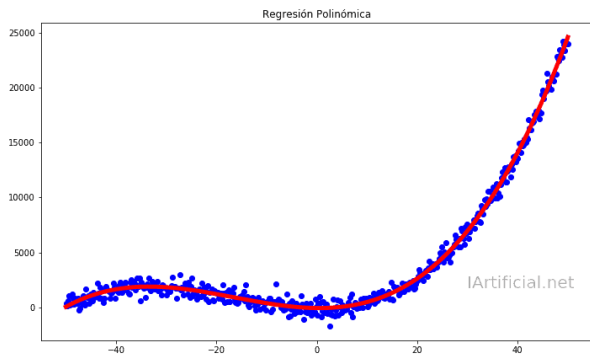


¿Entonces distorsionamos las funciones?

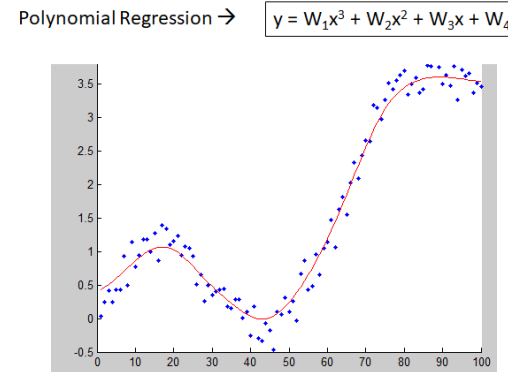
4. Problema



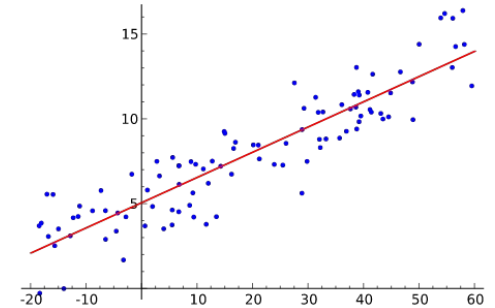
- Que nuestras funciones sufran una distorsión para romper la linealidad



+ ... +

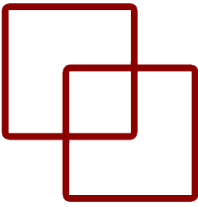


=/=

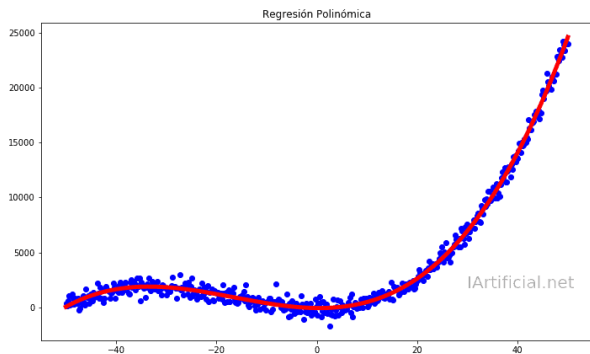


Así el resultado **no es Lineal**

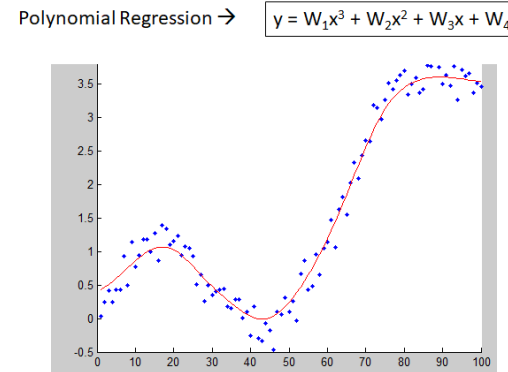
4. Problema



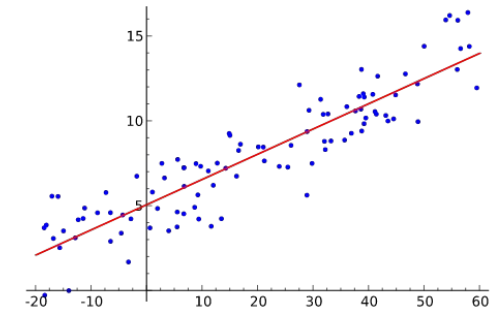
- Que nuestras funciones sufran una distorsión para romper la linealidad



+ ... +

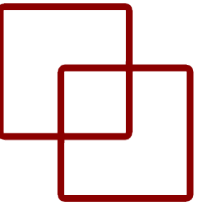


=/=

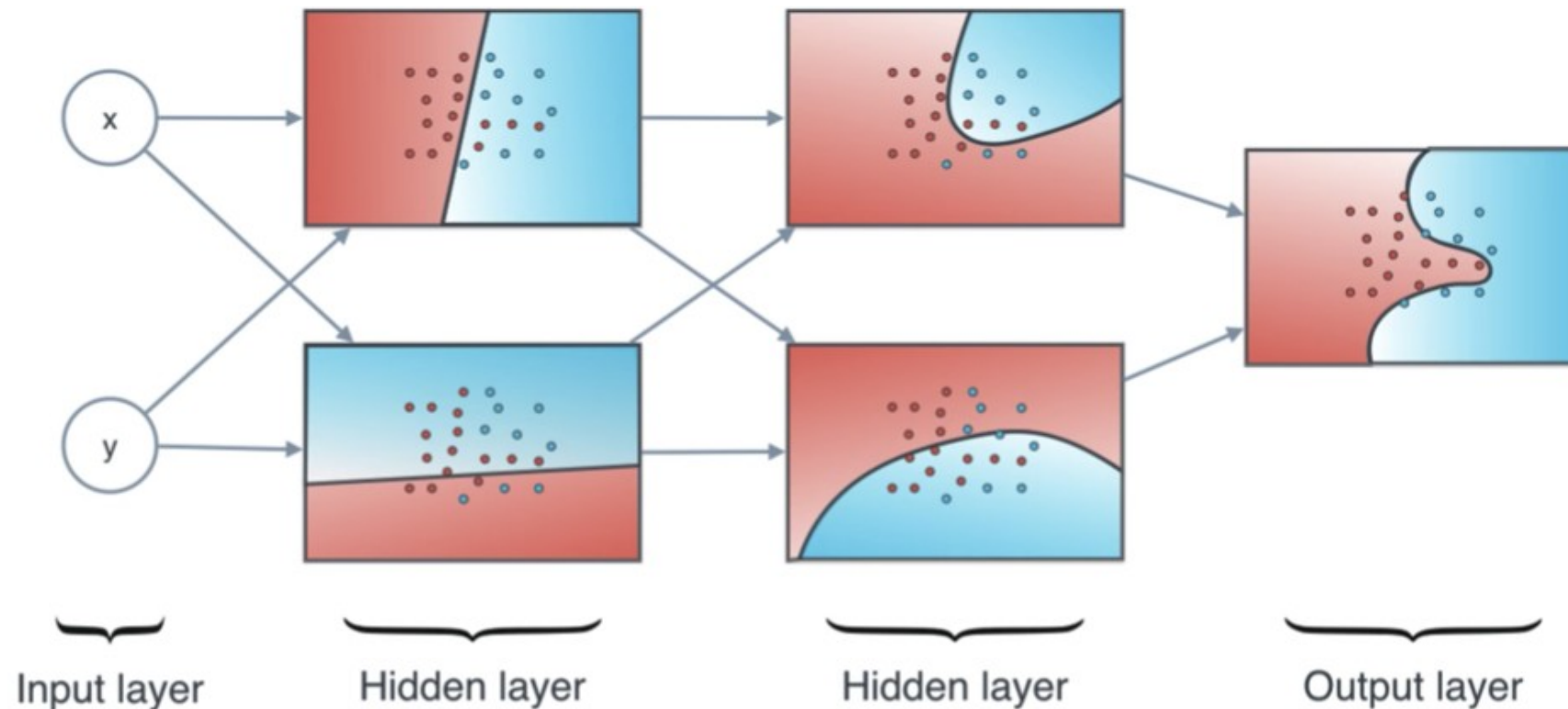


Añadimos **funciones de activación**

5. Arquitectura de redes neuronales



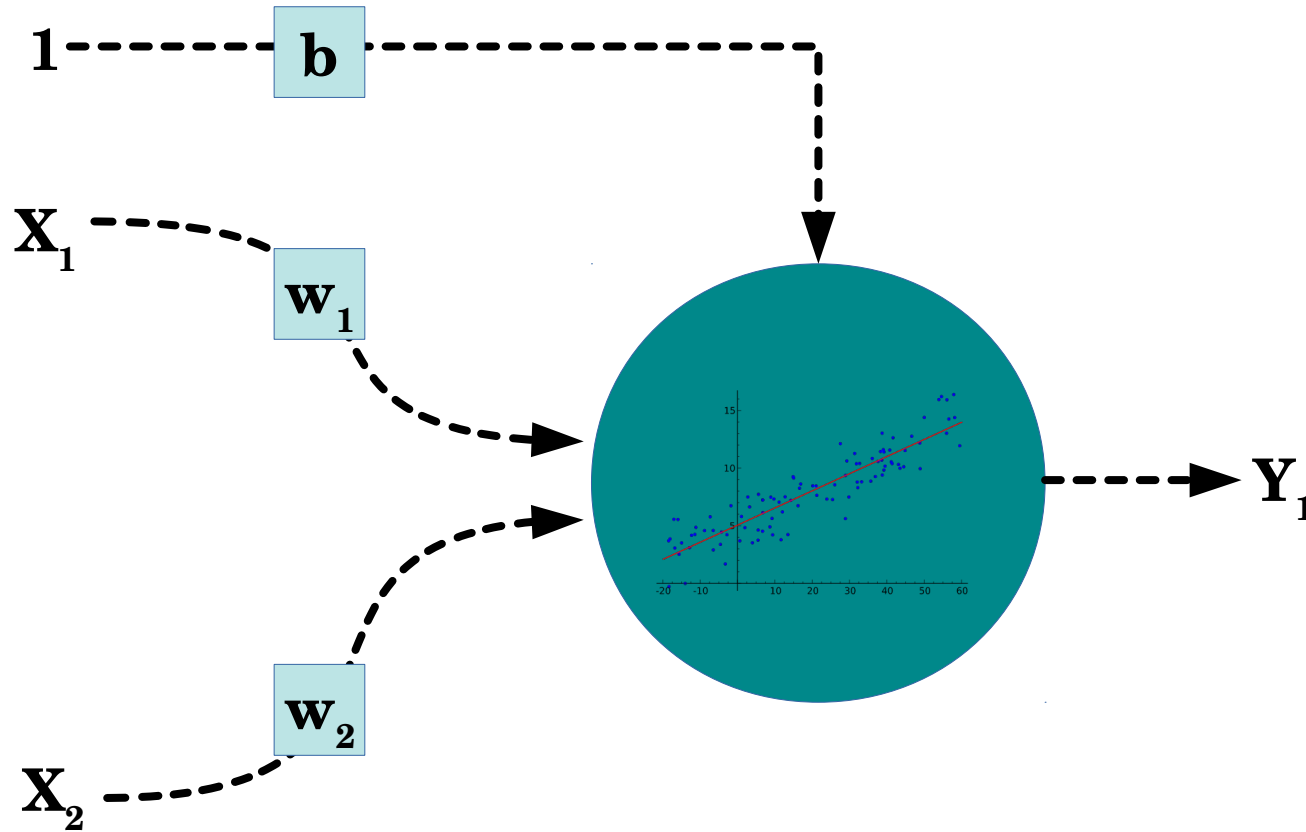
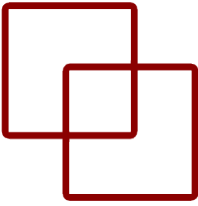
- Son la base de las **redes profundas**, actualmente muy utilizadas con mucho éxito.
- Cada **capa extrae características** cada vez más complejas





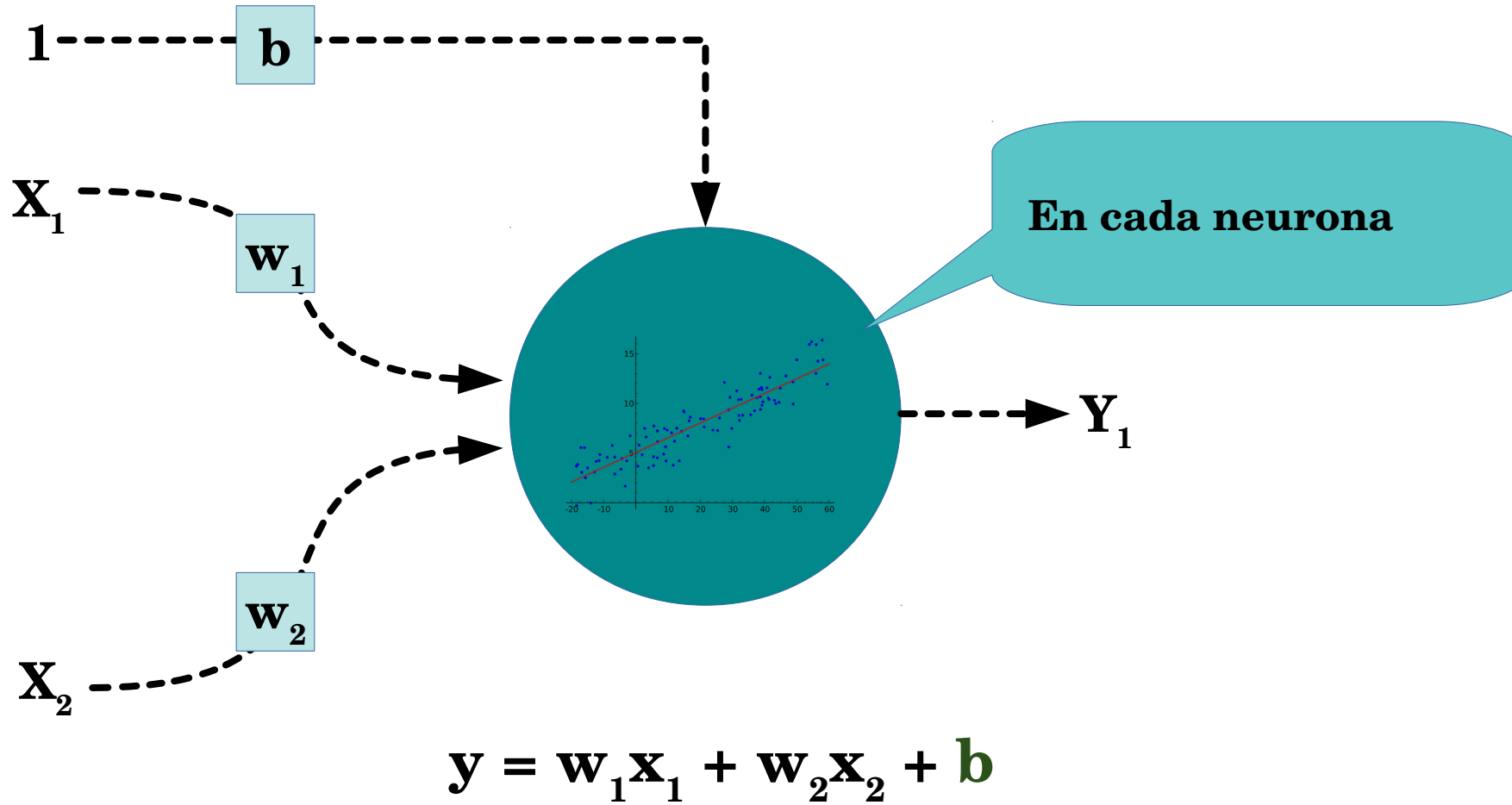
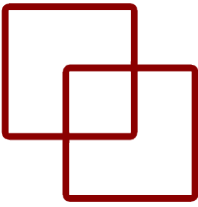
Función de activación

1. ¿Donde se encuentra?

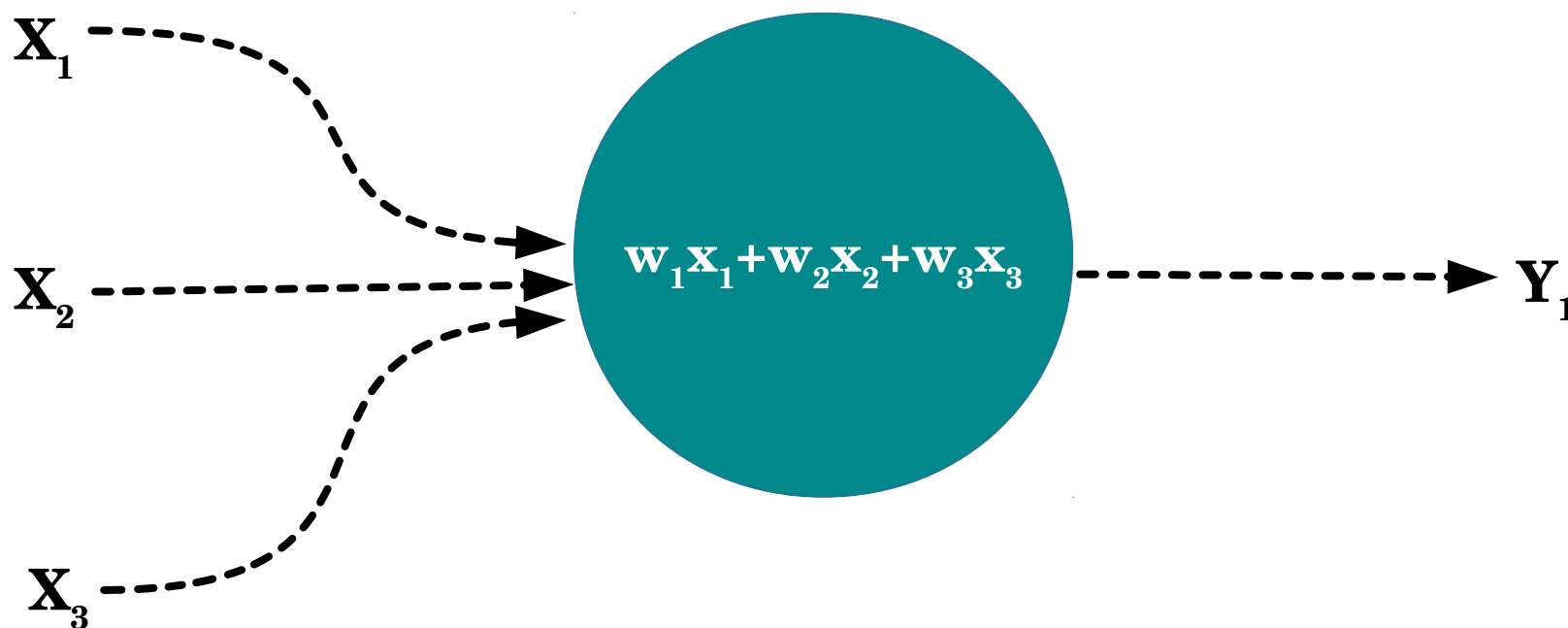
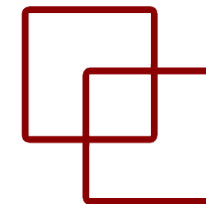


$$y = w_1 x_1 + w_2 x_2 + b$$

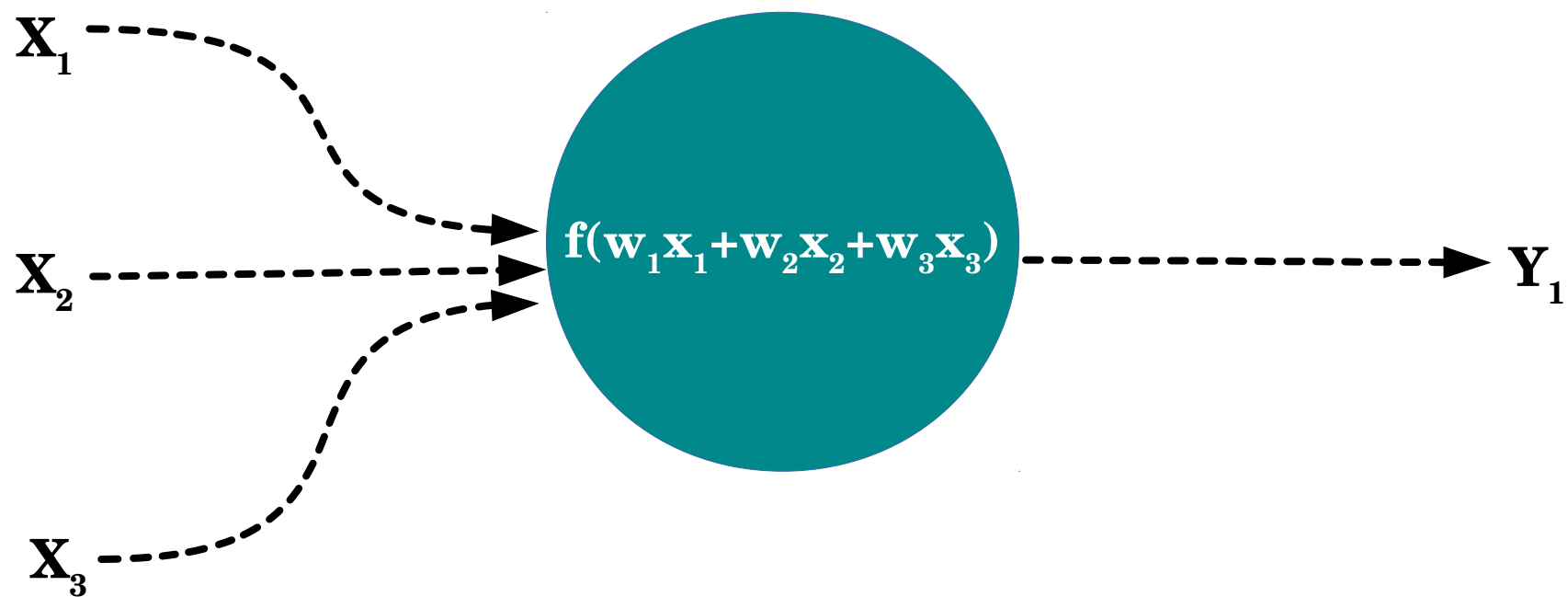
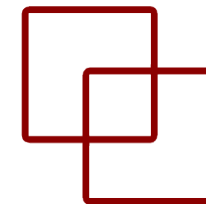
1. ¿Donde se encuentra?



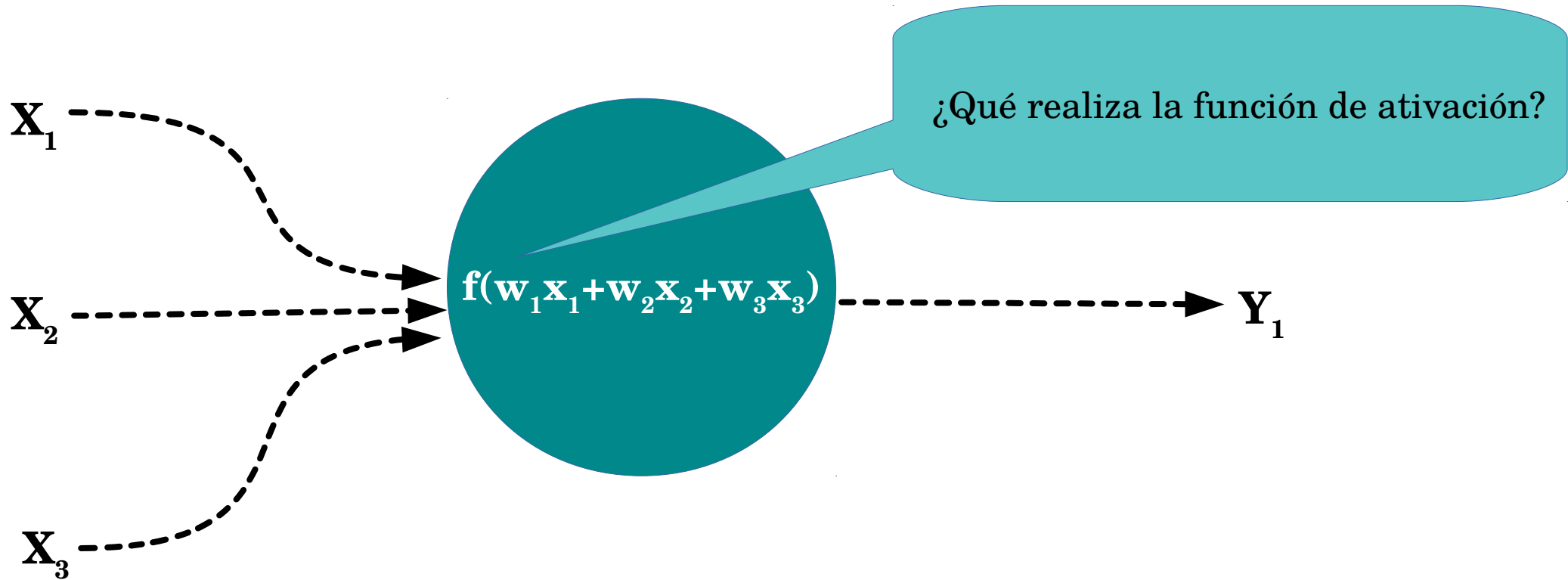
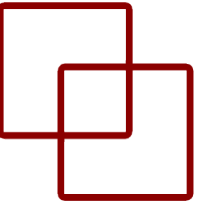
2. ¿Como la representamos?



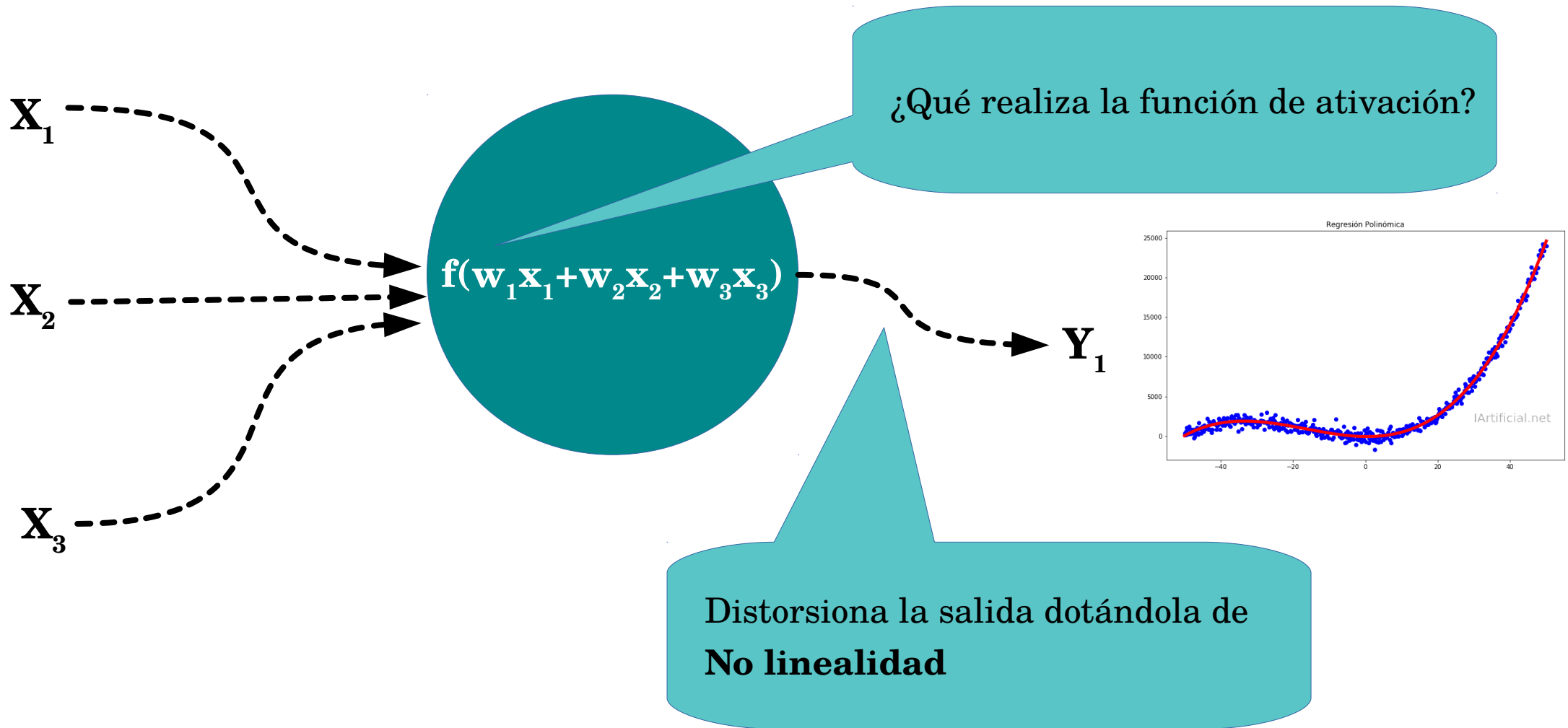
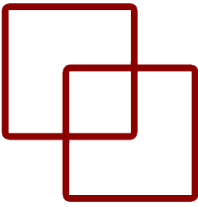
2. ¿Como la representamos?



2. ¿Como la representamos?

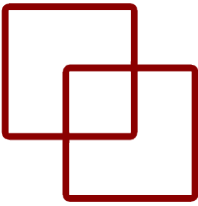


2. ¿Como la representamos?



3. Recordemos la que vimos

Función de activación Binaria



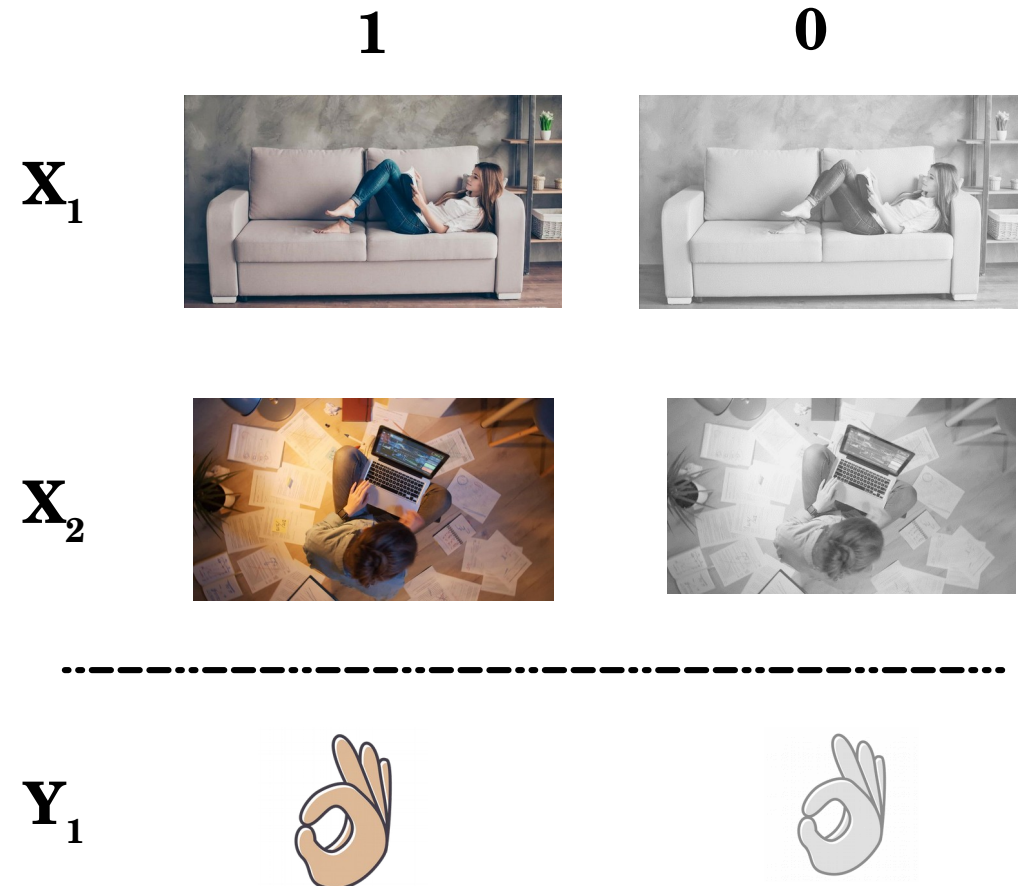
Estudiar + Estar en casa = Aprobar el examen

$$WX \leq \text{UMBRAL} \rightarrow Y = 0$$

$$WX > \text{UMBRAL} \rightarrow Y = 1$$

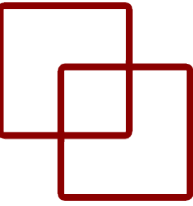
.....

$$\text{BIAS} = -\text{UMBRAL}$$



3. Recordemos la que vimos

Función de activación Binaria



$$WX \leq \text{UMBRAL} \rightarrow Y = 0$$

$$WX > \text{UMBRAL} \rightarrow Y = 1$$



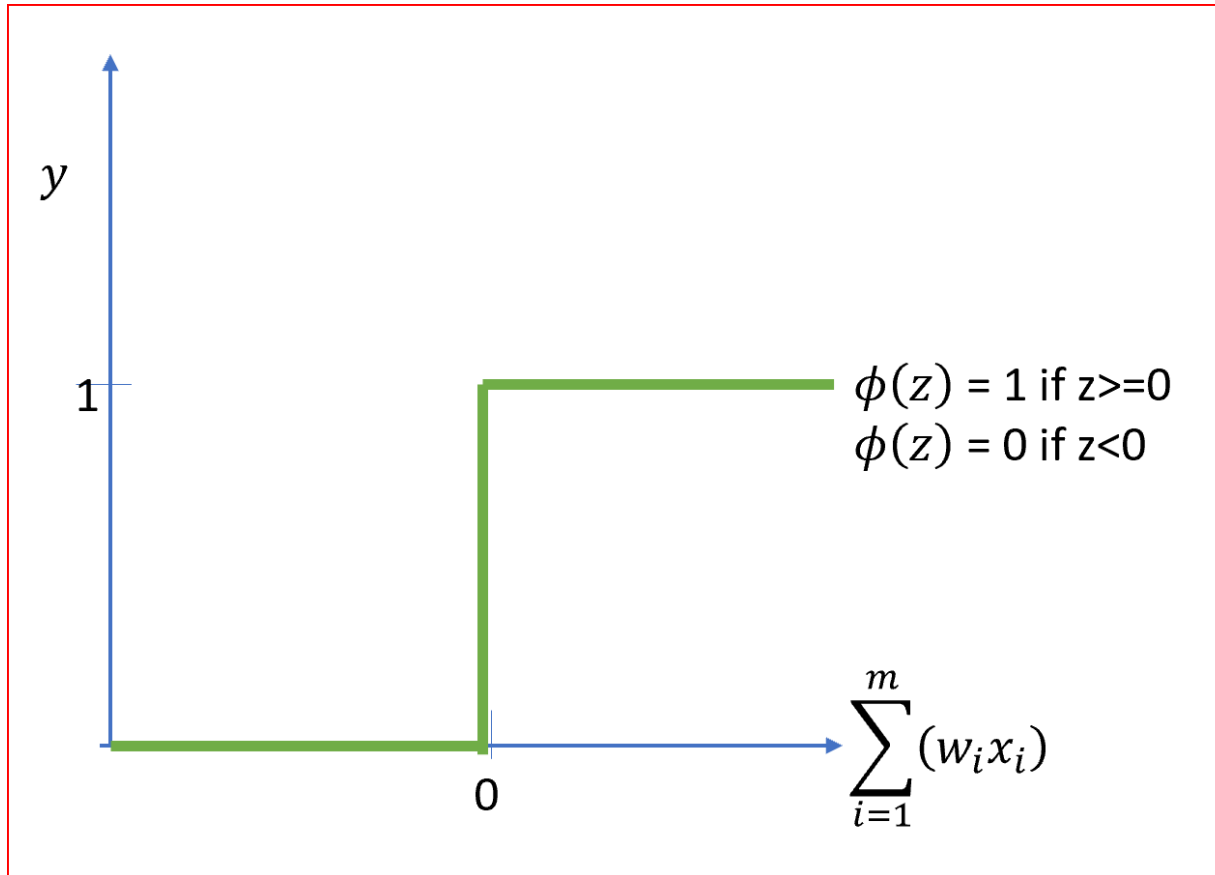
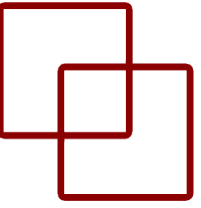
$$f(WX) \rightarrow Y = \{0, 1\}$$

.....

$$\text{BIAS} = -\text{UMBRAL}$$

3. Recordemos la que vimos

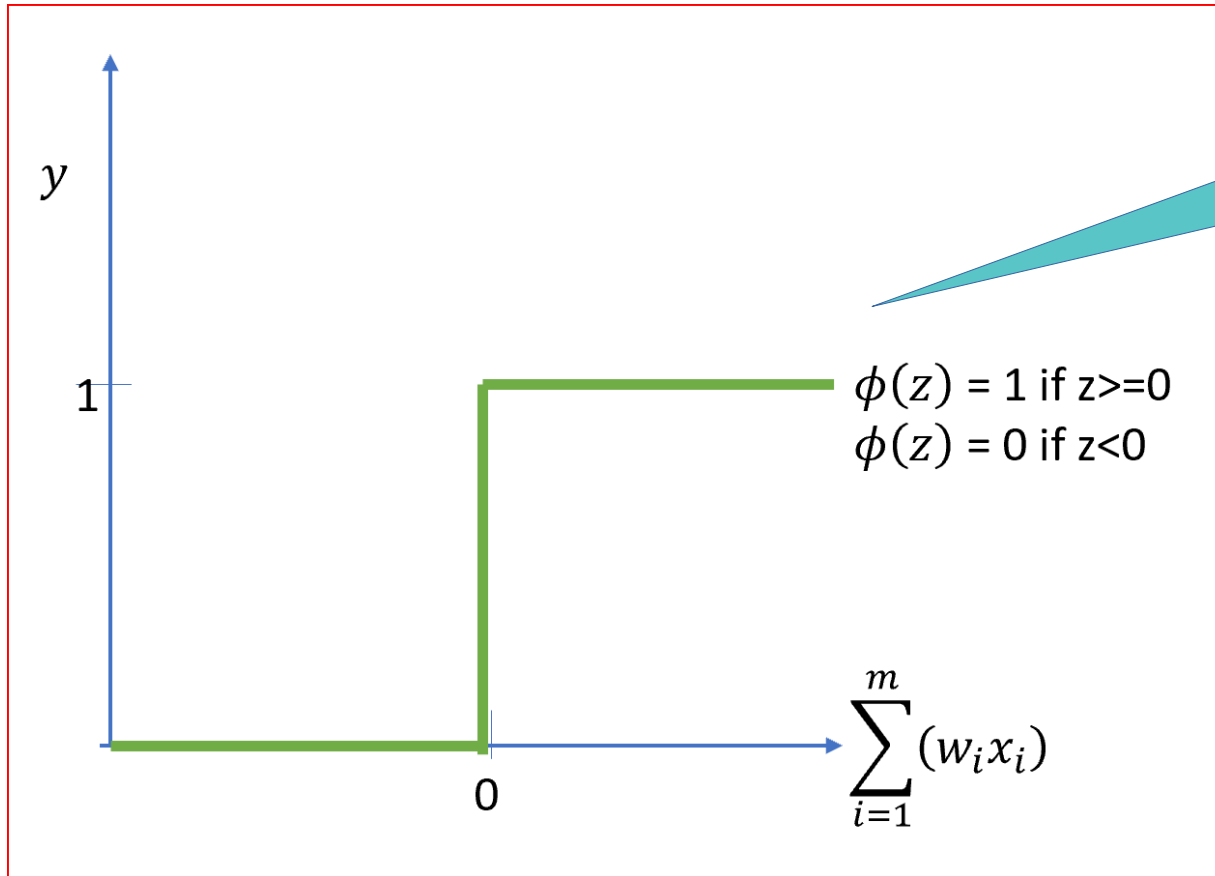
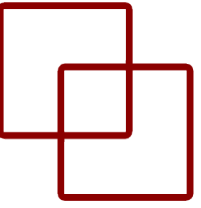
Función de activación Binaria



← **$f(WX) \rightarrow Y = \{0, 1\}$**

3. Recordemos la que vimos

Función de activación Binaria

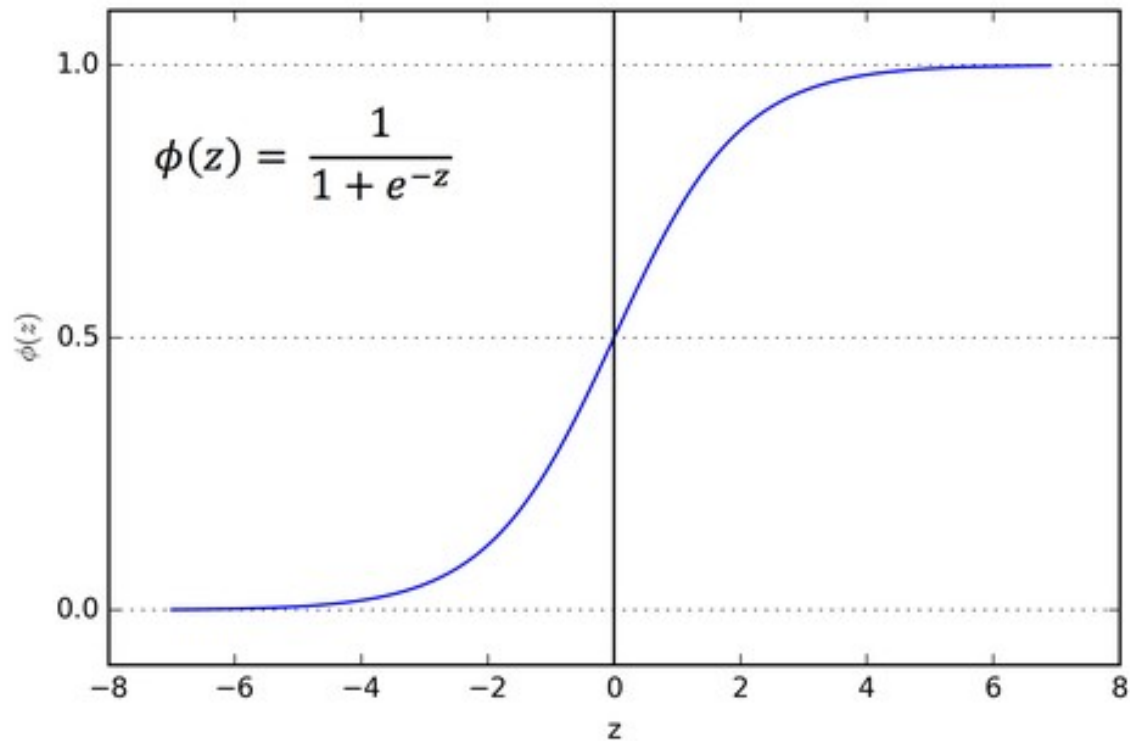
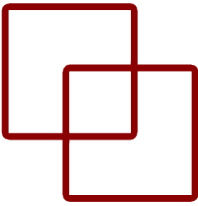


Dependiendo del umbral es 0 ó 1
Problema lineal $\rightarrow$ no nos interesa

$\leftarrow \mathbf{f(WX)} \rightarrow \mathbf{Y = \{0, 1\}}$

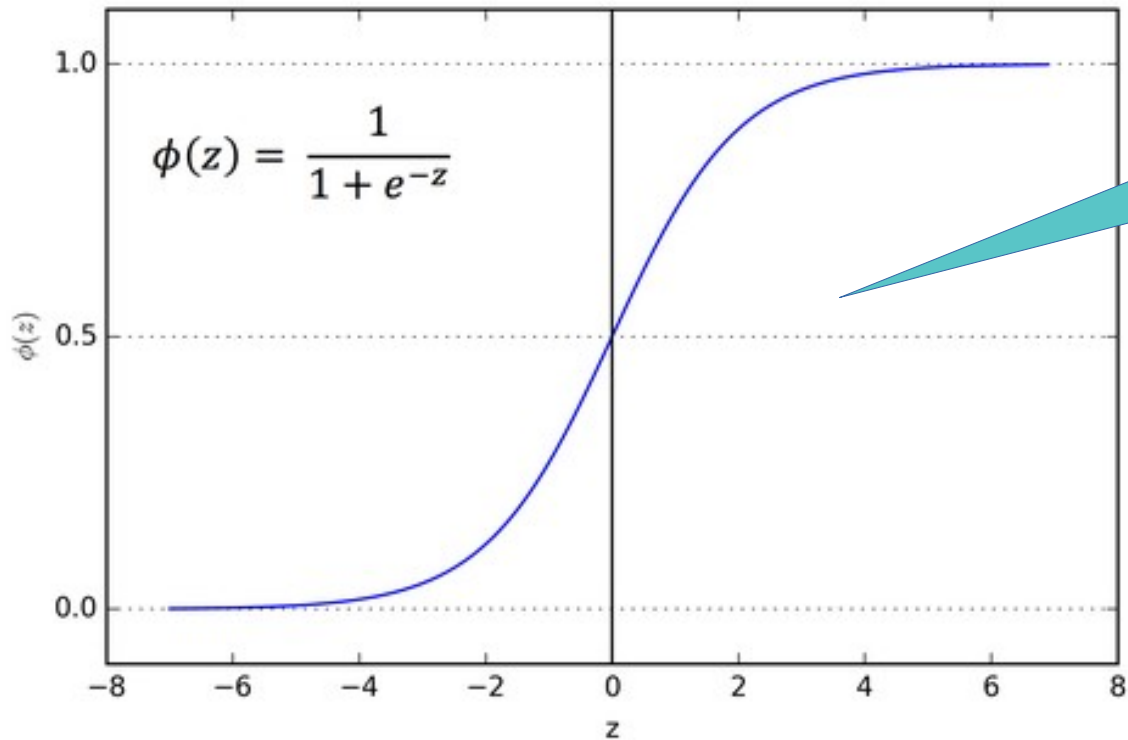
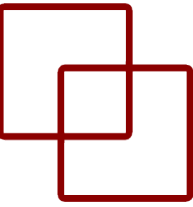
4. Función de activación

Sigmoide



4. Función de activación

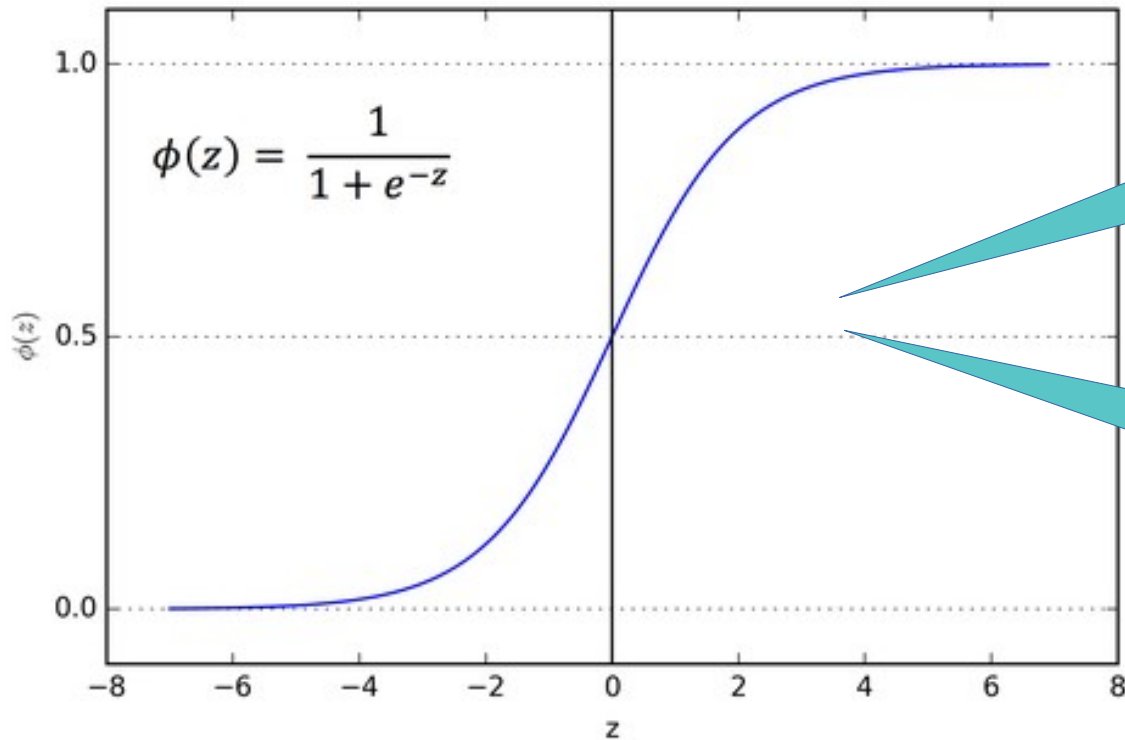
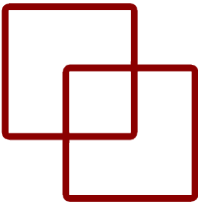
Sigmoide



Los valores muy **grandes** se saturan en 1
Los valores muy **pequeños** se saturan en 0

4. Función de activación

Sigmoide



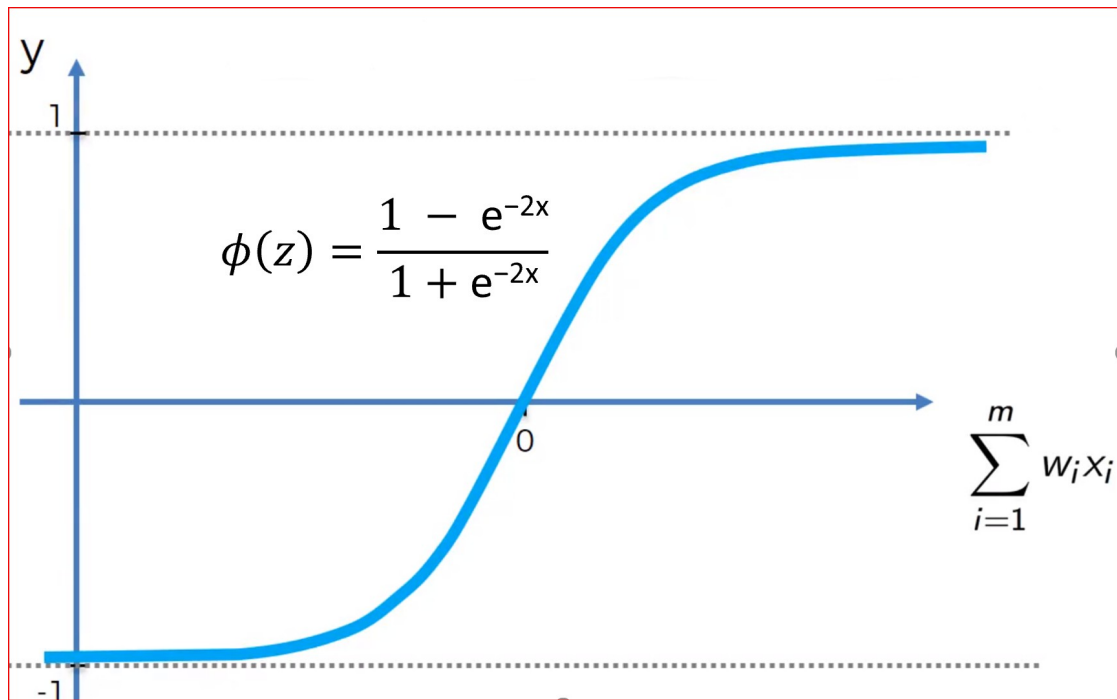
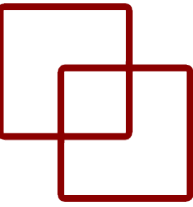
Los valores muy **grandes** se saturan en 1
Los valores muy **pequeños** se saturan en 0

Por tanto, nos permite:

- 1) **Distorsionar** la señal
- 2) Representar **probabilidades** entre 0 y 1

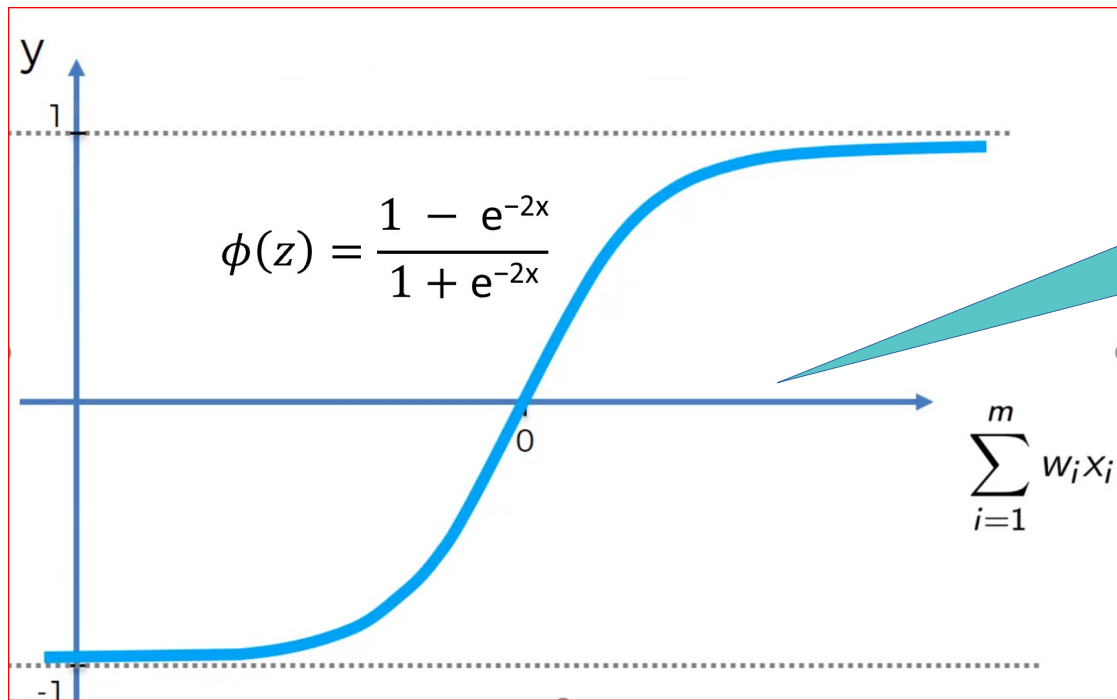
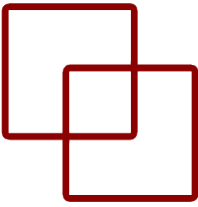
4. Función de activación

Tangente hiperbólica (TANH)



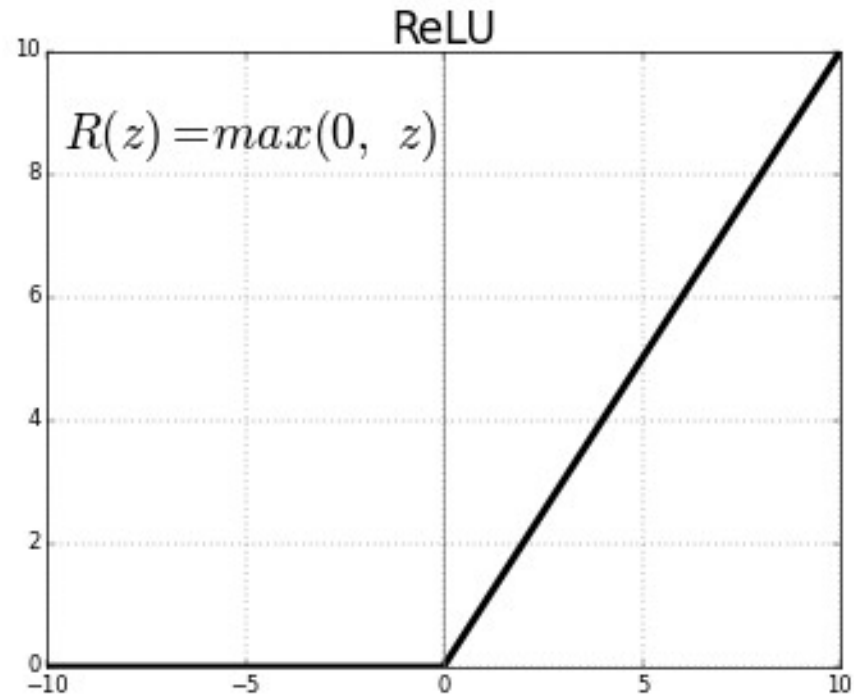
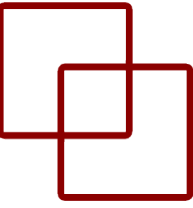
4. Función de activación

Tangente hiperbólica (TANH)



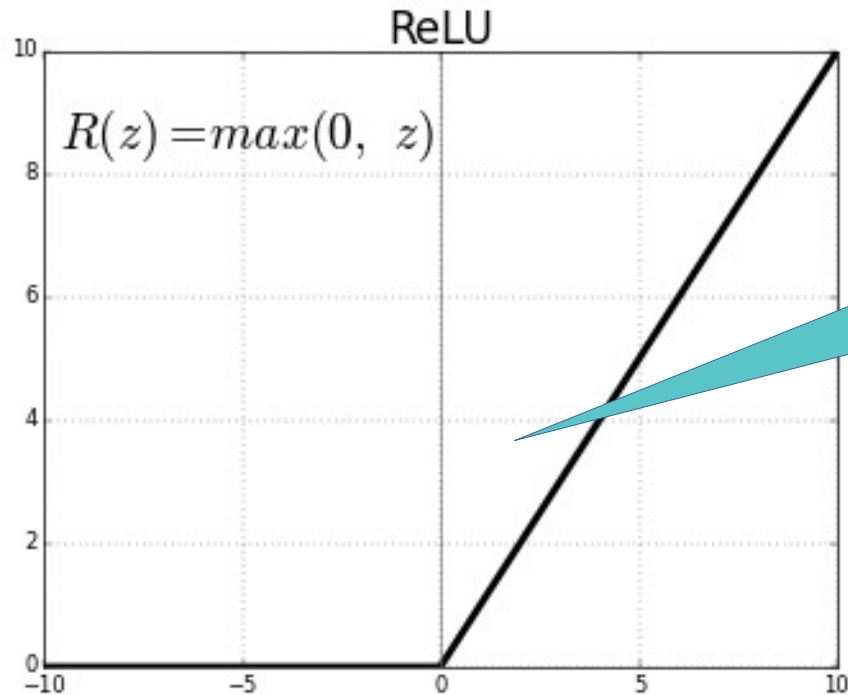
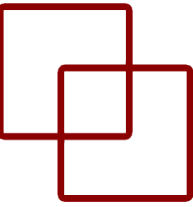
Parecida a la Sigmoide pero con un rango de -1 a 1

4. Función de activación Rectificada Lineal (ReLU)



4. Función de activación

Rectificada Lineal (ReLU)

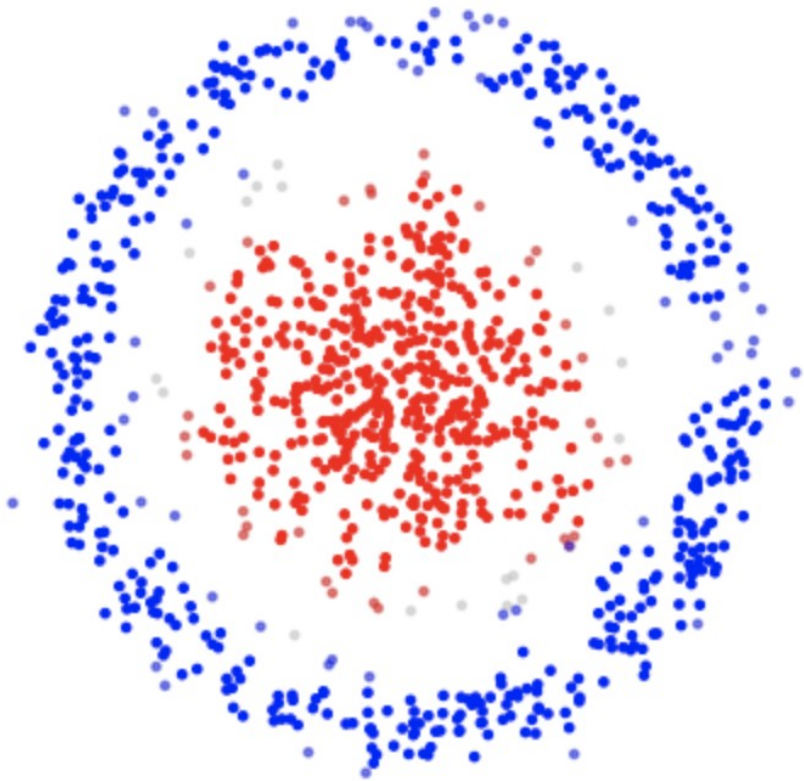
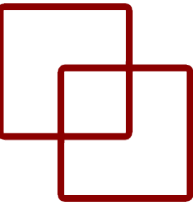


Función lineal cuando es positiva
Constante a 0 cuando el valor de entrada es negativo

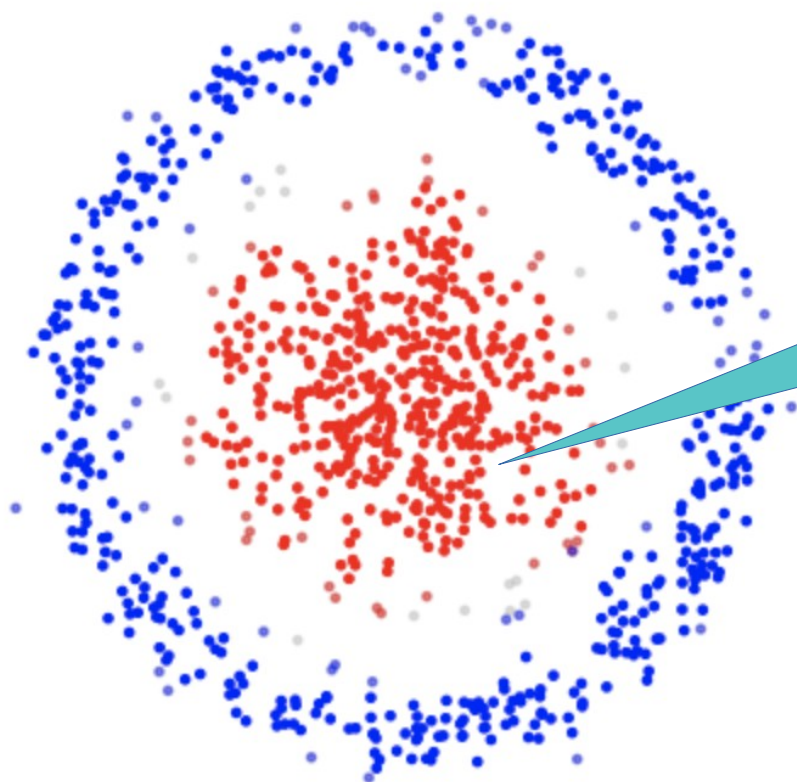
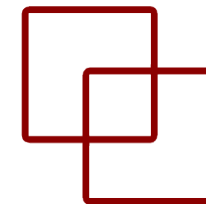


Cómo opera el MLP

1. Problema

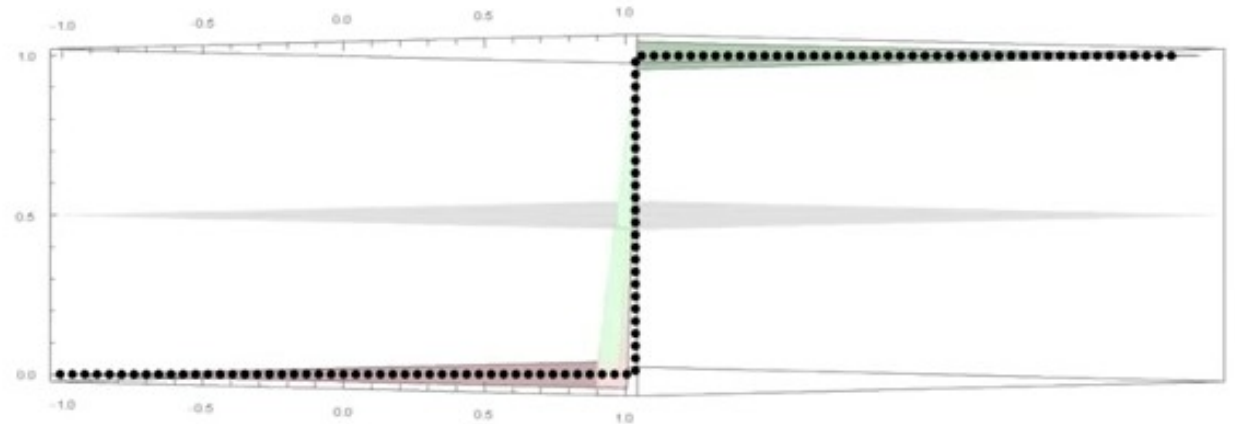
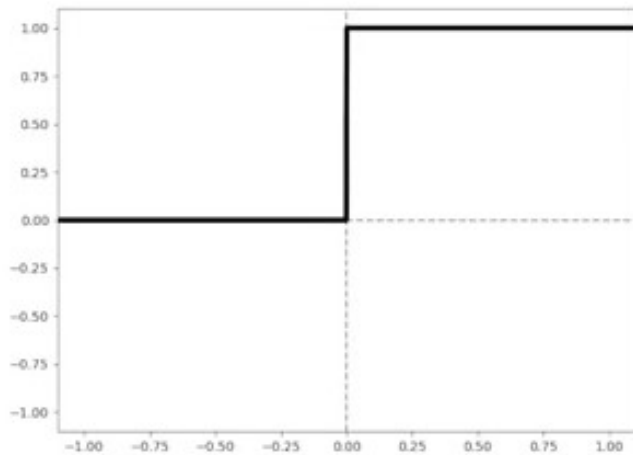
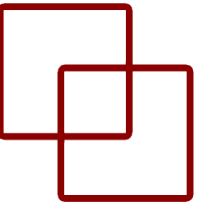


1. Problema

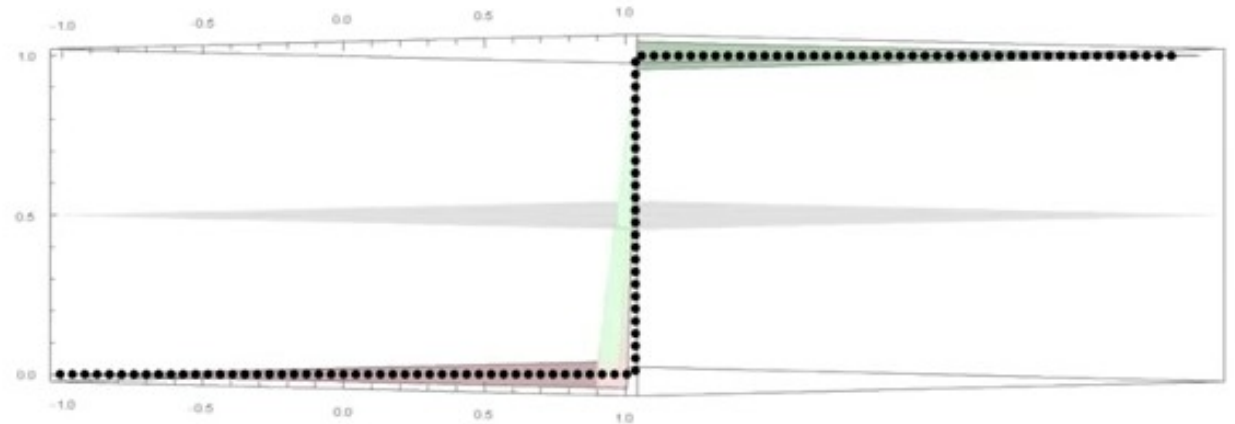
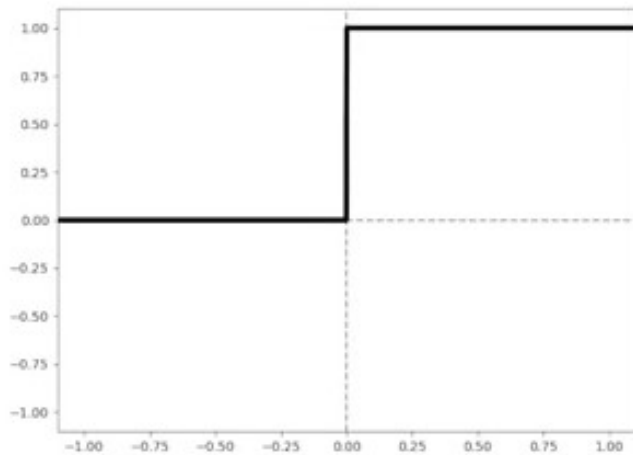
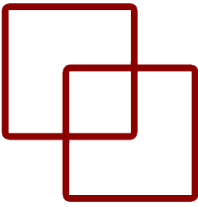


¿Cómo ver geoméricamente el efecto de una función de activación?

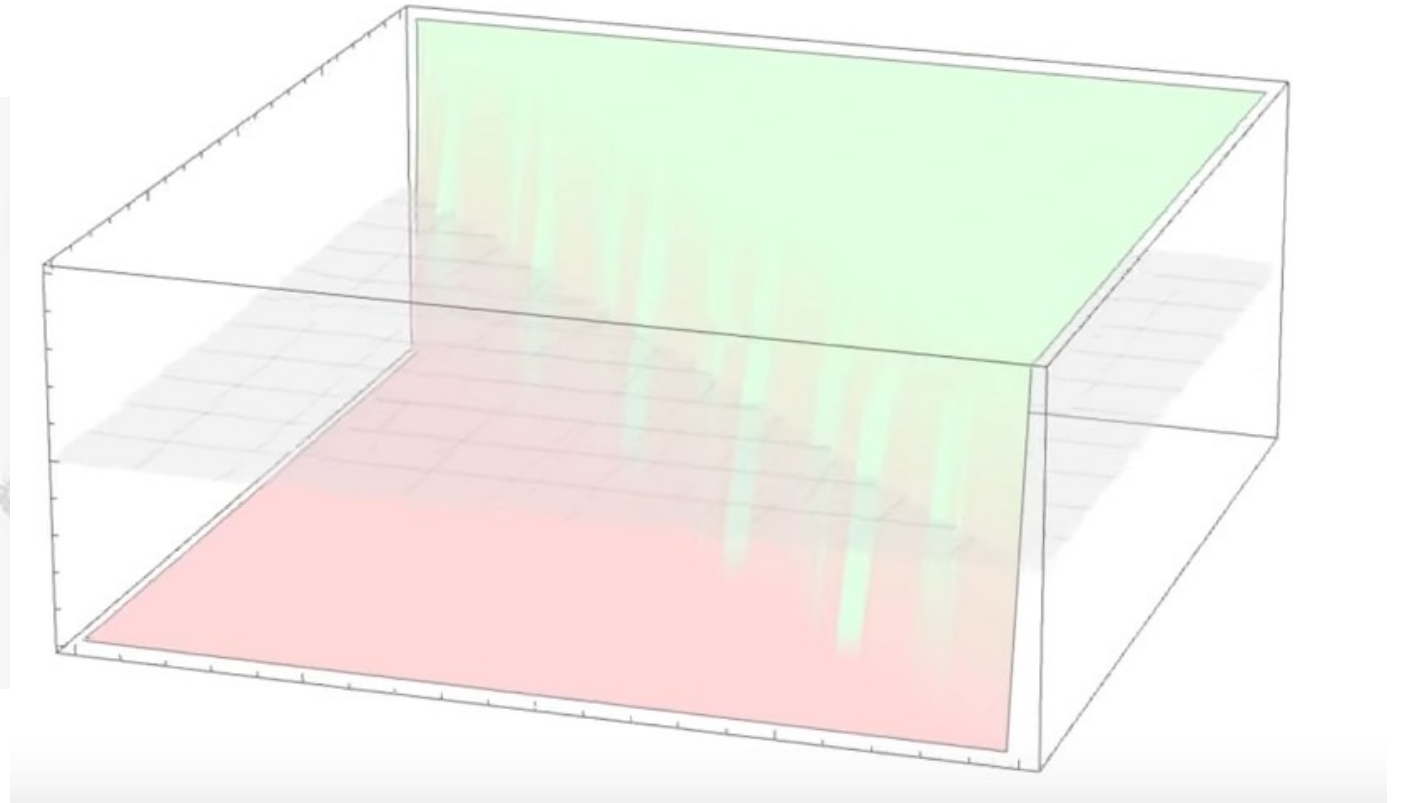
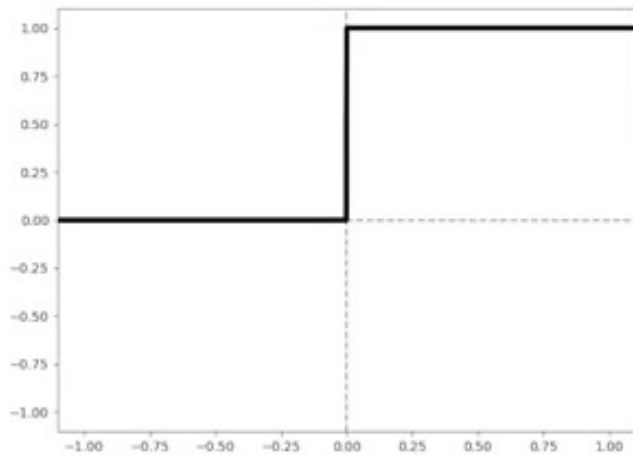
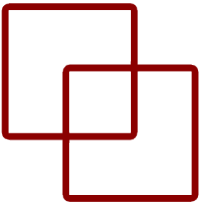
2. Función de activación binaria



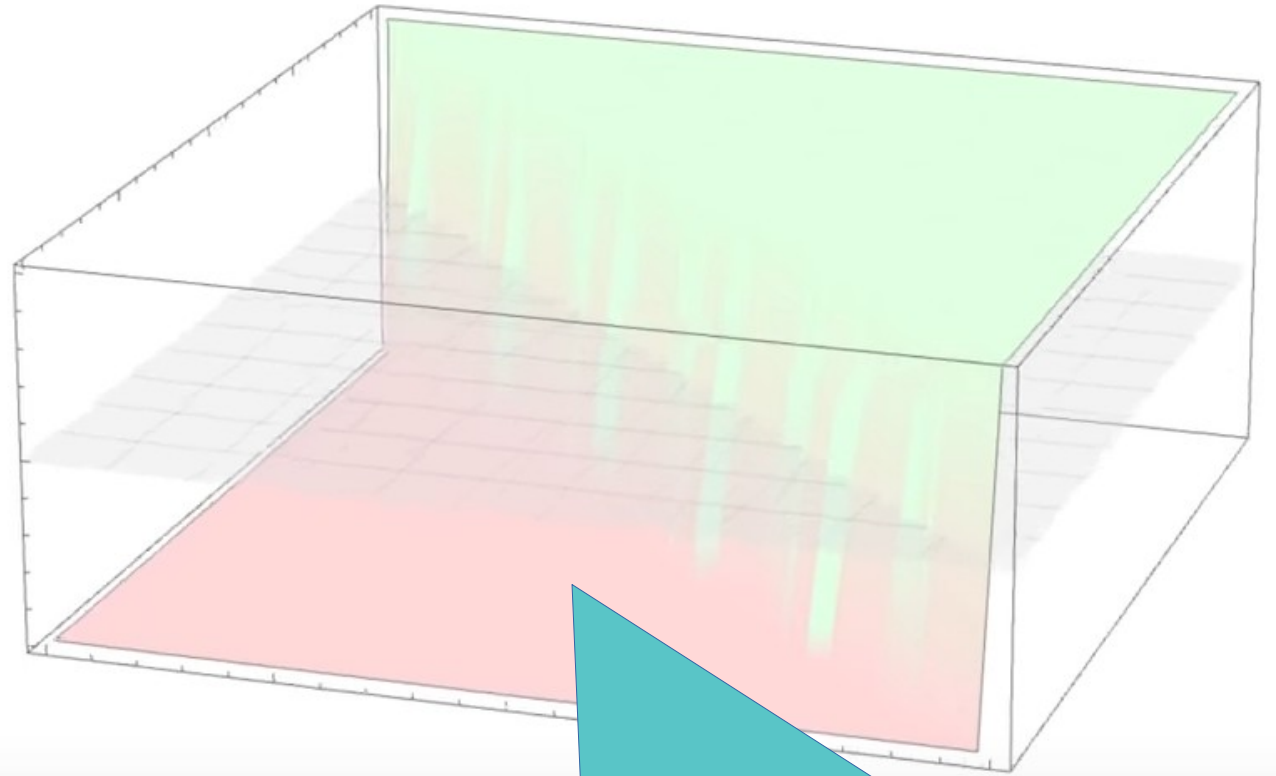
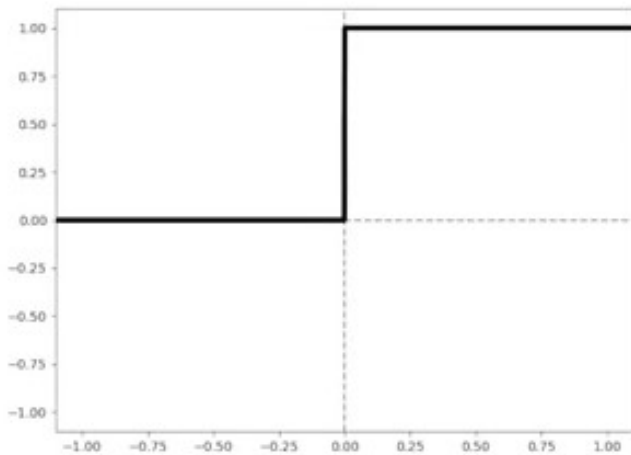
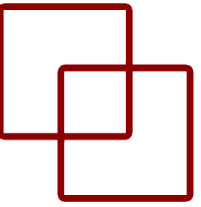
2. Función de activación binaria



2. Función de activación binaria

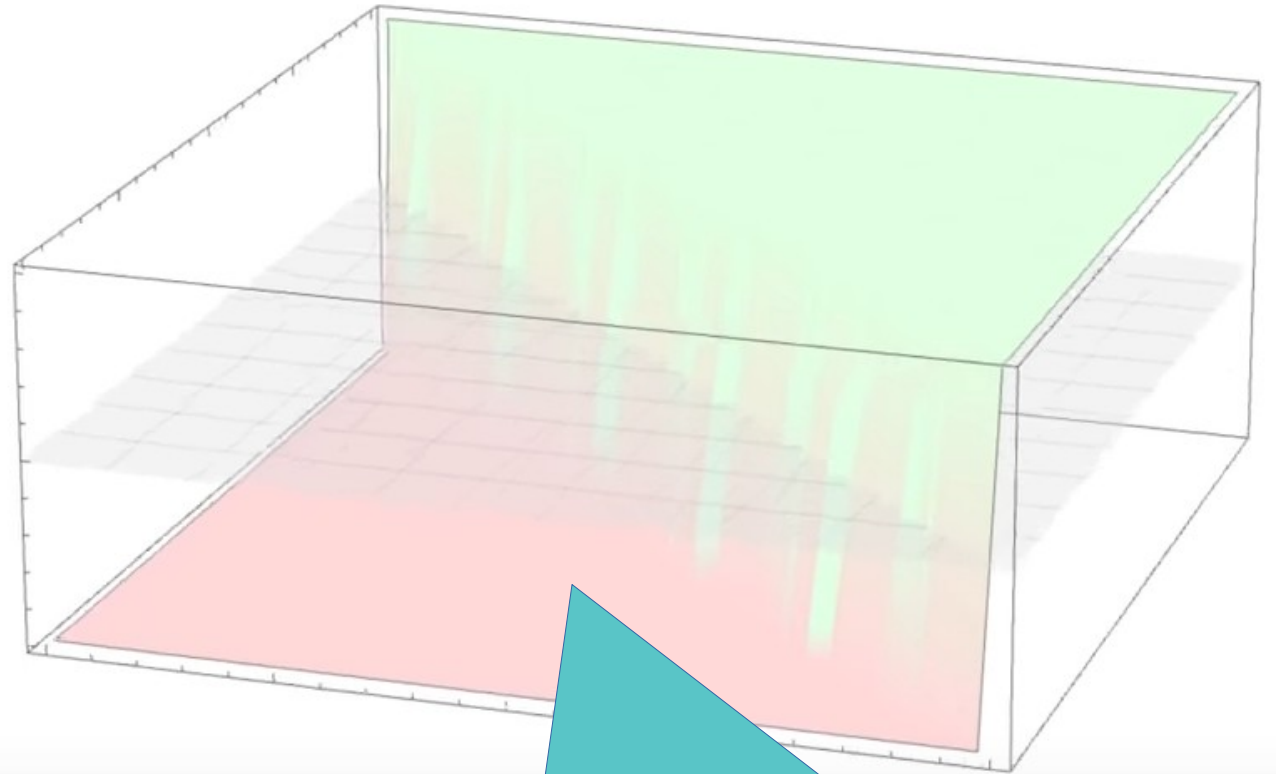
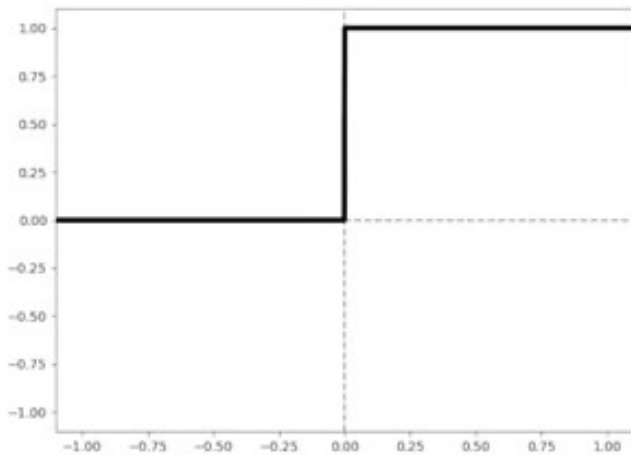
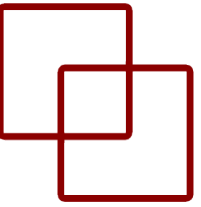


2. Función de activación binaria



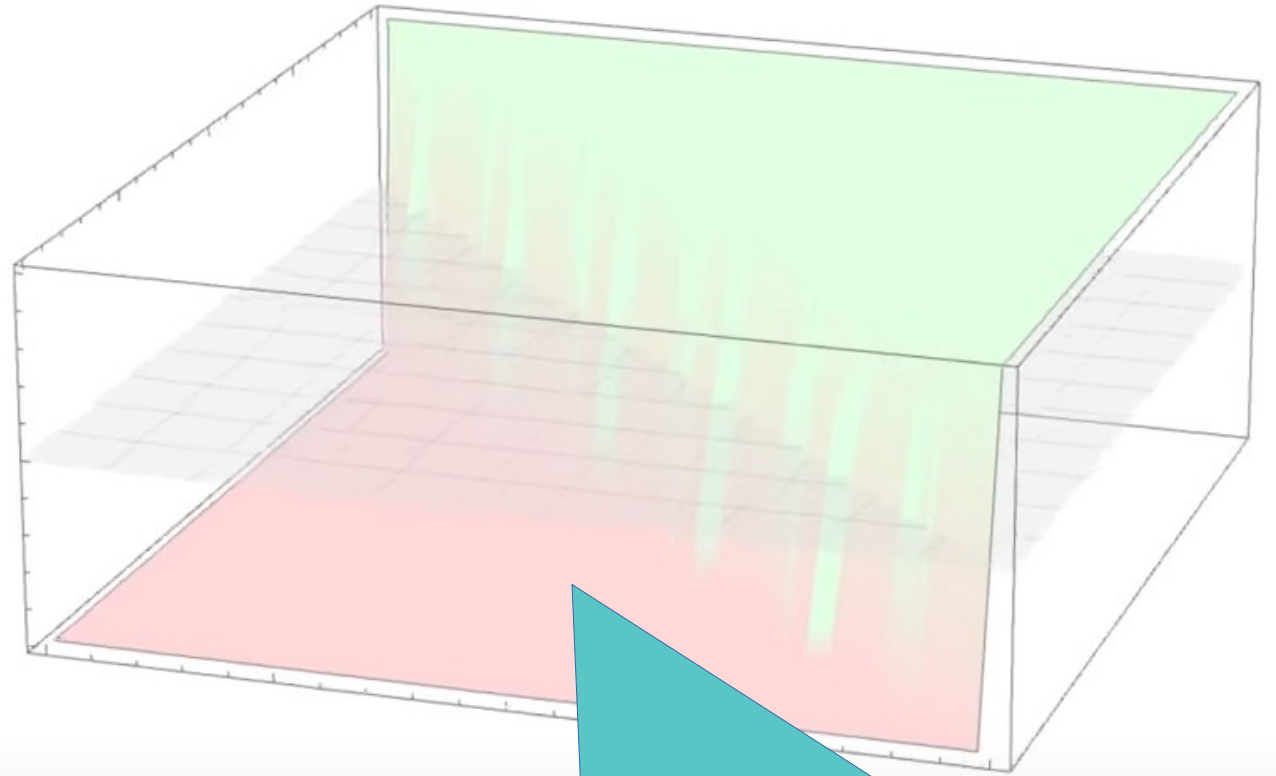
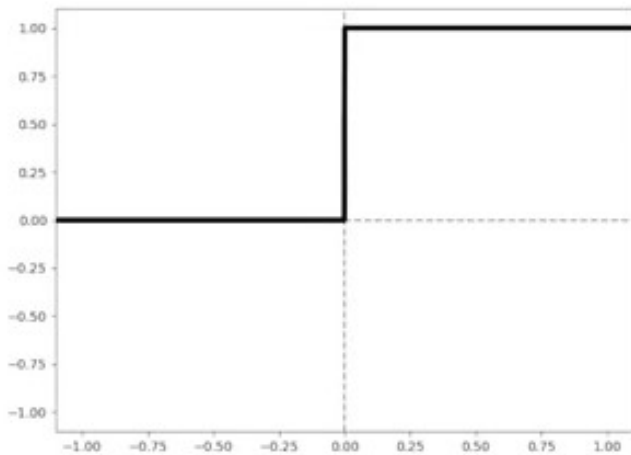
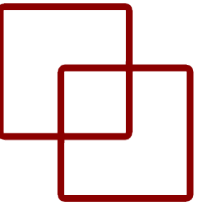
Todo lo que está encima del hiperplano será una clase y lo que está debajo otra

2. Función de activación binaria



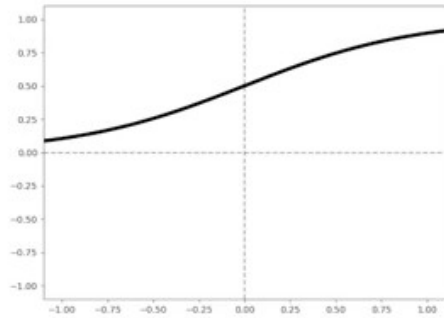
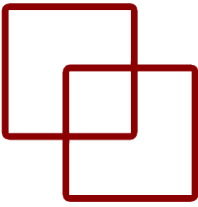
Todo lo que está encima del hiperplano será una clase y lo que está debajo otra

2. Función de activación binaria

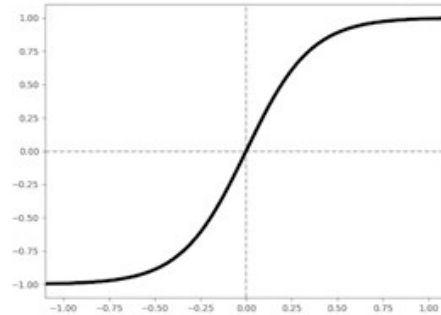


Neceistamos distorsionar la señal
para acomodar el problema inicial

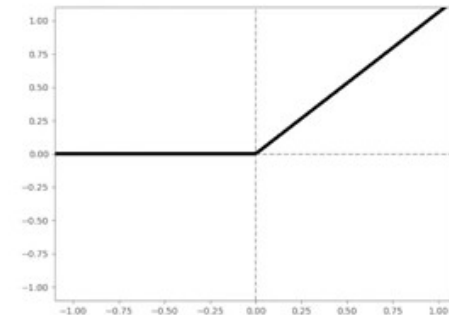
3. Otras Funciones de activación



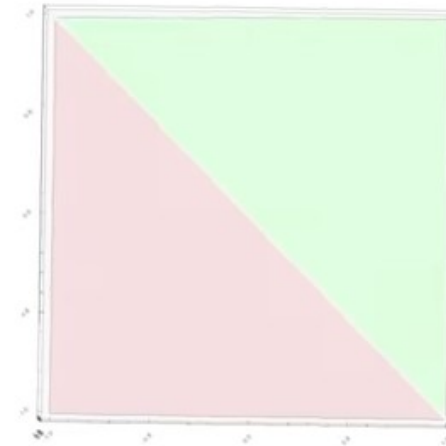
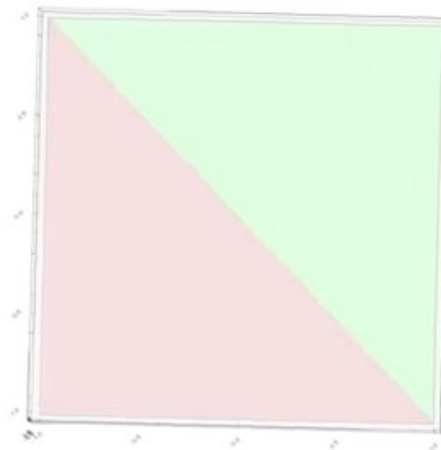
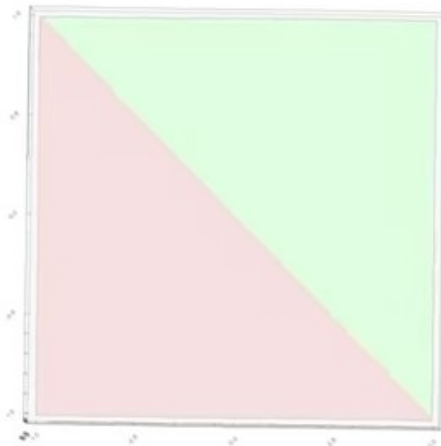
SIGMOIDE



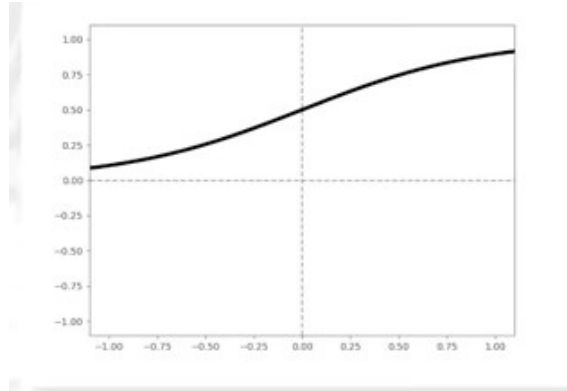
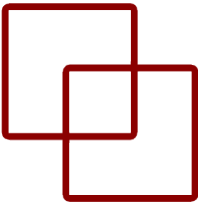
TANH



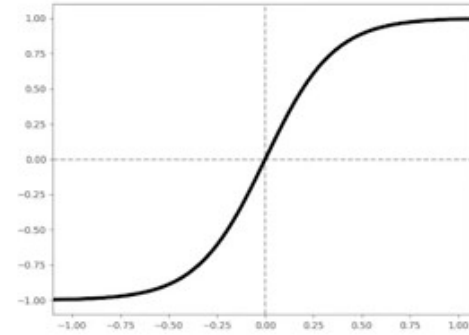
RELU



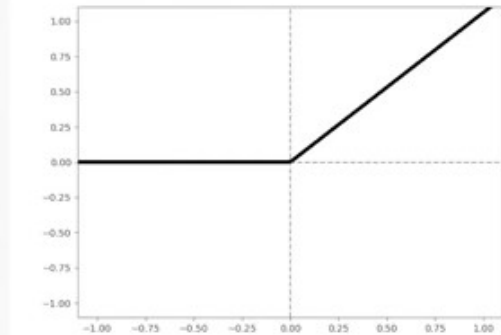
3. Otras Funciones de activación



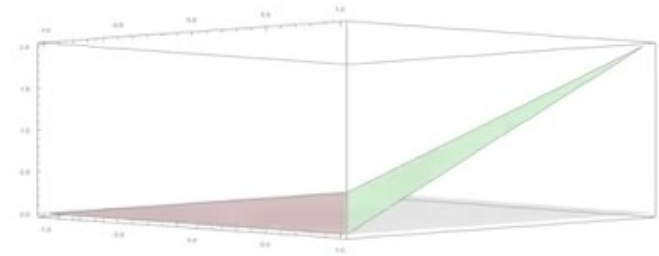
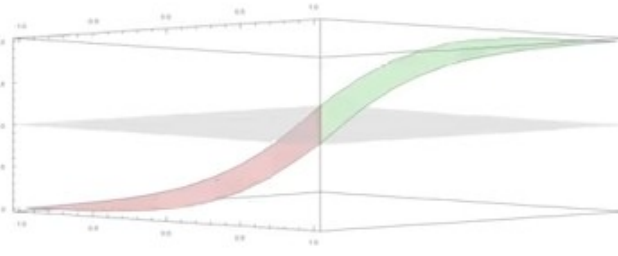
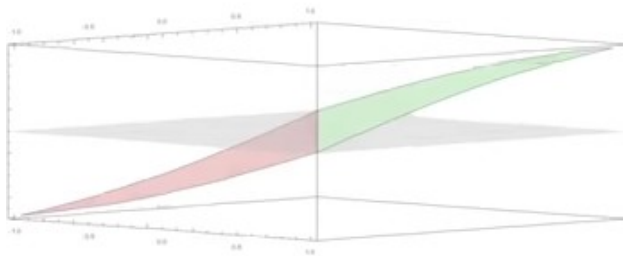
SIGMOIDE



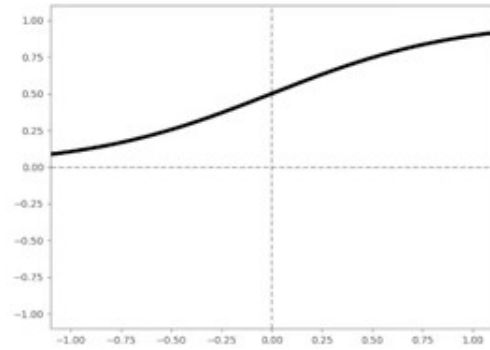
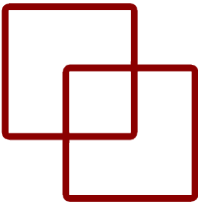
TANH



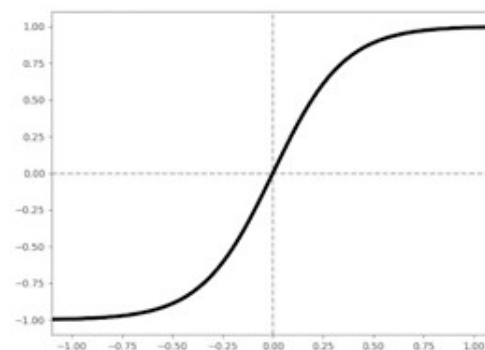
RELU



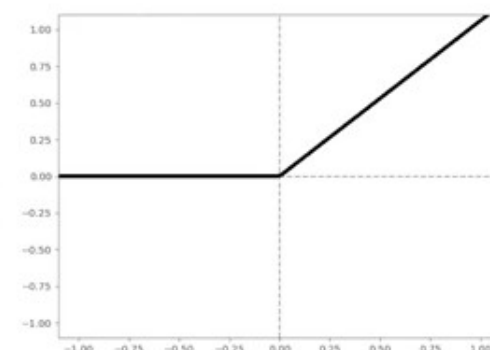
3. Otras Funciones de activación



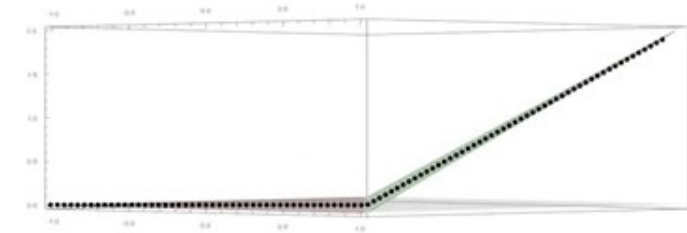
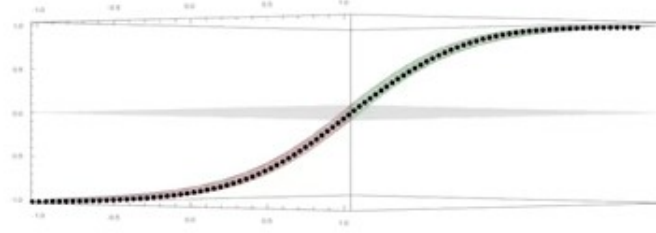
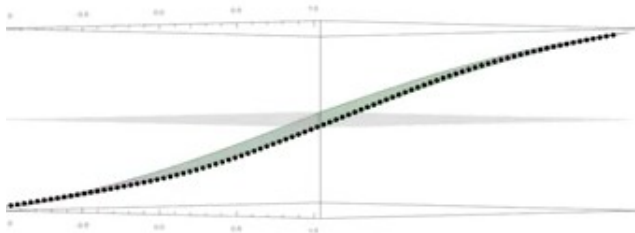
SIGMOIDE



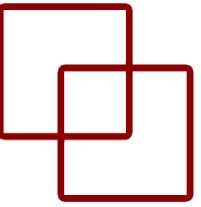
TANH



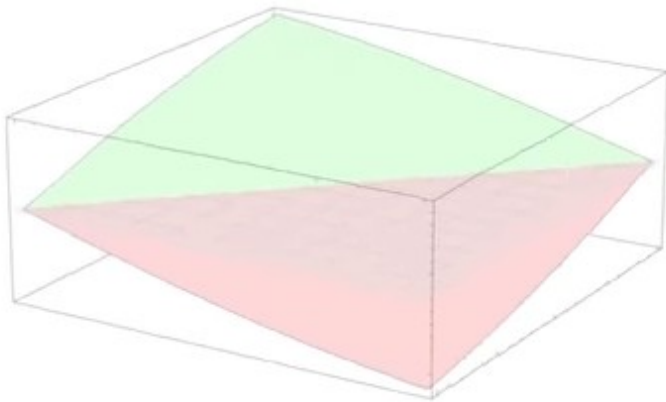
RELU



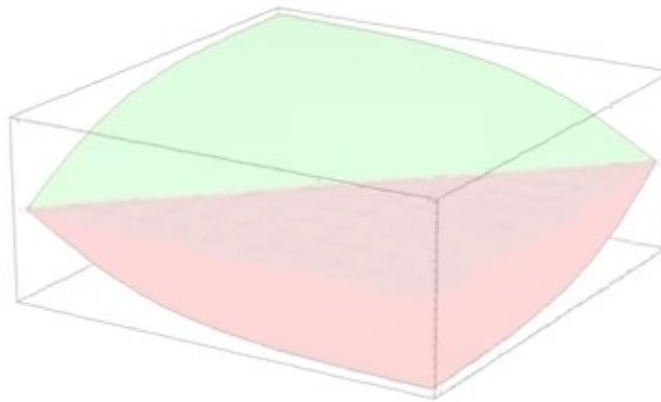
3. Otras Funciones de activación



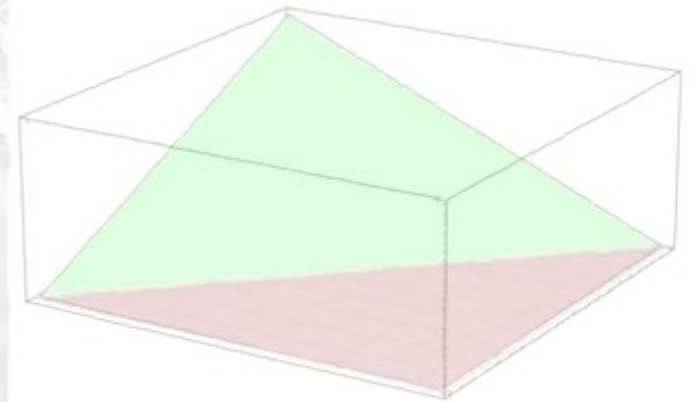
SIGMOIDE



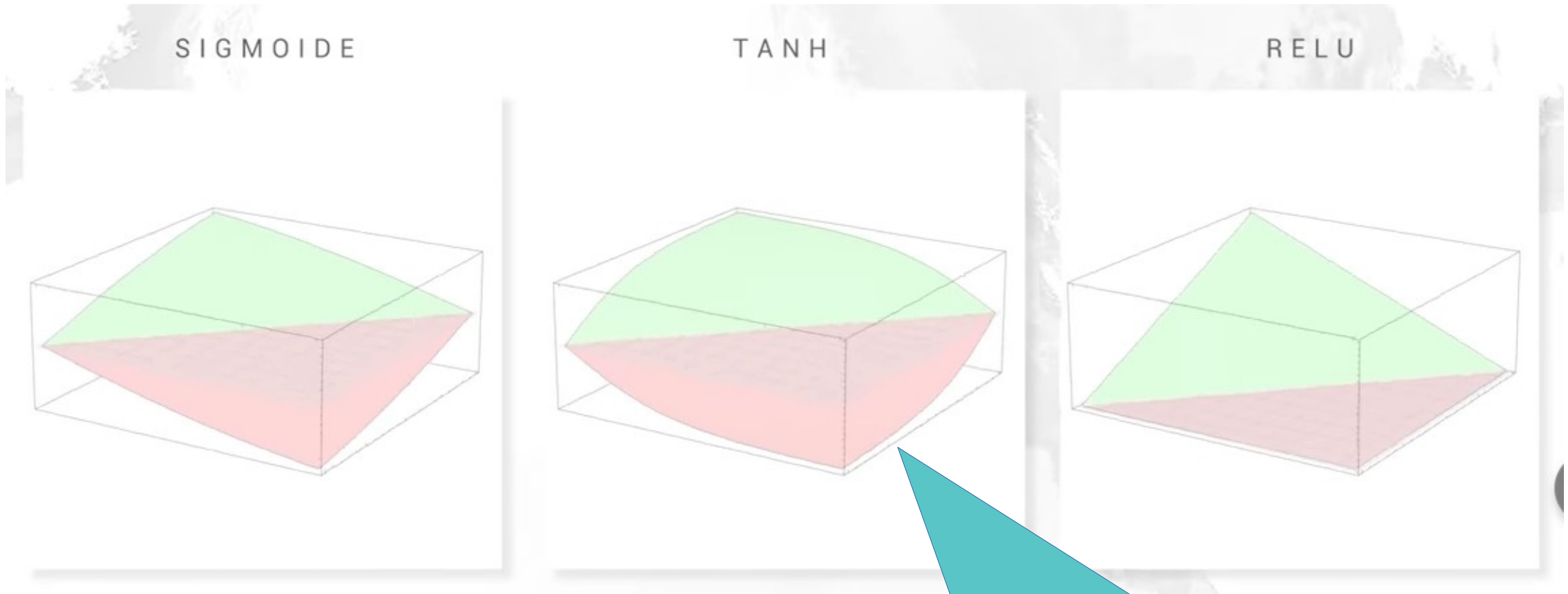
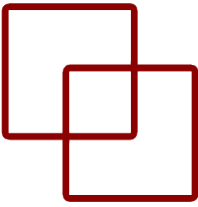
TANH



RELU

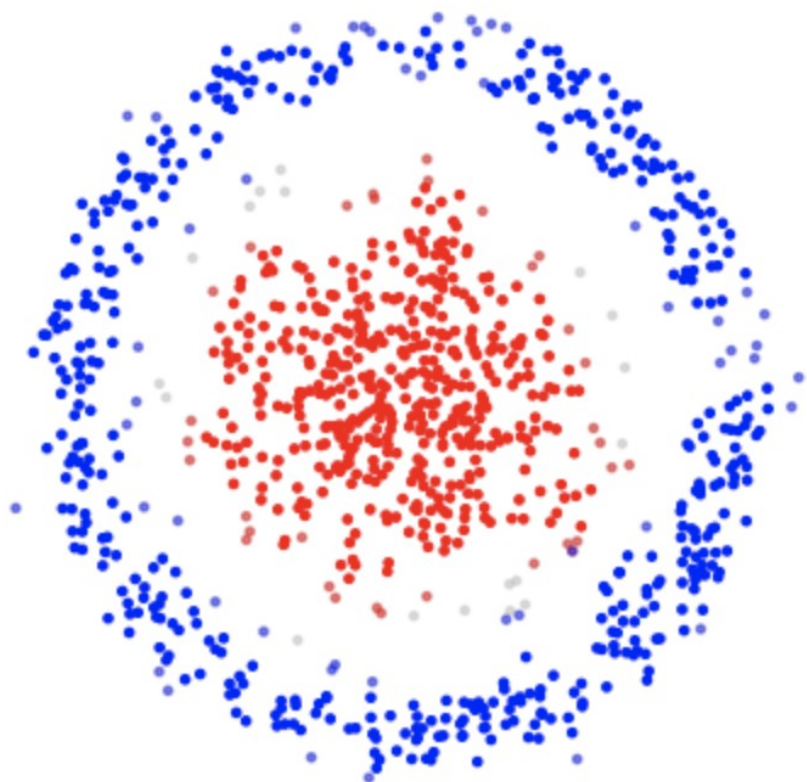
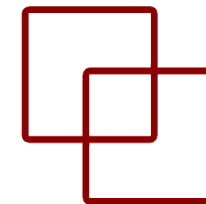


3. Otras Funciones de activación

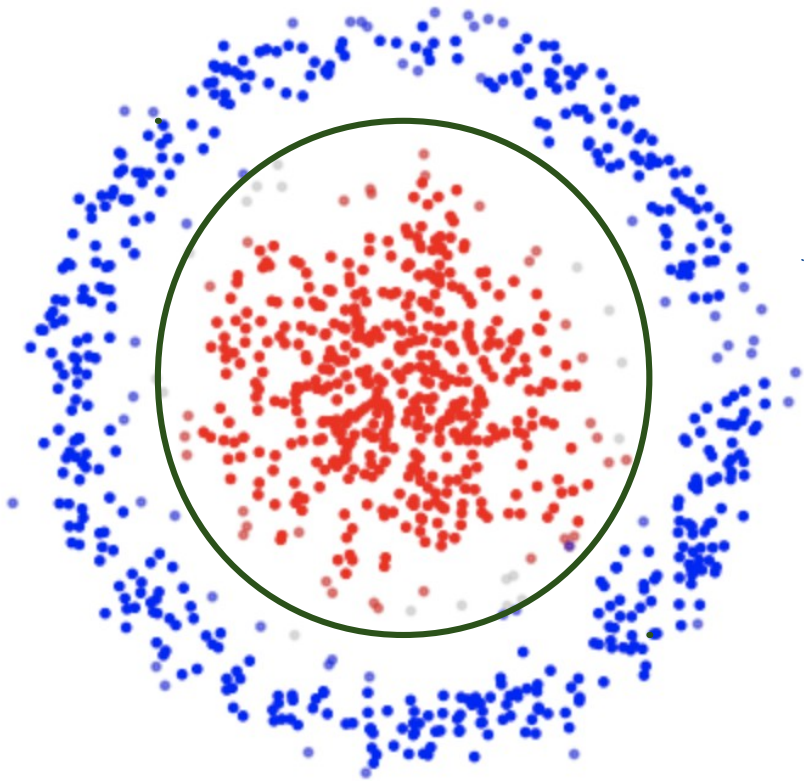
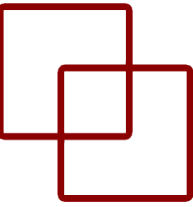


Pero la frontera no deja de ser una
Línea recta, ¿Cómo resolvemos el problema?

4. Encontrar el hiperplano

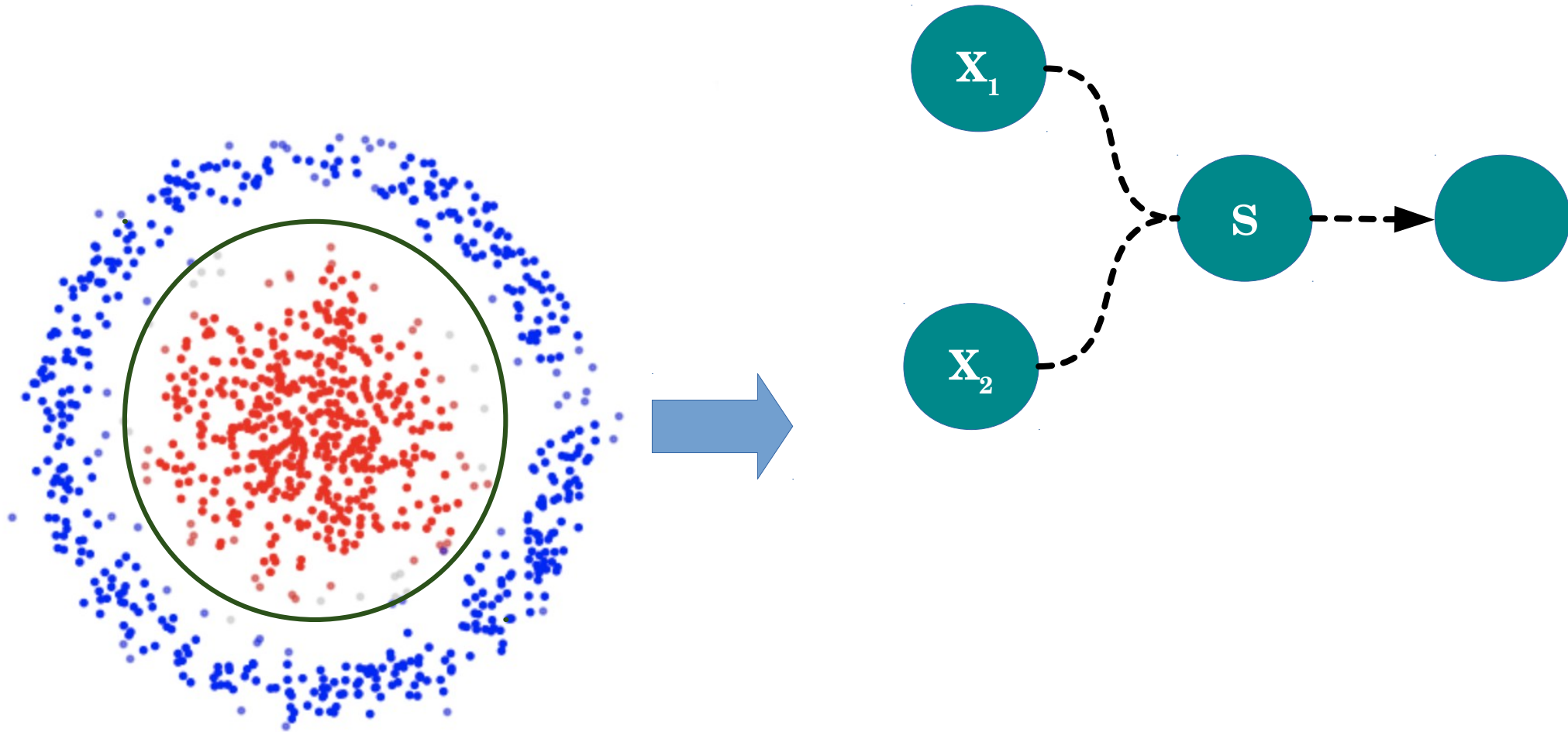
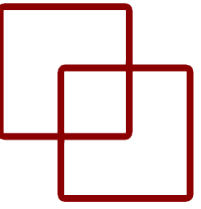


4. Encontrar el hiperplano

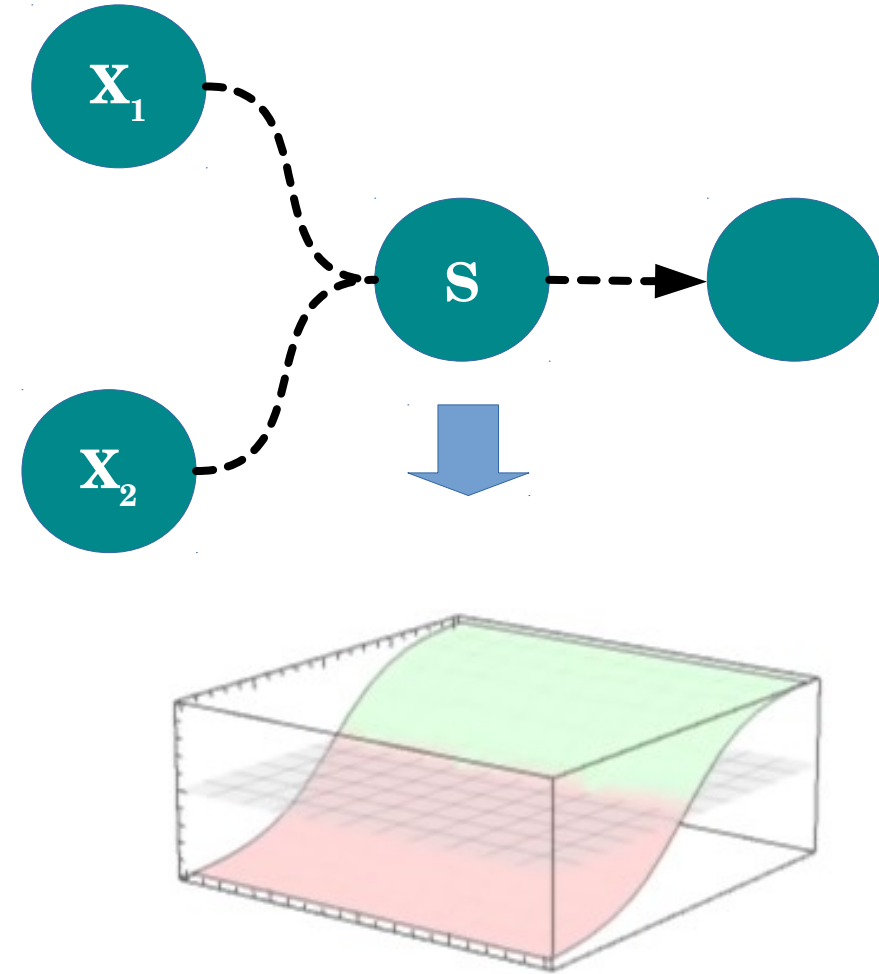
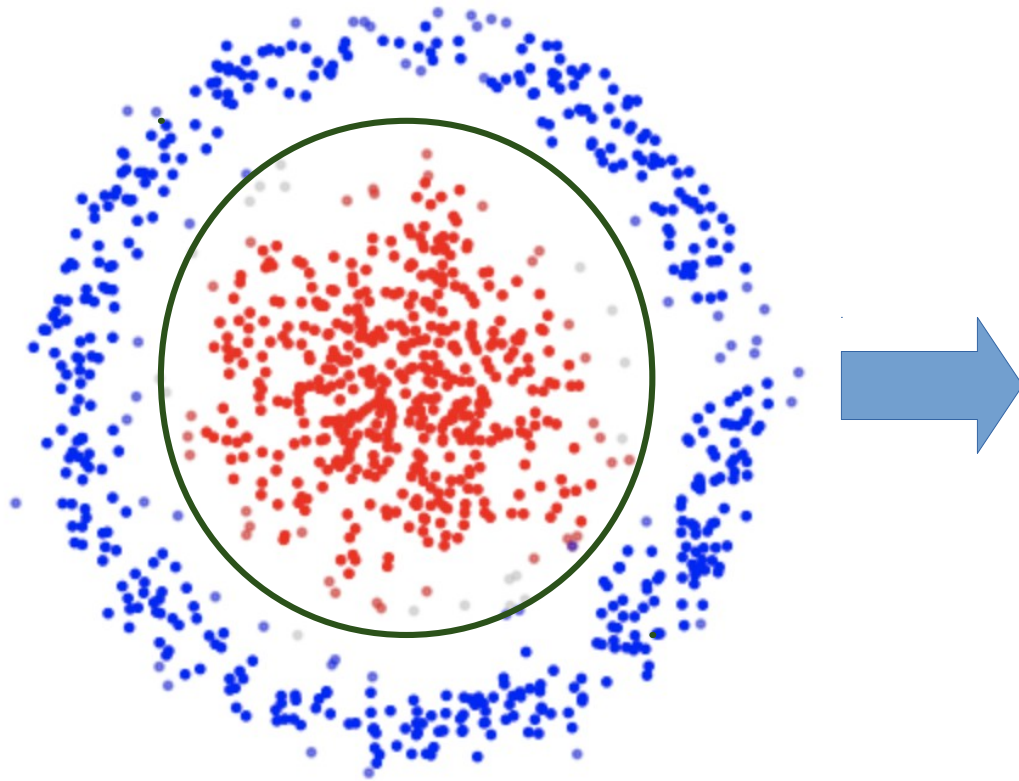
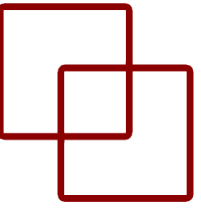


¿Cómo trazar esa frontera?

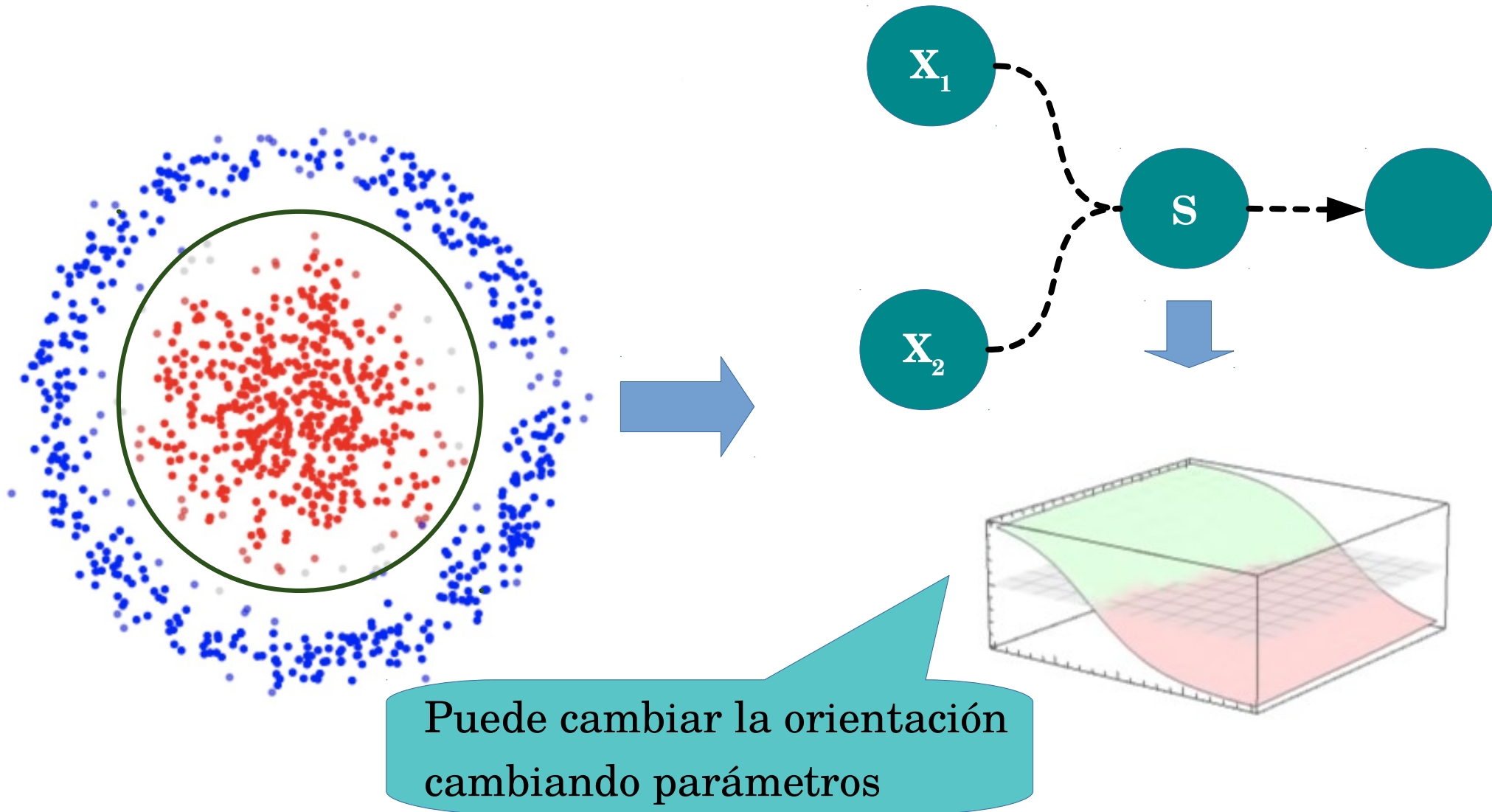
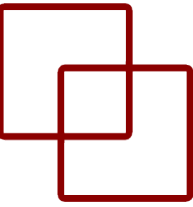
4. Encontrar el hiperplano



4. Encontrar el hiperplano

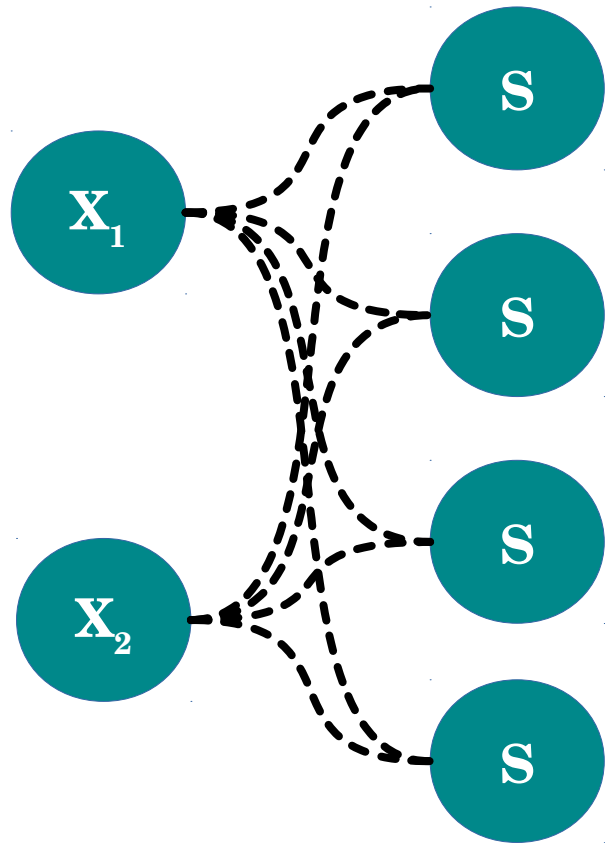
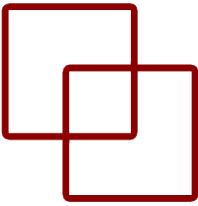


4. Encontrar el hiperplano



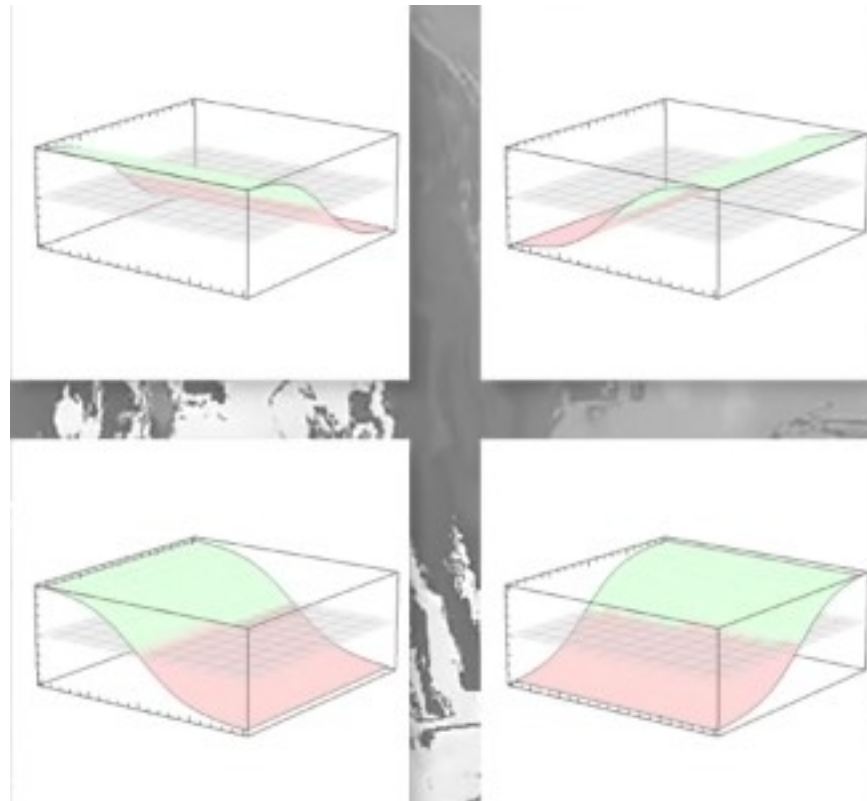
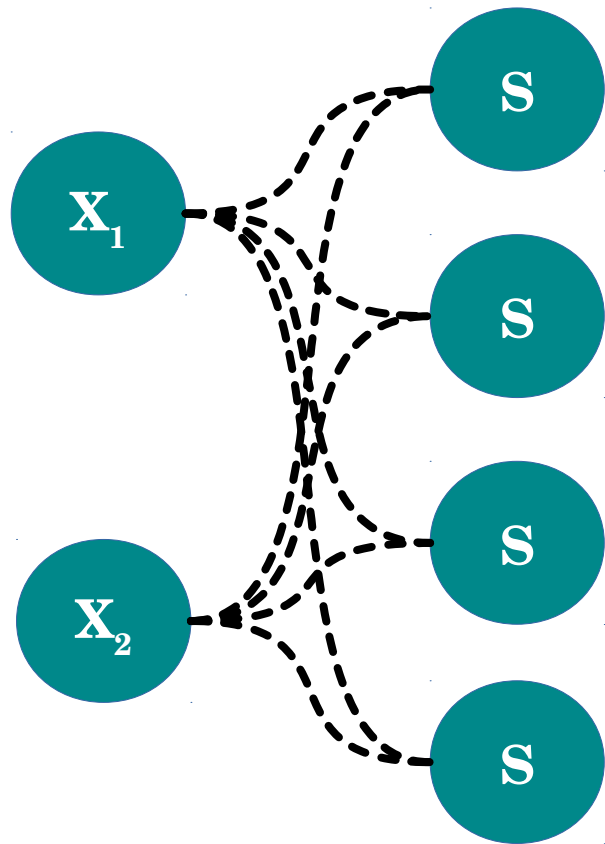
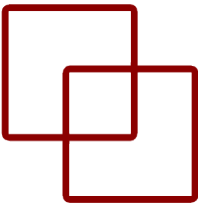
4. Encontrar el hiperplano

Aumentamos las neuronas



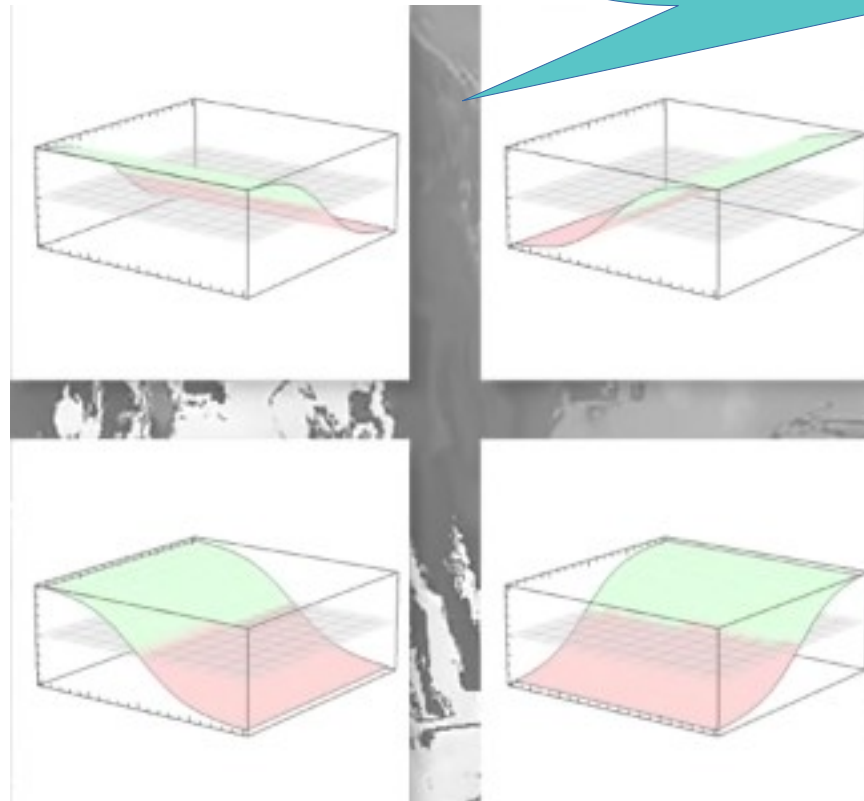
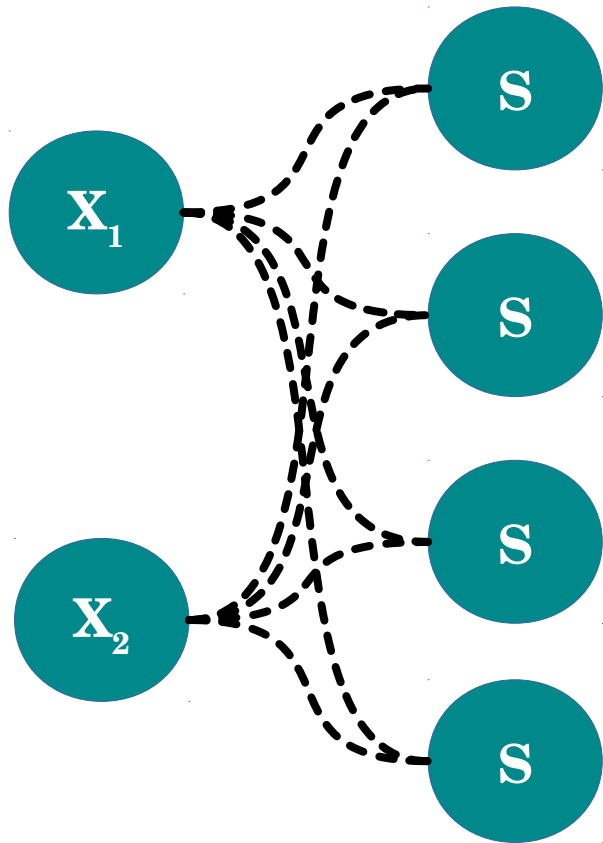
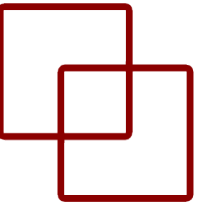
4. Encontrar el hiperplano

Aumentamos las neuronas



4. Encontrar el hiperplano

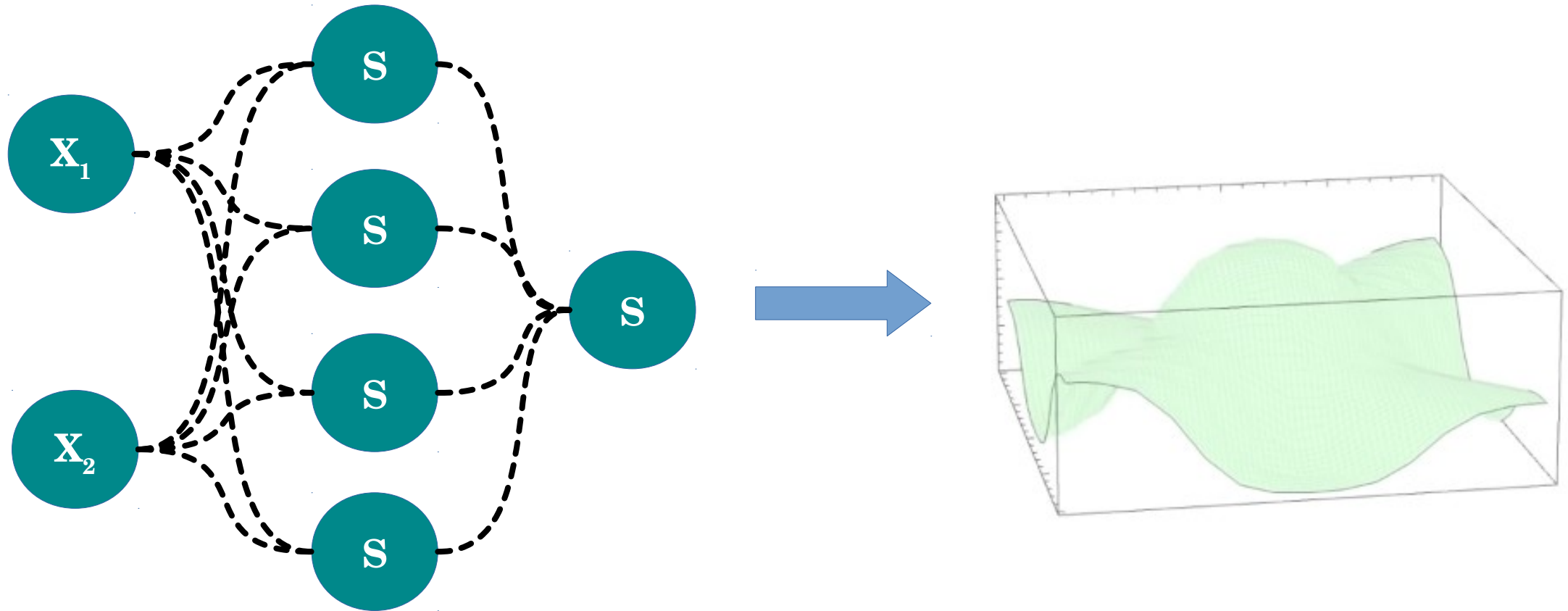
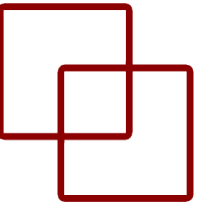
Aumentamos las neuronas



Pero cada una tiene una
orientación diferente
Las juntamos...

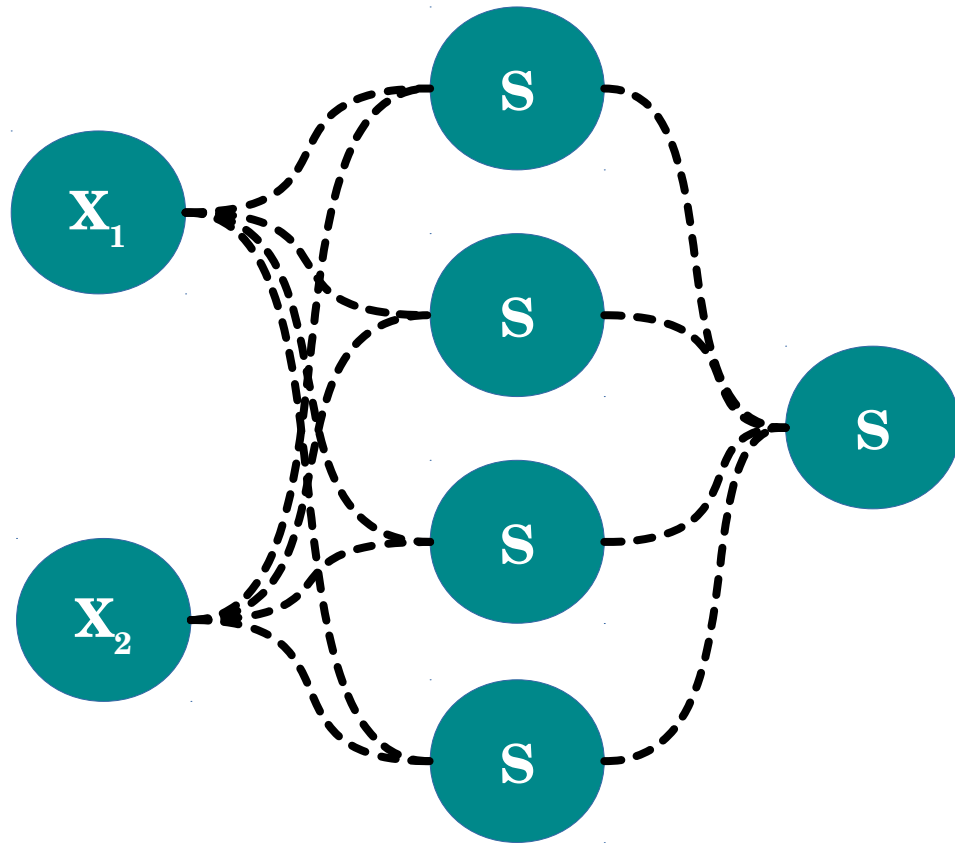
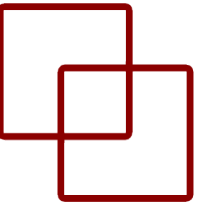
4. Encontrar el hiperplano

Segunda capa oculta

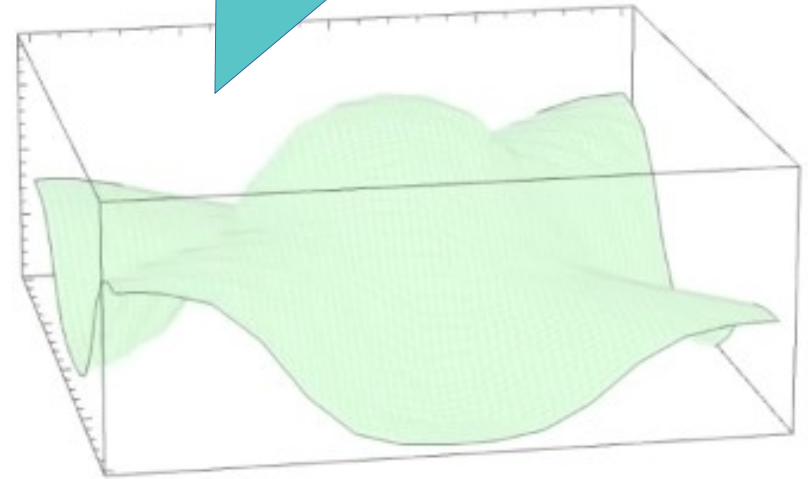


4. Encontrar el hiperplano

Segunda capa oculta

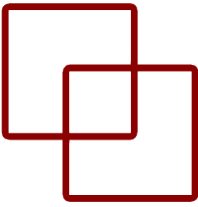


Obtenemos la superposición geométrica con un “bulto” al medio

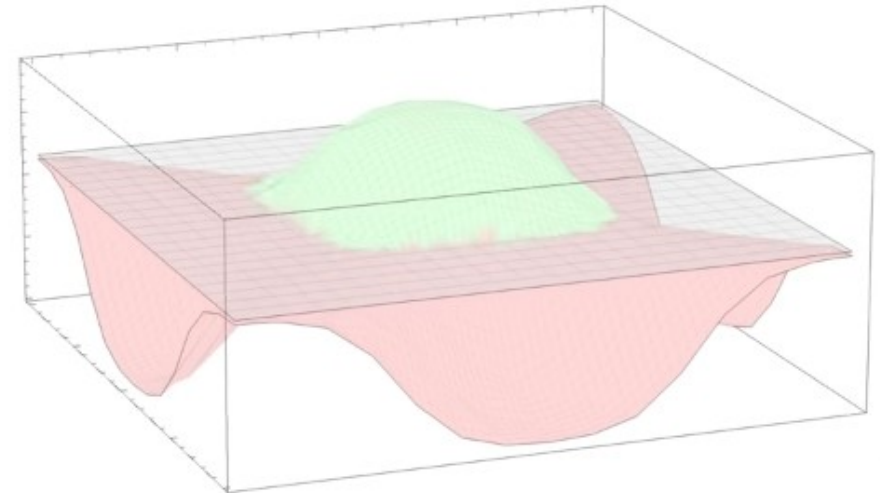
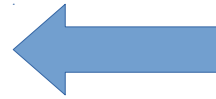
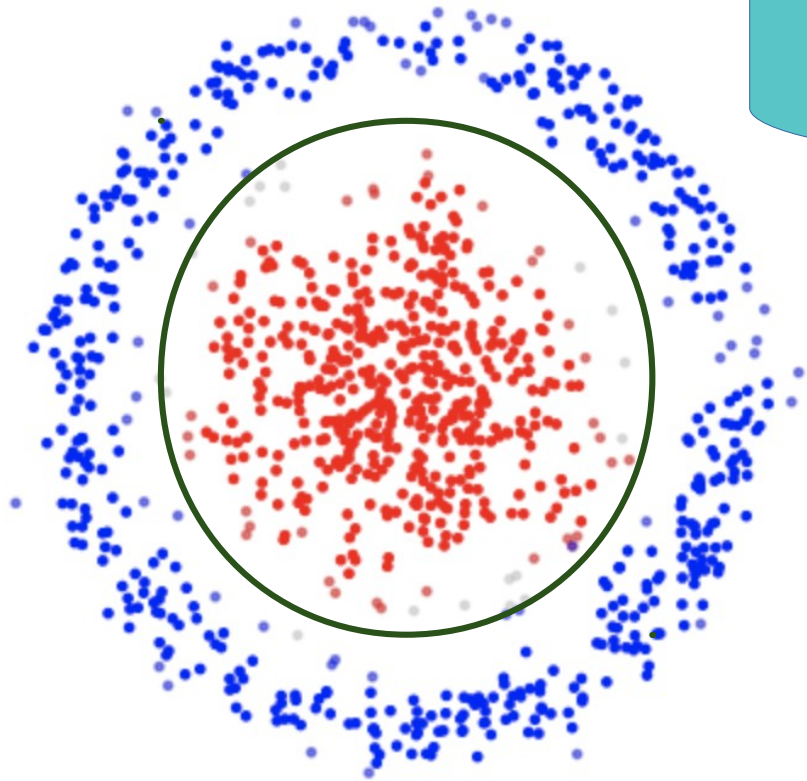


4. Encontrar el hiperplano

Segunda capa oculta

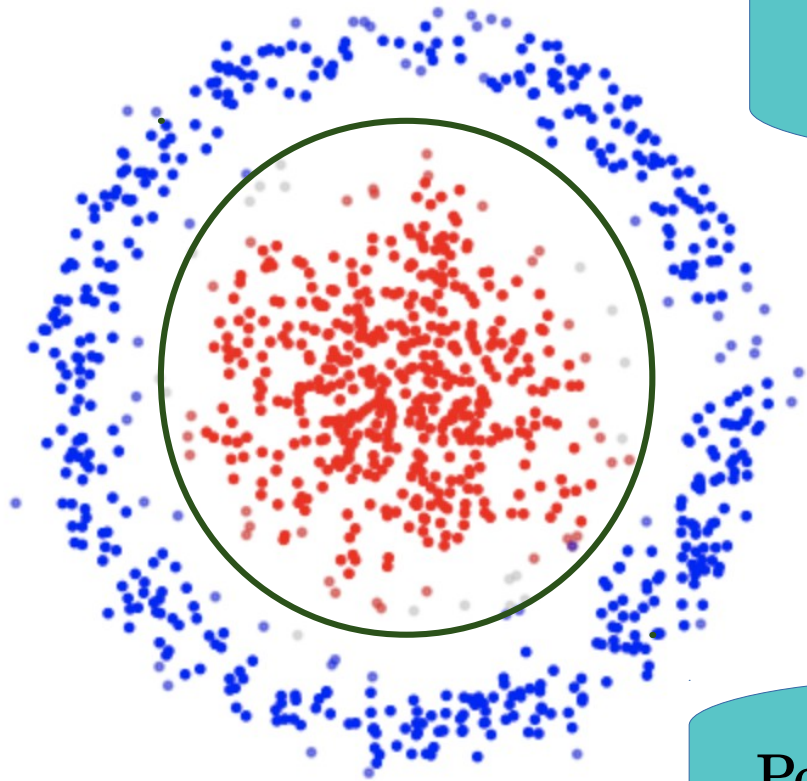
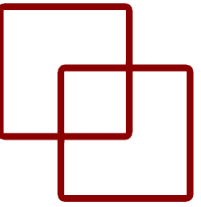


Qué podemos “cortar” con la intersección del hiperplano con la “montaña”

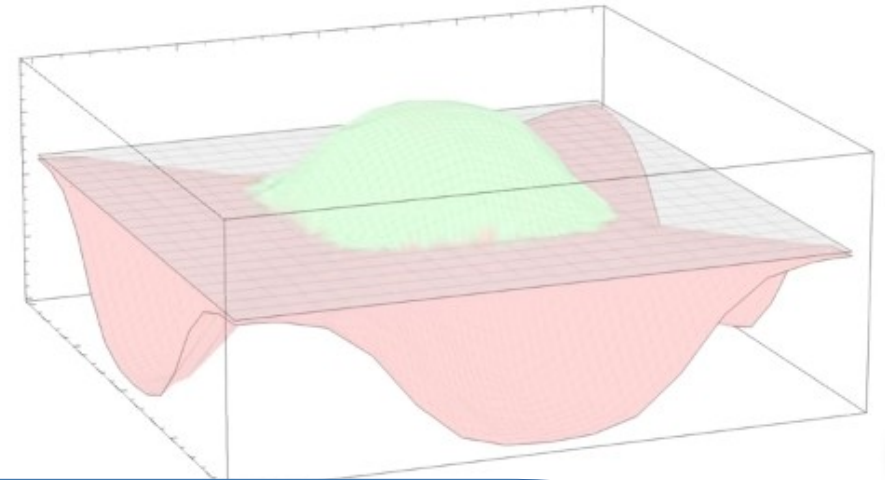


4. Encontrar el hiperplano

Segunda capa oculta

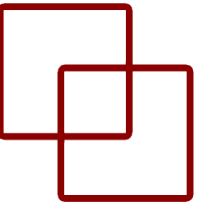


Qué podemos “cortar” con la intersección del hiperplano con la “montaña”

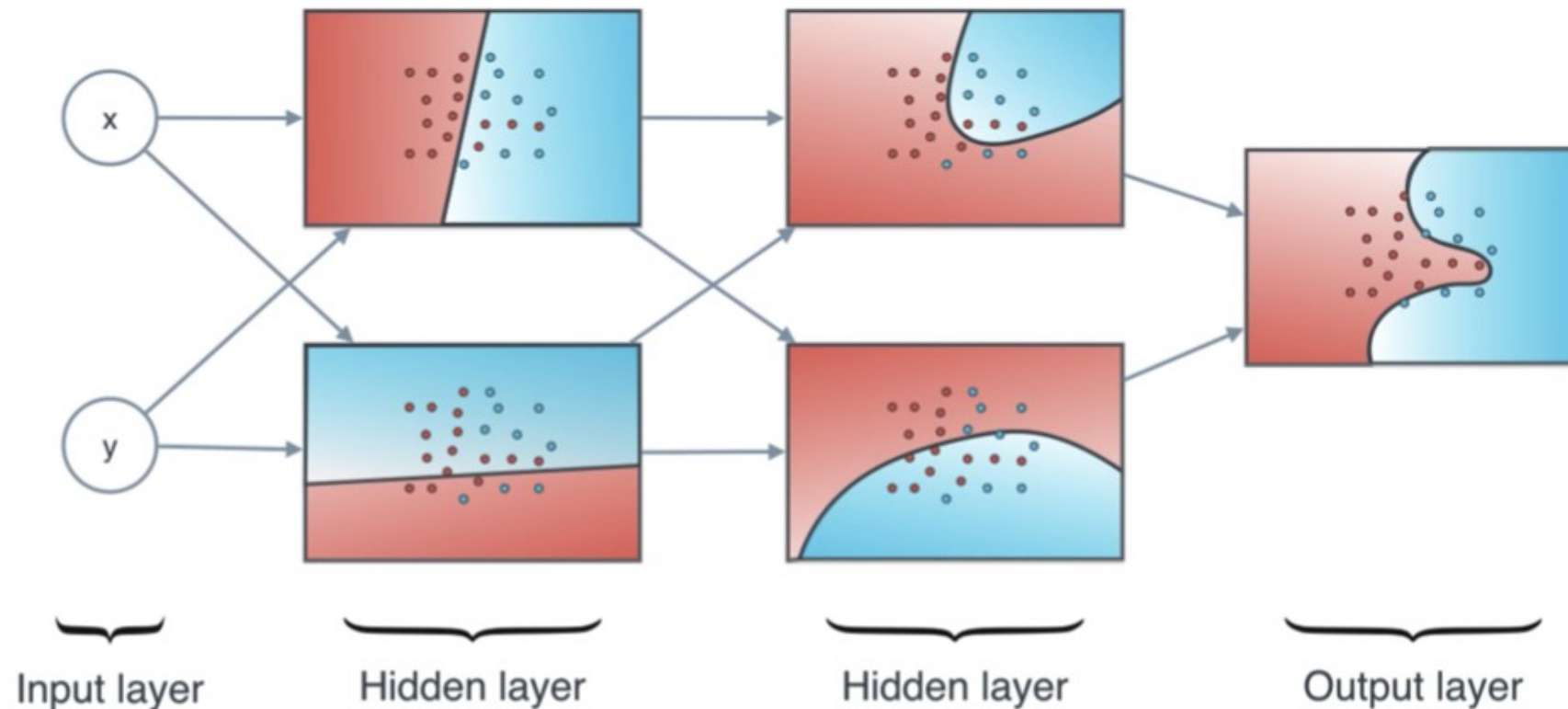


Por lo que se pueden resolver problemas complejos

5. Arquitectura de redes neuronales

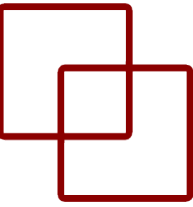


- Son la base de las **redes profundas**, actualmente muy utilizadas con mucho éxito.
- Cada **capa extrae características** cada vez más complejas
 - Necesitamos un conjunto de neuronas para resolver problemas no lineales.





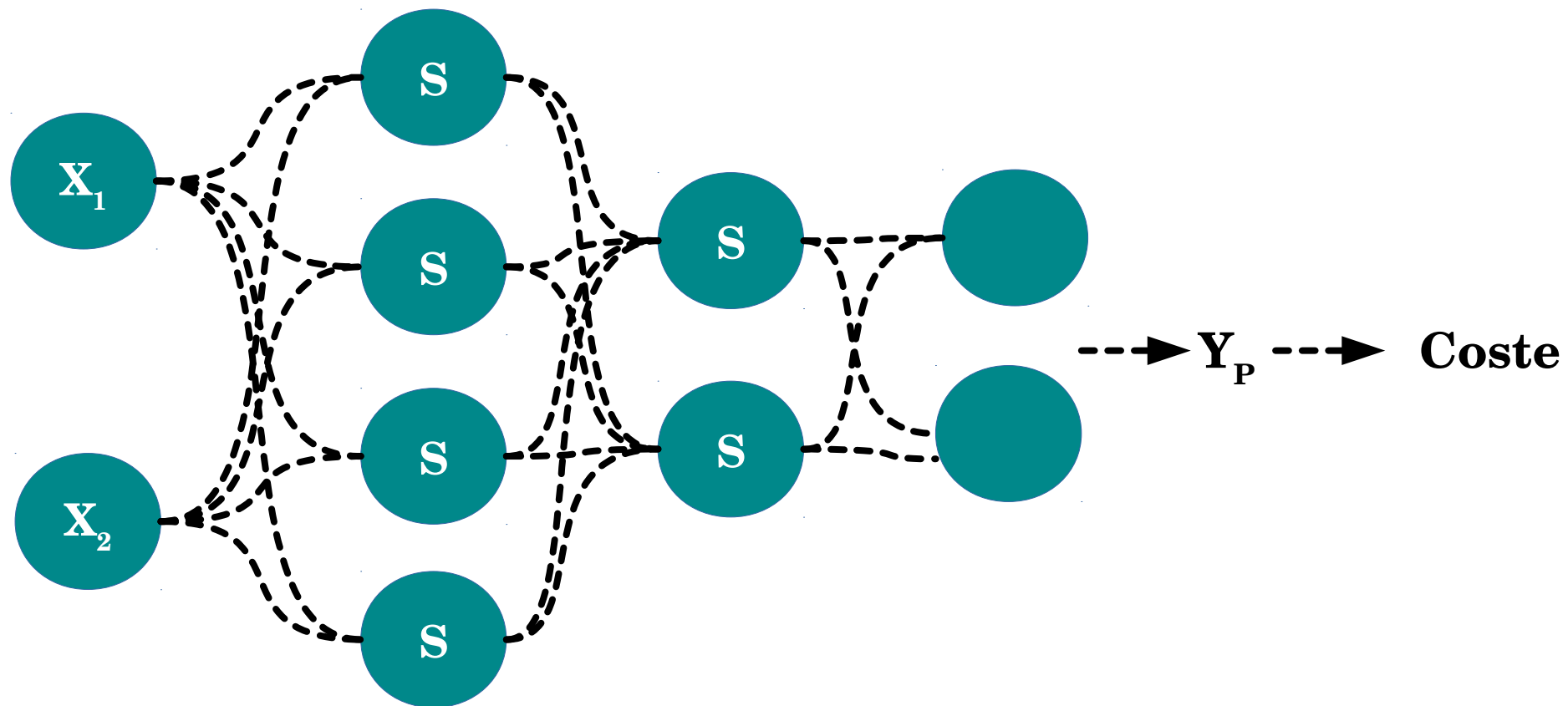
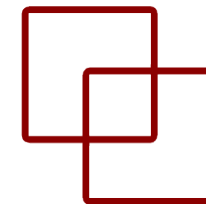
Backpropagation



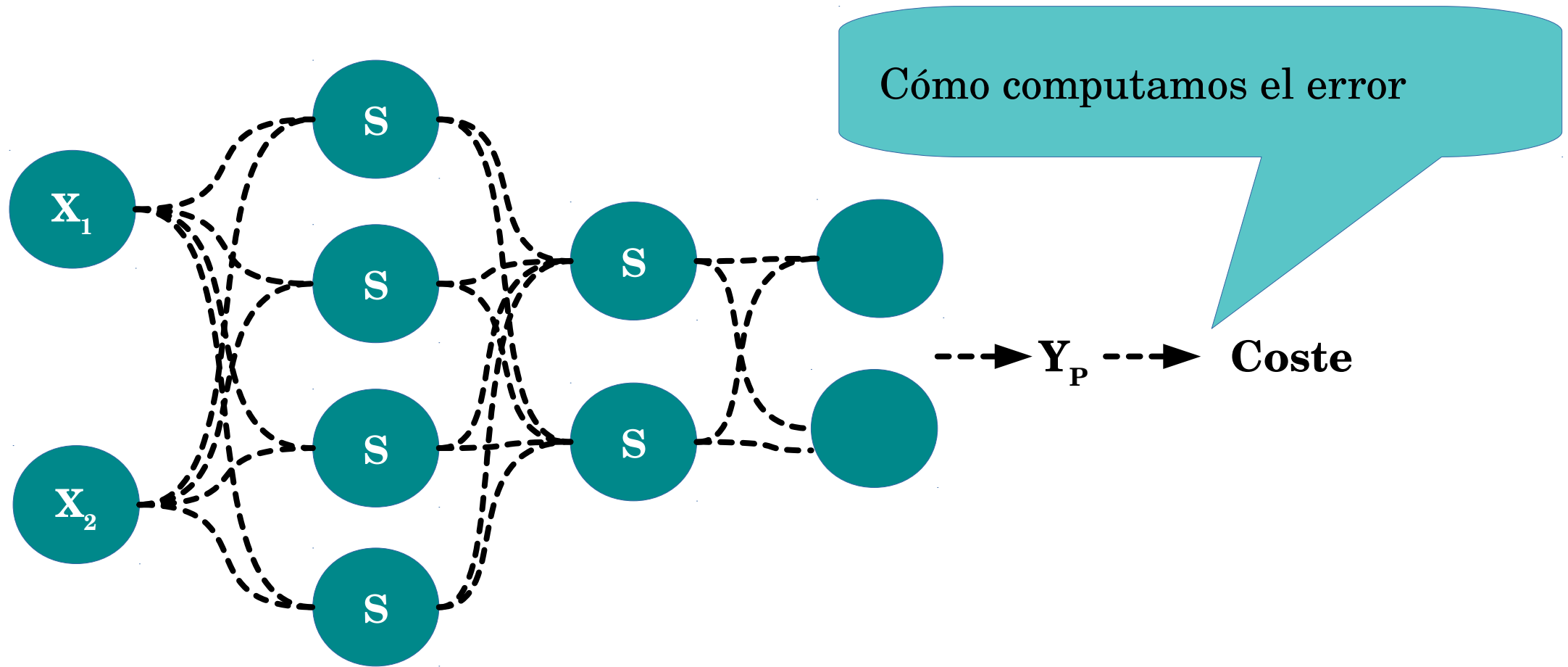
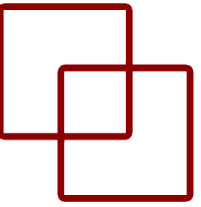
1. Contexto

- En 1958 se publica el artículo de Perceptron, i.e., desarrollo de una única neurona.
 - Pero al poco tiempo muestran diferentes estudios con todas las limitaciones como hemos visto.
 - Se llega al periodo de “**Invierno de la Inteligencia Artificial**” que dura 15 años de inactividad en desarrollo.
 - Básicamente las limitaciones son muchas y se corta la financiación.
- Pero volvemos a la “**Primavera de la IA**” en 1986 con la llegada del artículo que desarrolla algoritmo Backpropagation de David E. Rumelhart et al.
 - Básicamente exponen un algoritmo el cual una red neuronal **ajusta** “automáticamente” sus **parámetros**.
 - Entra en acción el “Gradiente Descendiente” pero lo veremos en la siguiente sesión.

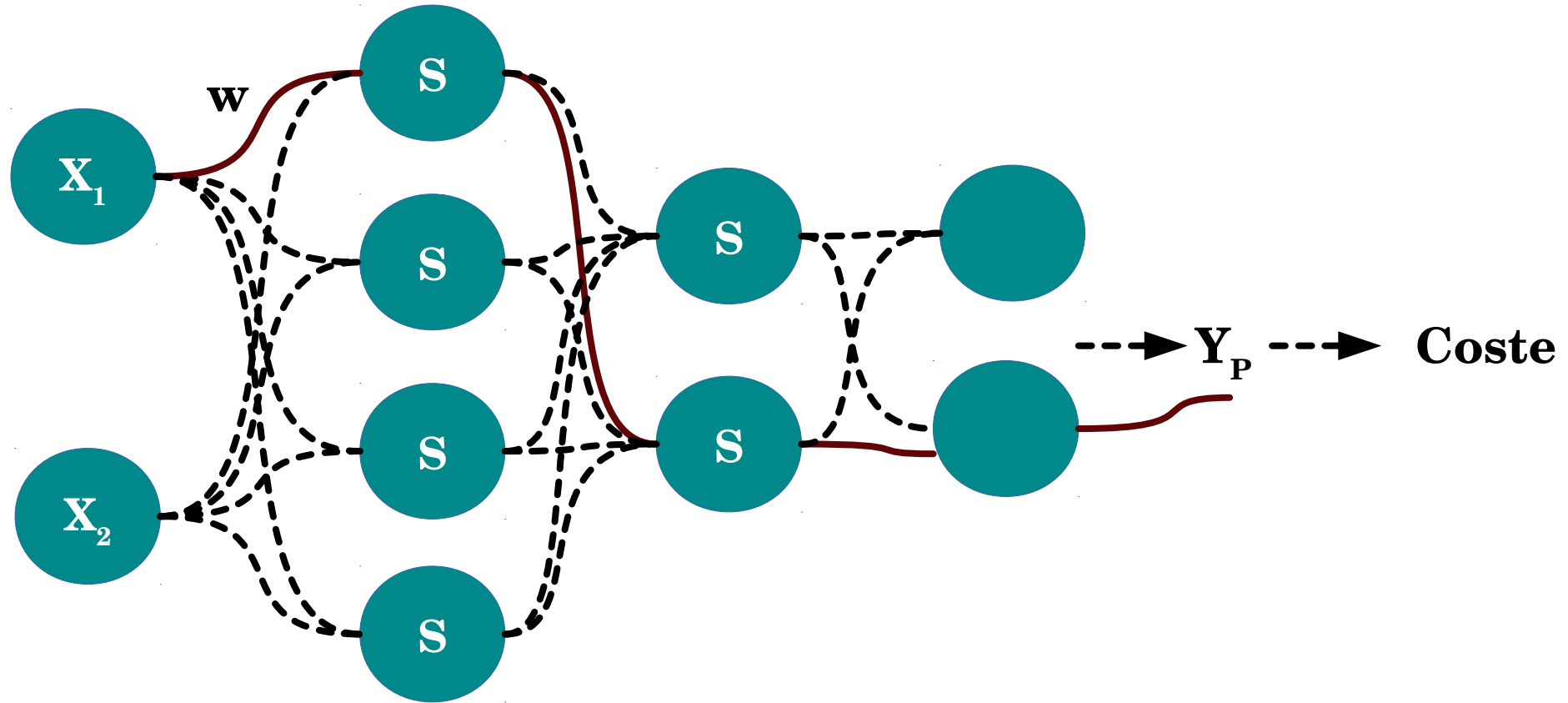
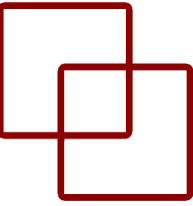
1. Contexto



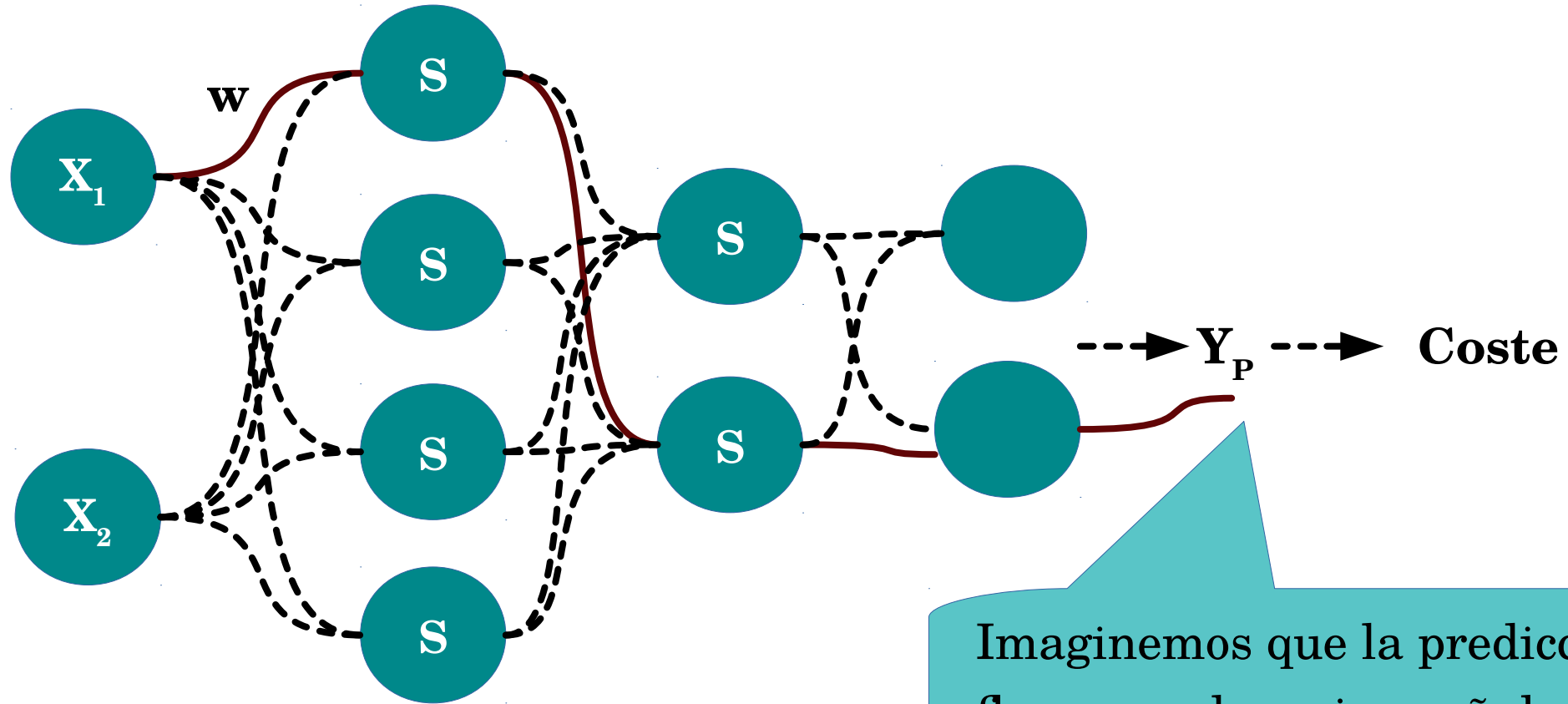
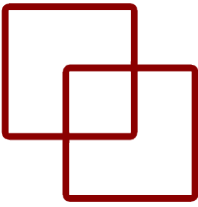
2. Conexiones e iteraciones



2. Conexiones e iteraciones

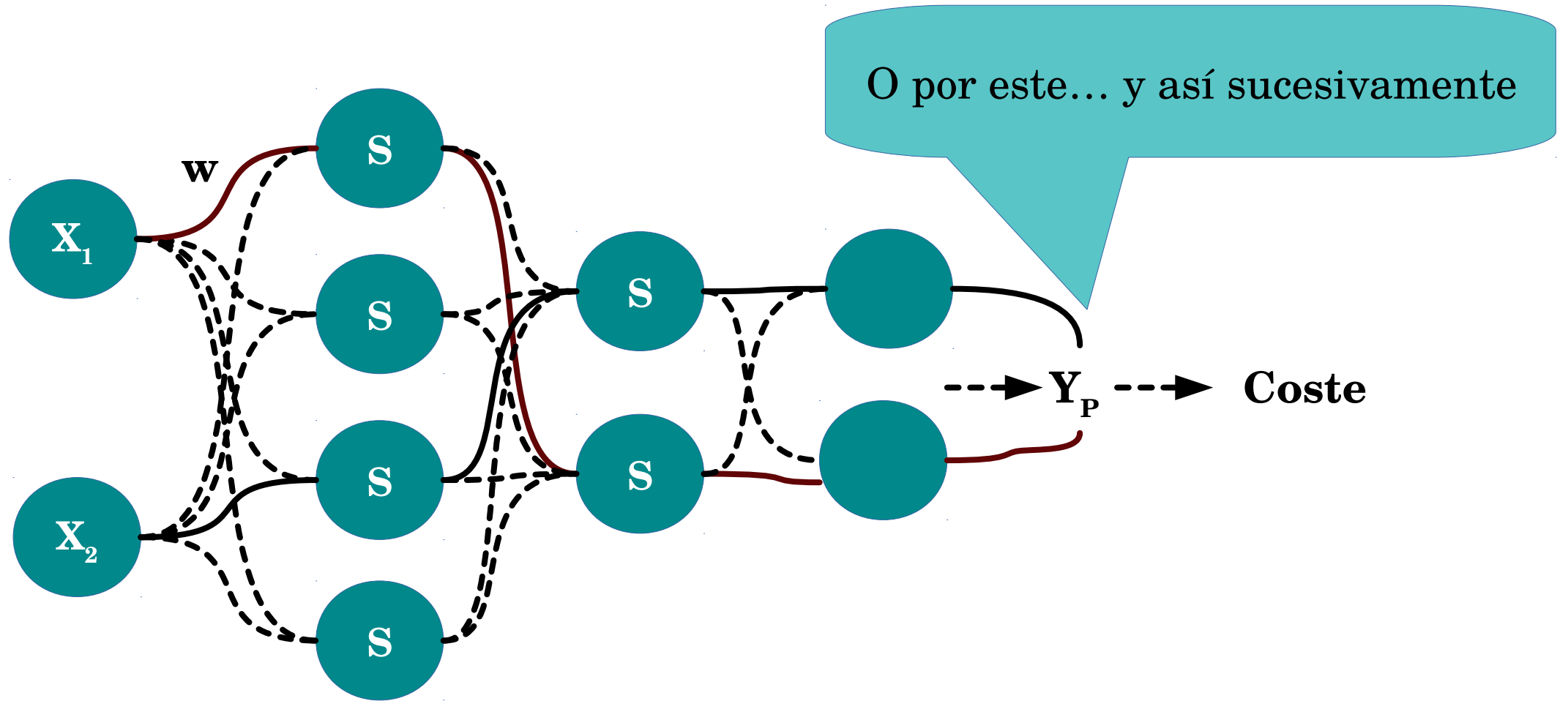
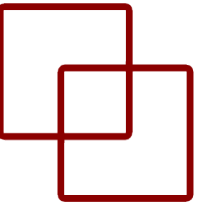


2. Conexiones e iteraciones

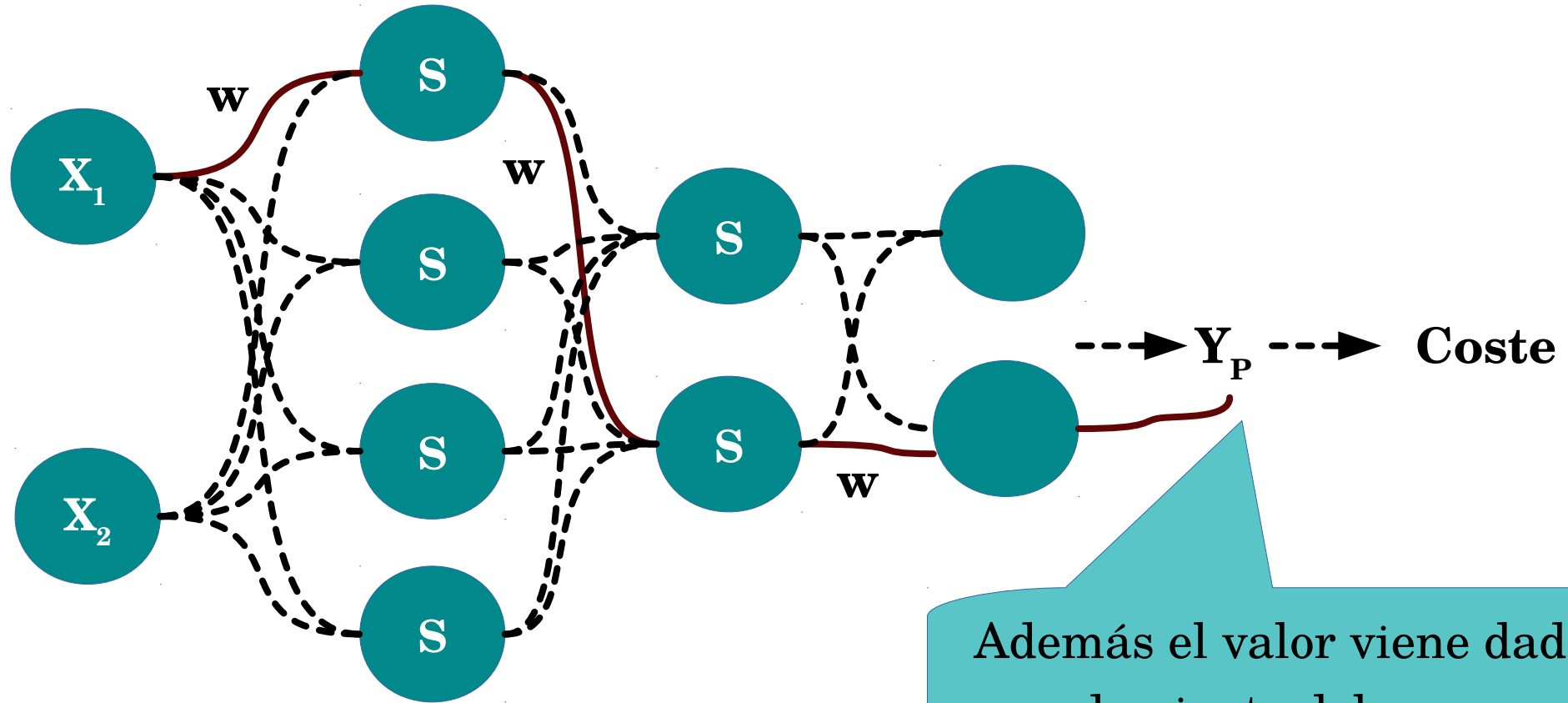
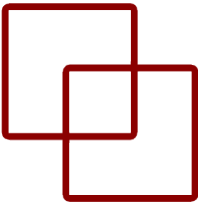


Imaginemos que la predicción fluye por el camino señalado

2. Conexiones e iteraciones

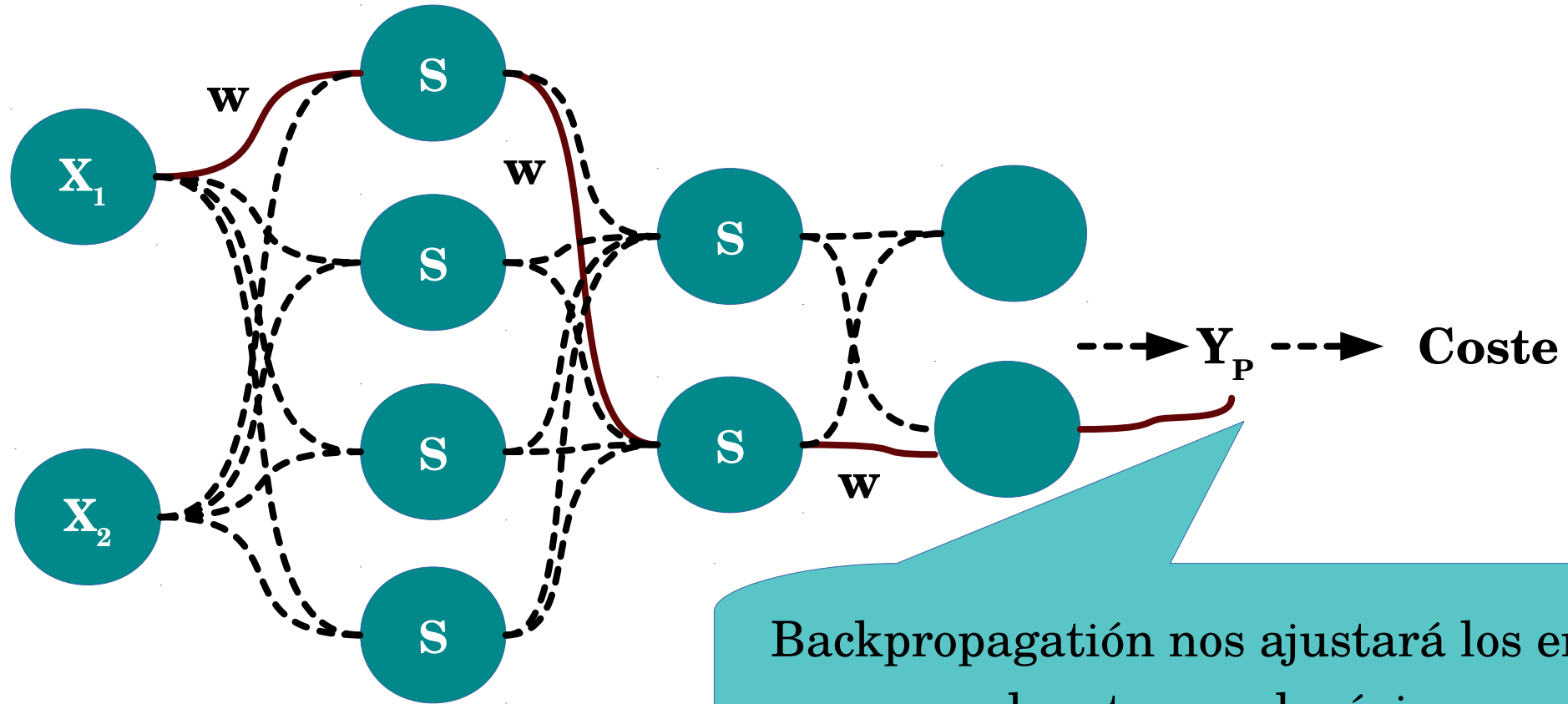
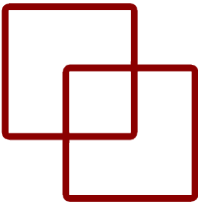


2. Conexiones e iteraciones



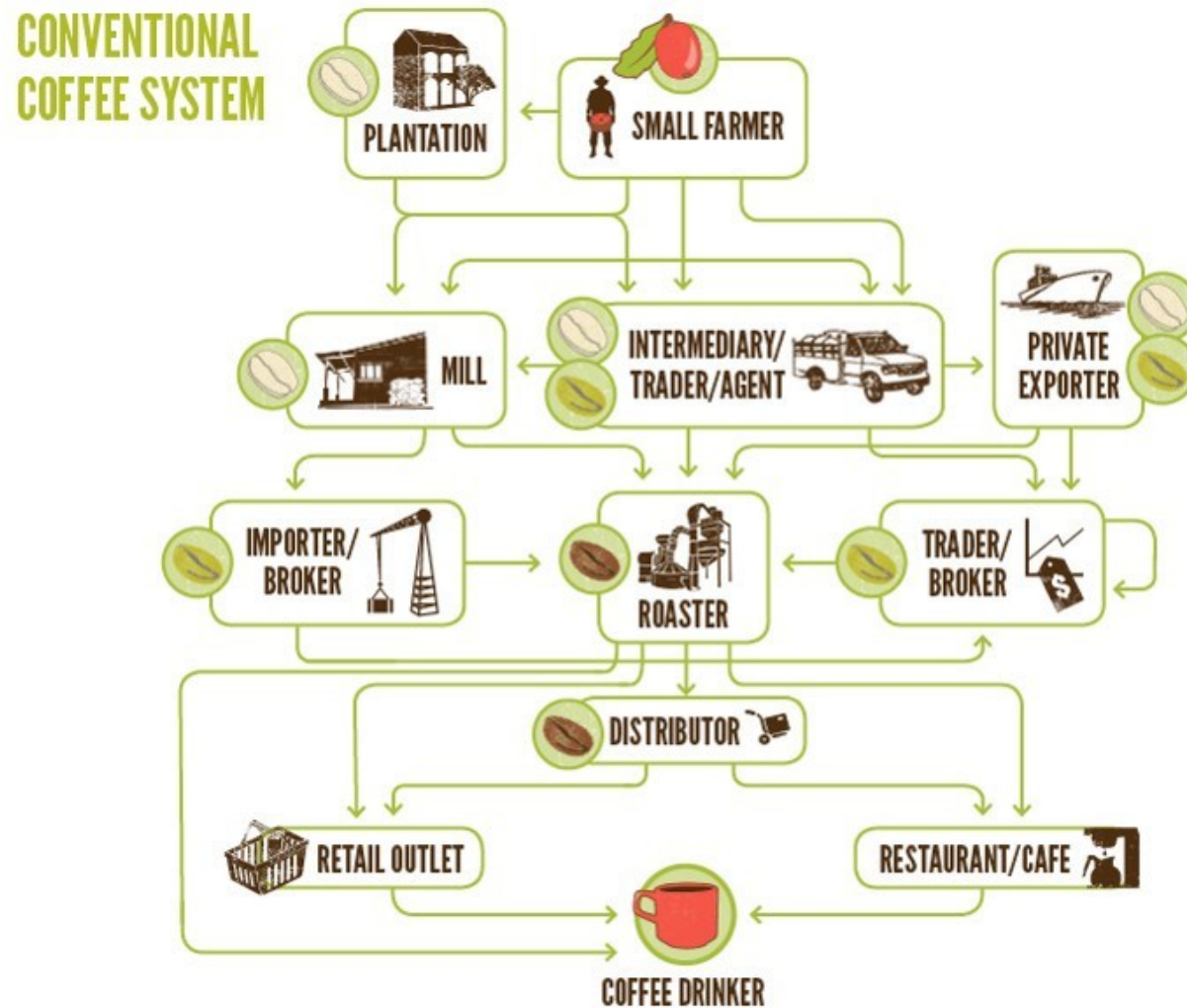
Además el valor viene dado por encadenamiento del error previo

2. Conexiones e iteraciones



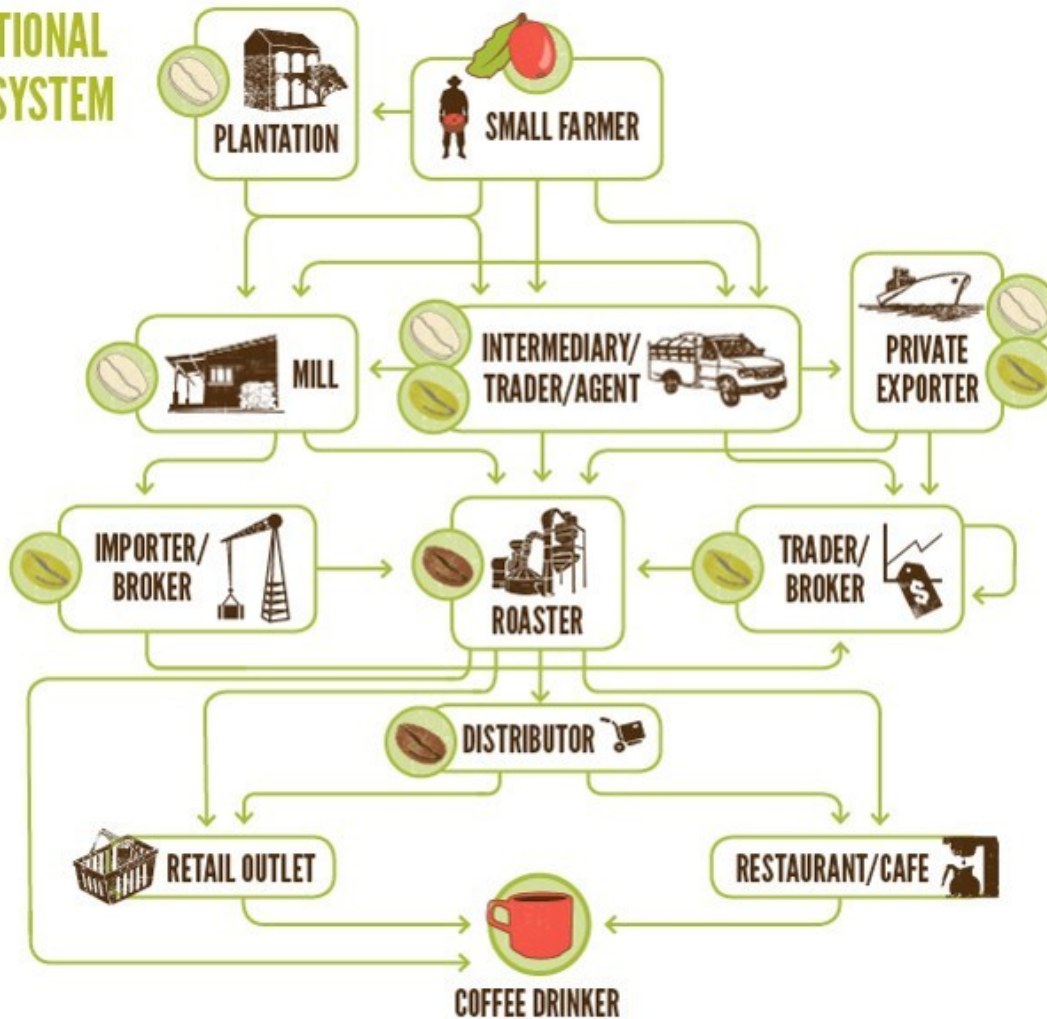
Backpropagación nos ajustará los errores para que el coste sea el mínimo.
Para ello, utilizará el Gradiente Descendiente

3. Pensemos en el día a día

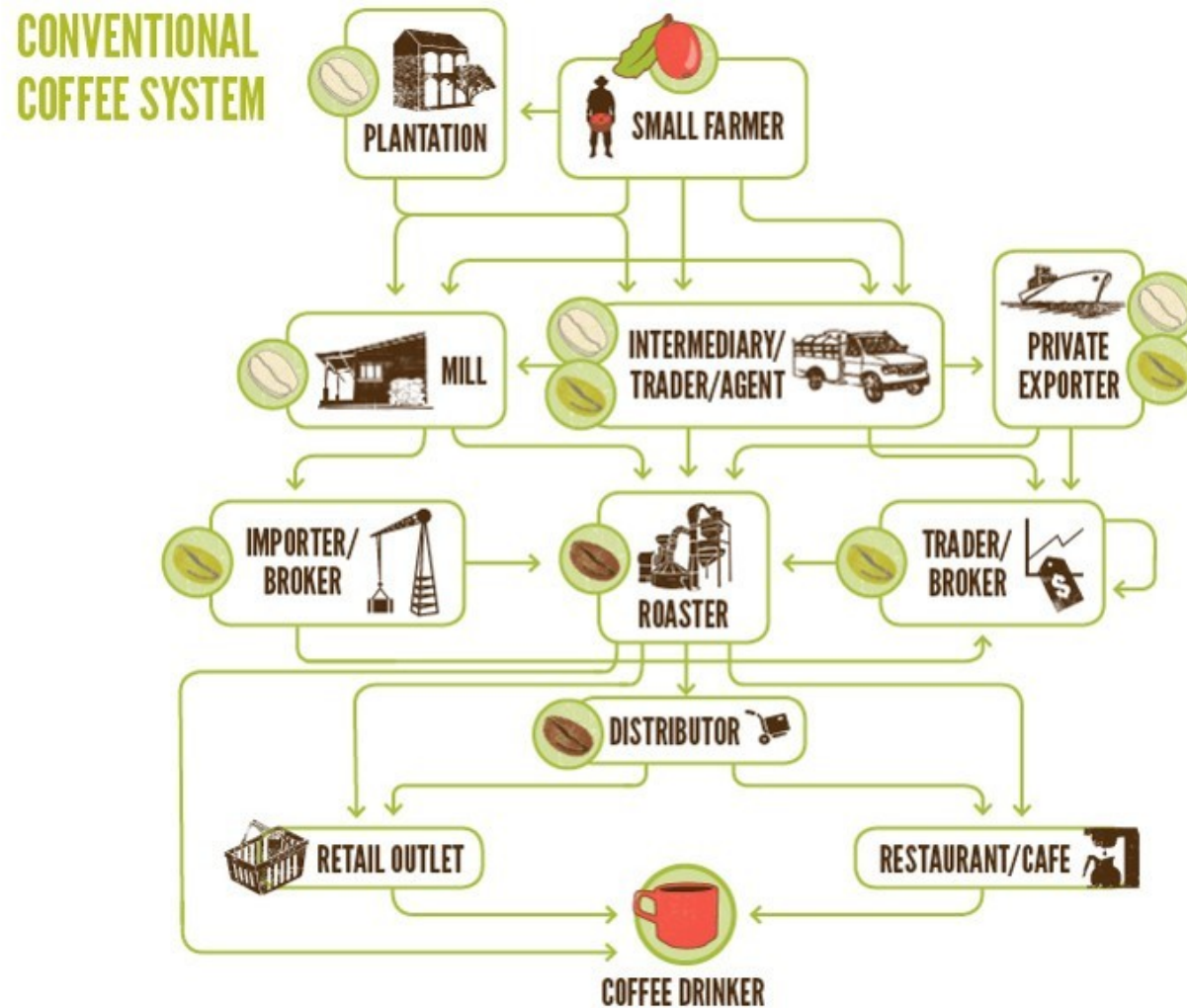


3. Pensemos en el día a día

CONVENTIONAL COFFEE SYSTEM



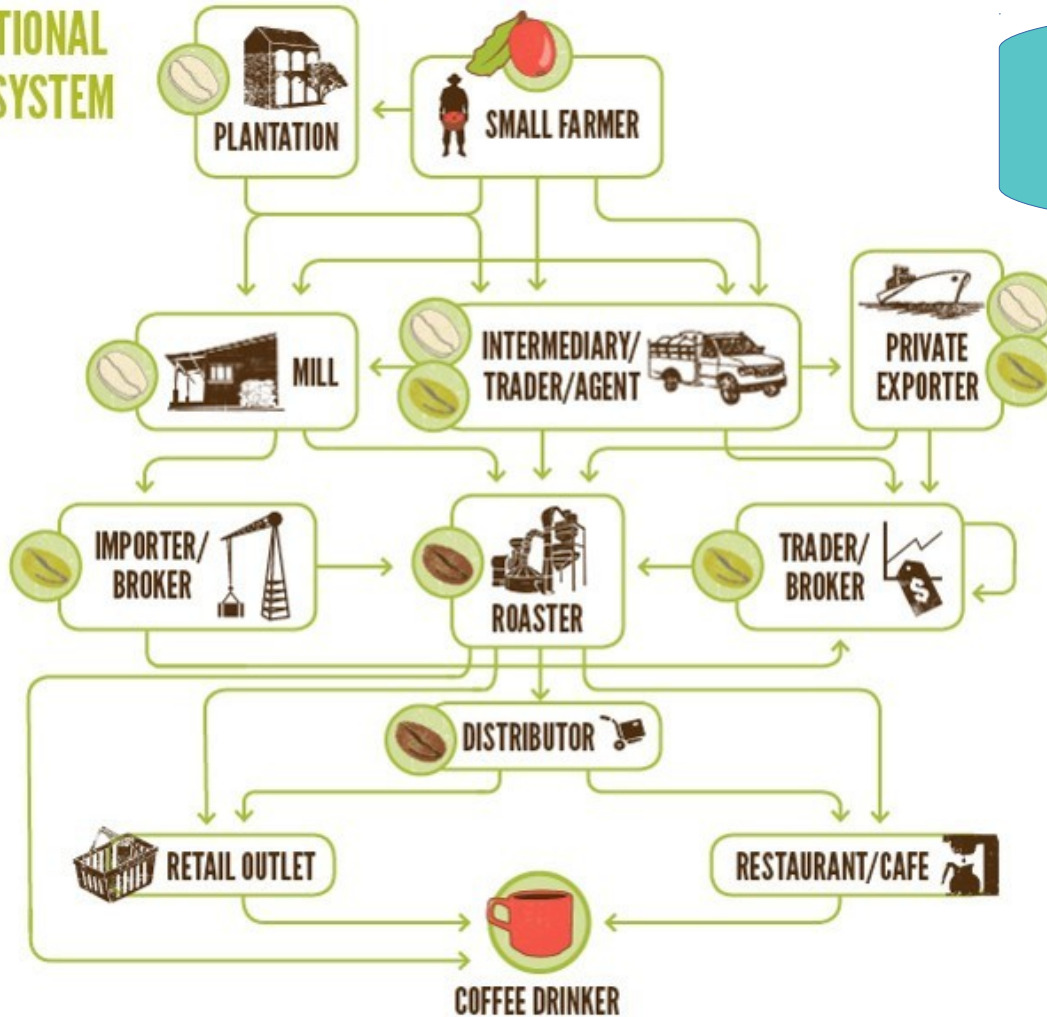
3. Pensemos en el día a día



¿De quién es la culpa?

3. Pensemos en el día a día

CONVENTIONAL COFFEE SYSTEM

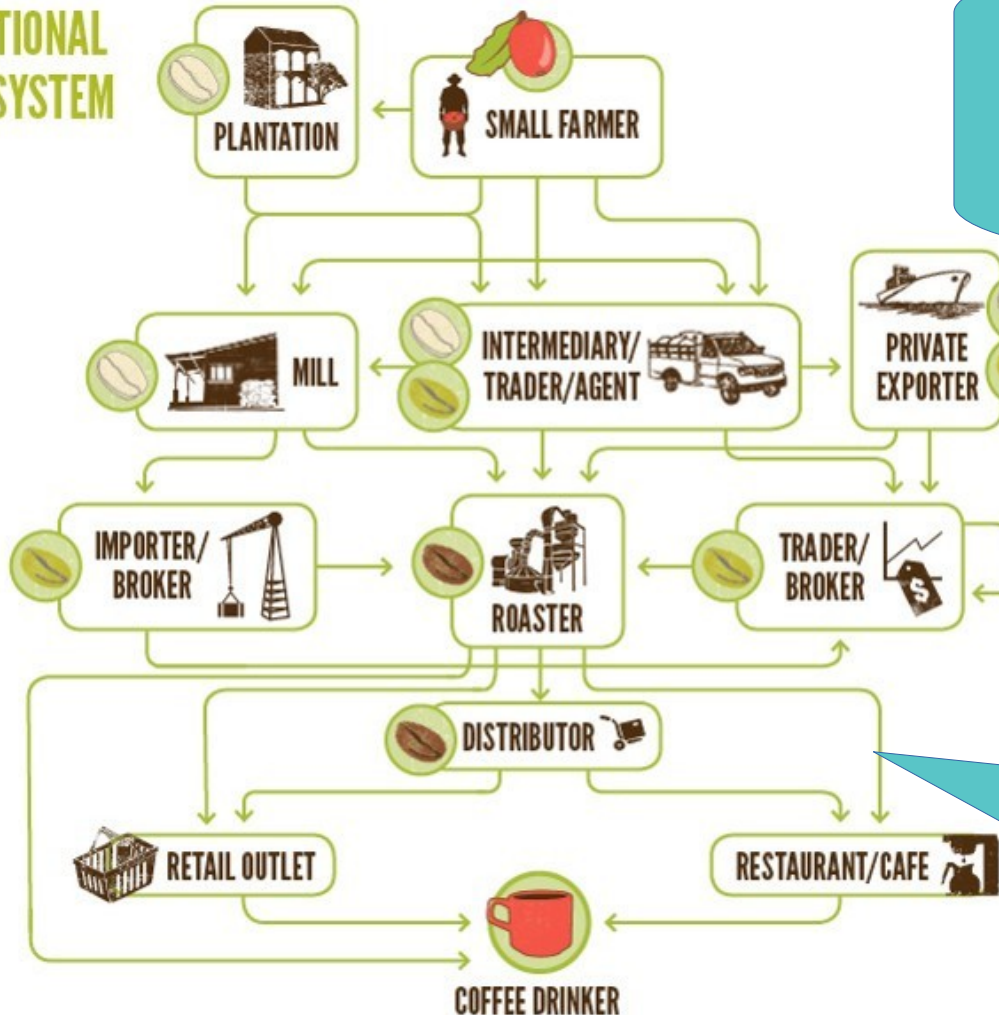


Tenemos que analizar toda la Cadena para ver quién es el culpable



3. Pensemos en el día a día

CONVENTIONAL COFFEE SYSTEM



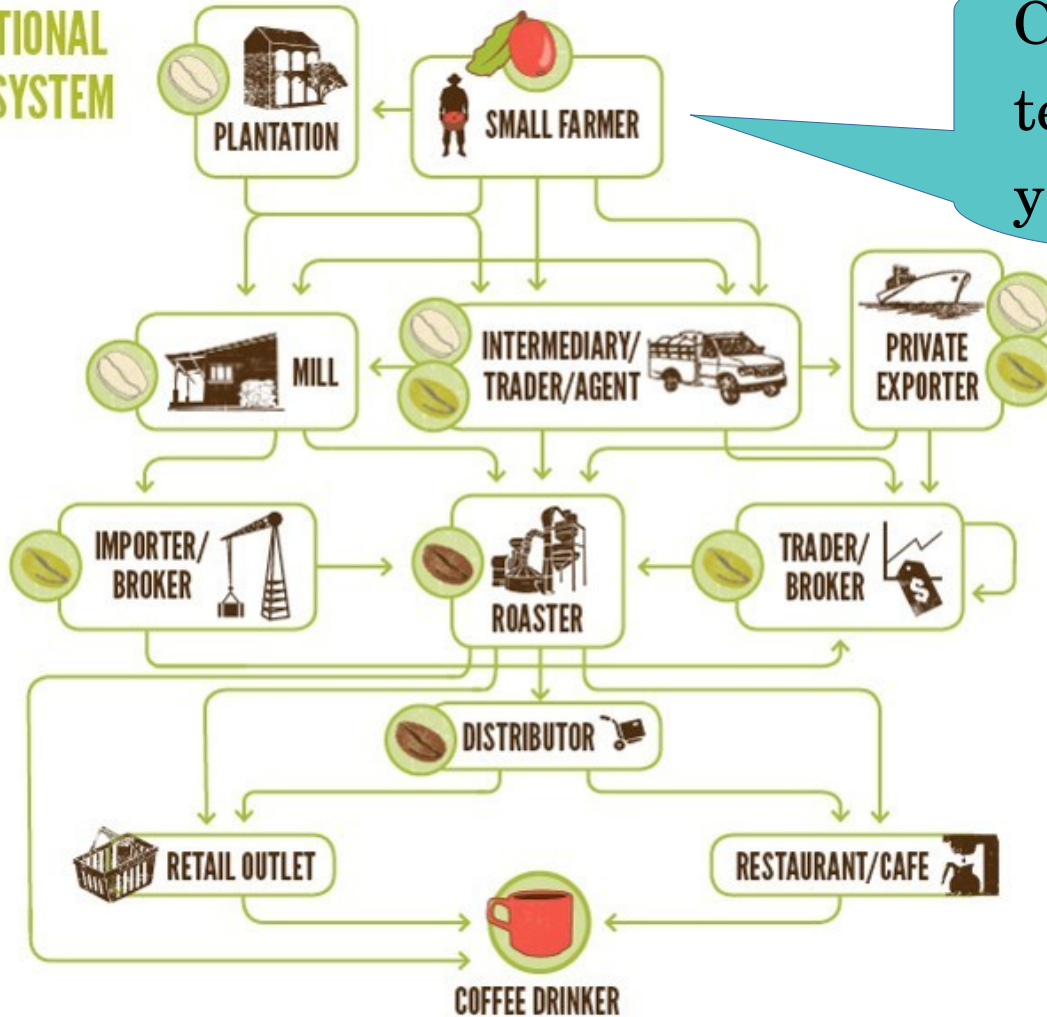
Viendo el porcentaje de implicación de cada neurona en la predicción y, así, corregirlo



Para ello, analizamos el error y vamos analizando hacia atrás para corregirlo

3. Pensemos en el día a día

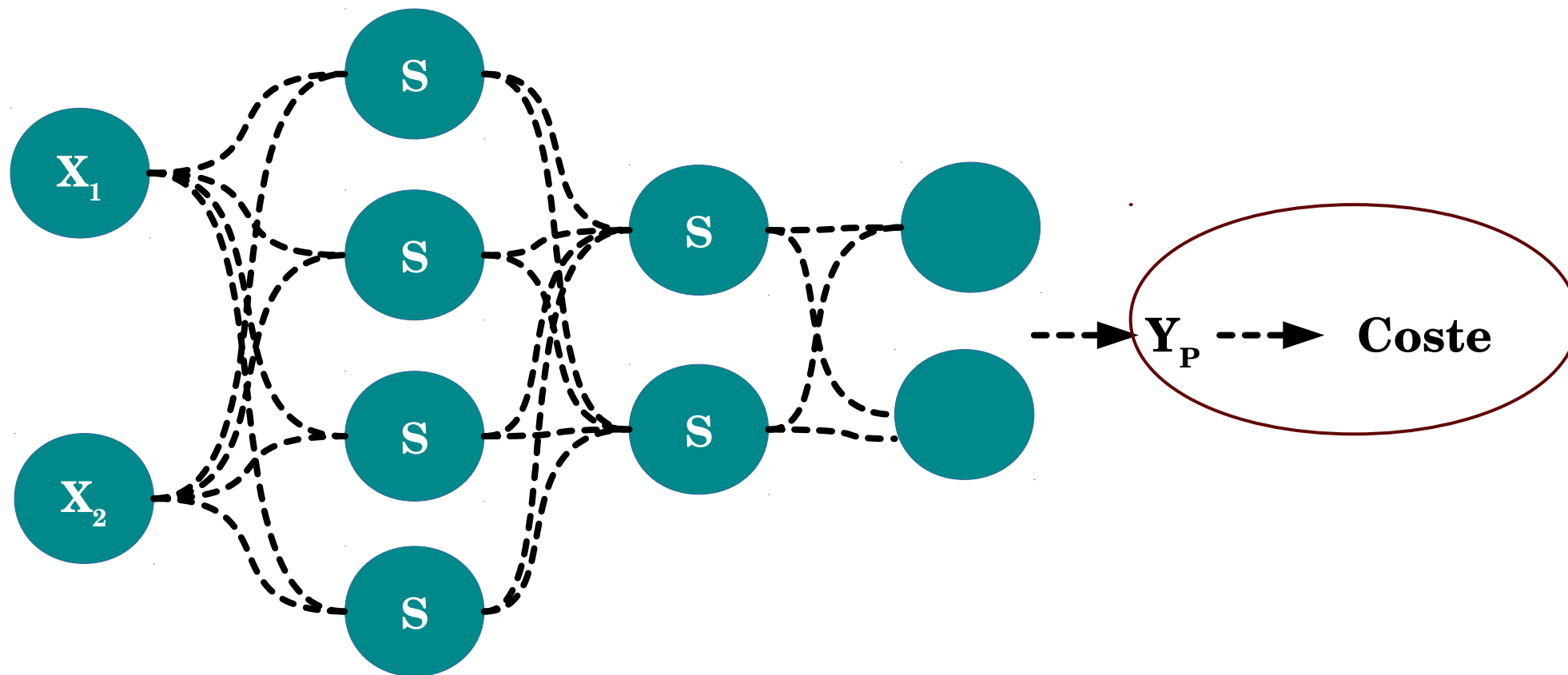
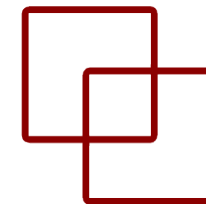
CONVENTIONAL COFFEE SYSTEM



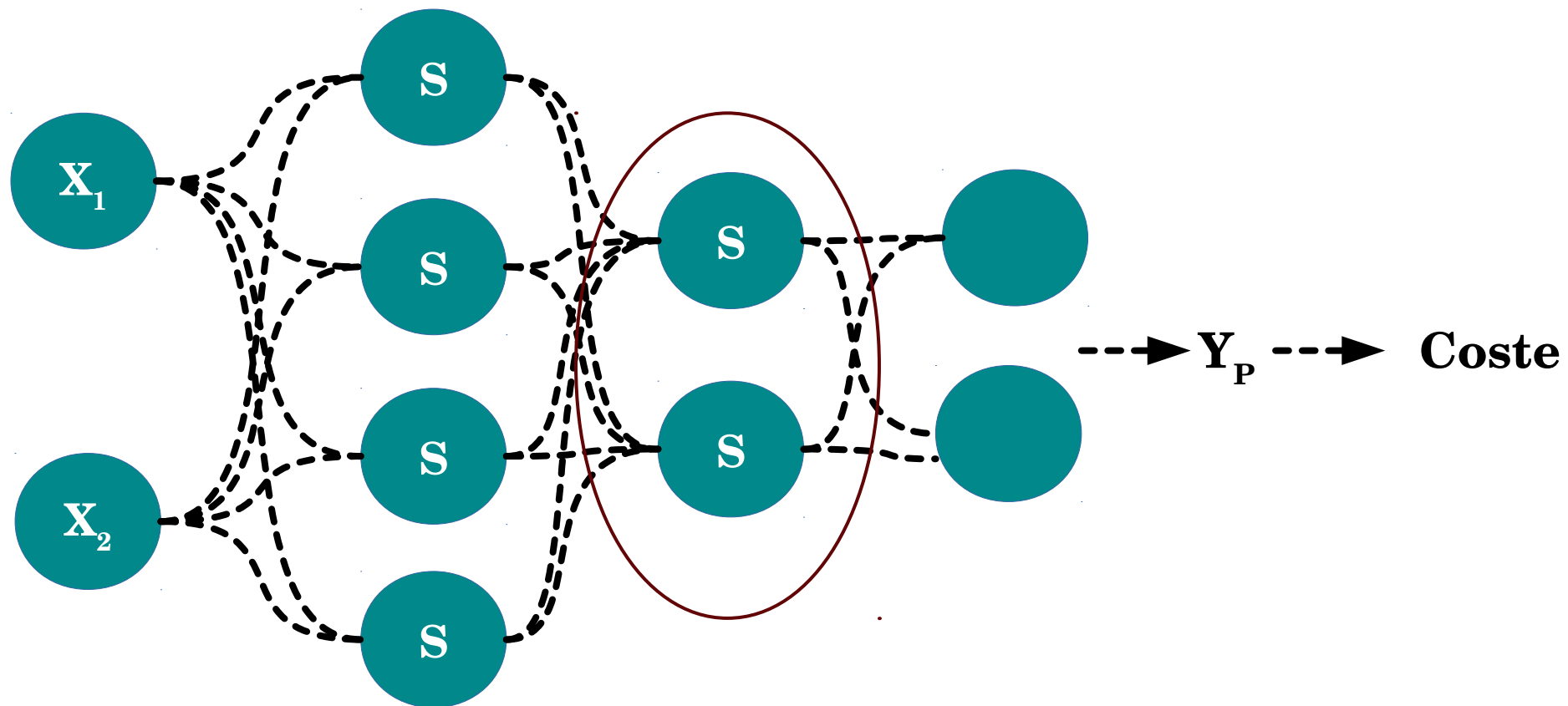
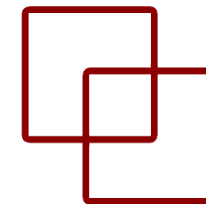
Cuando lleguemos a la primera capa tendremos el error de cada neurona y sus parámetros



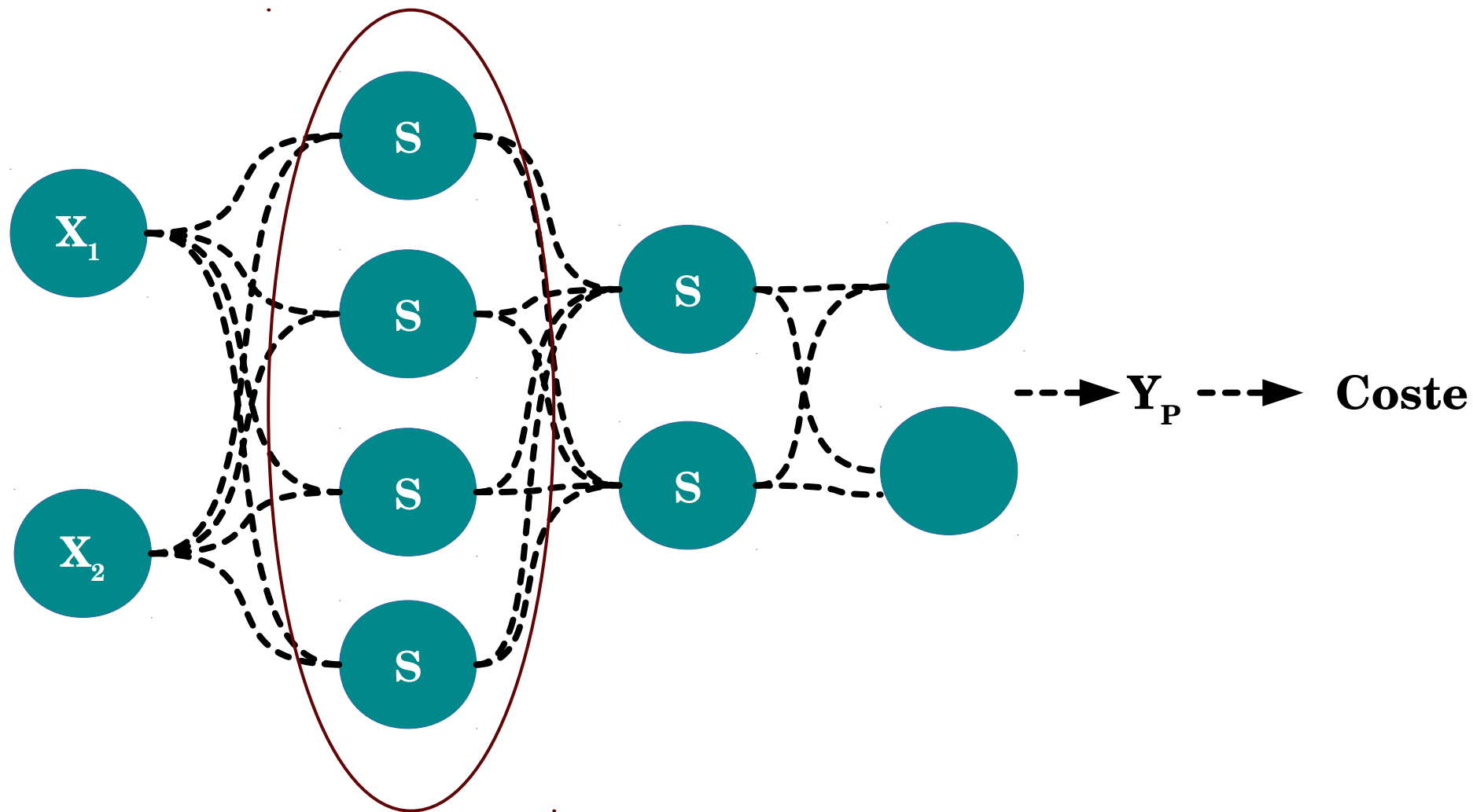
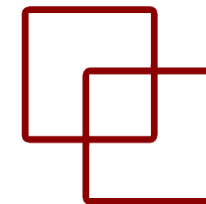
4. En MLP



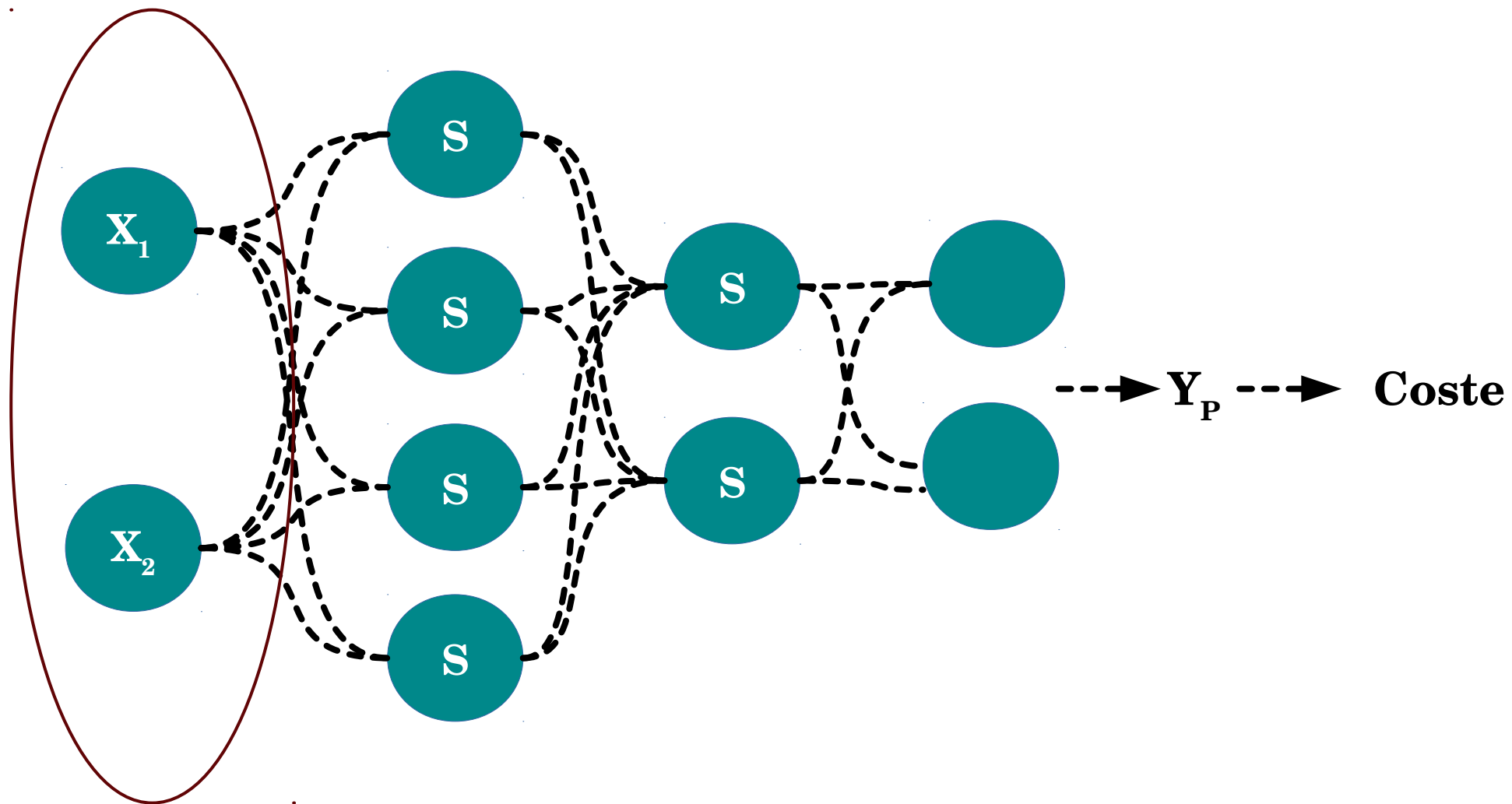
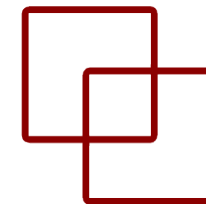
4. En MLP



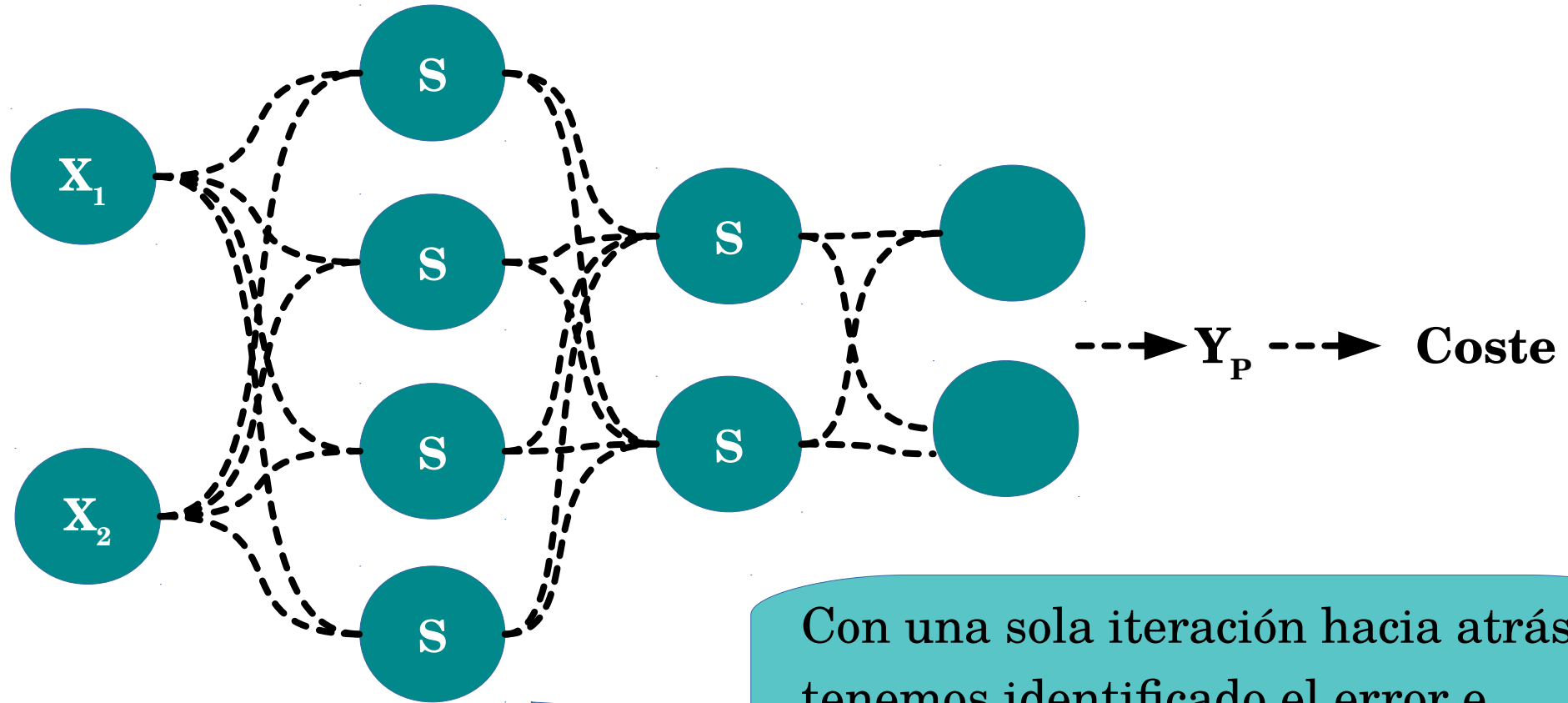
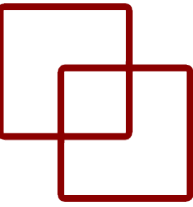
4. En MLP



4. En MLP



4. En MLP



Con una sola iteración hacia atrás
tenemos identificado el error e
implicancia de parámetros y neuronas



Veamos una simulación

¡GRACIAS!



Dr. Manuel Castillo-Cara
Deep Learning con Python y Keras
Web: www.smartcityperu.org